

The Theory of Timed I/O Automata

Dilsun K. Kaynar and Nancy Lynch
MIT Computer Science and Artificial Intelligence Laboratory
{dilsun,lynch@csail.mit.edu}

Roberto Segala
Dipartimento di Informatica, Università di Verona
roberto.segala@univr.it

Frits Vaandrager
Institute for Computing and Information Sciences,
Radboud University Nijmegen
F.Vaandrager@cs.ru.nl

March 2, 2005

Abstract

This monograph presents the *Timed Input/Output Automaton (TIOA)* modeling framework, a basic mathematical framework to support description and analysis of timed systems. An important feature of this model is its support for decomposing timed system descriptions. In particular, the framework includes a notion of *external behavior* for a timed I/O automaton, which captures its discrete interactions with its environment. The framework also defines what it means for one TIOA to *implement* another, based on an inclusion relationship between their external behavior sets, and defines notions of *simulations*, which provide sufficient conditions for demonstrating implementation relationships. The framework includes a *composition* operation for TIOAs, which respects external behavior, and a notion of *receptiveness*, which implies that a TIOA does not block the passage of time.

Contents

1	Introduction	4
1.1	Overview	4
1.2	Evolution of the TIOA framework	6
1.3	Related work	7
1.4	Paper Organization	9
2	Mathematical Preliminaries	9
2.1	Functions and Relations	9
2.2	Sequences	10
2.3	Partial Orders	11
2.4	A Basic Graph Lemma	12
3	Describing Timed System Behavior	12
3.1	Time	12
3.2	Static and Dynamic Types	13
3.3	Trajectories	14
3.3.1	Basic Definitions	14
3.3.2	Prefix Ordering	14
3.3.3	Concatenation	15
3.4	Hybrid Sequences	16
3.4.1	Basic Definitions	16
3.4.2	Prefix Ordering	17
3.4.3	Concatenation	17
3.4.4	Restriction	18
4	Timed Automata	19
4.1	Definition of Timed Automata	19
4.2	Executions and Traces	30
4.3	Special Kinds of Timed Automata	34
4.4	Implementation Relationships	37

4.5	Simulation Relations	37
4.5.1	Forward Simulations	38
4.5.2	Refinements	42
4.5.3	Backward Simulations	43
4.5.4	History Relations	46
4.5.5	Prophecy Relations	49
5	Operations on Timed Automata	51
5.1	Composition	51
5.1.1	Definitions and Basic Results	51
5.1.2	Substitutivity Results	56
5.2	Hiding	60
5.3	Extending Timed Automata with Bounds	61
6	Timed I/O Automata	68
6.1	Definition of Timed I/O Automata	68
6.2	Executions and Traces	69
6.3	Special Kinds of Timed I/O Automata	69
6.3.1	Feasible and I/O Feasible TIOAs	69
6.3.2	Progressive TIOAs	70
6.3.3	Receptive Timed I/O Automata	71
6.4	Implementation Relationships	73
6.5	Simulation Relations	73
7	Operations on Timed I/O Automata	73
7.1	Composition	73
7.1.1	Definitions and Basic Results	73
7.1.2	Substitutivity Results	75
7.1.3	Composition of Special Kinds of TIOAs	84
7.2	Hiding	85
8	Conclusions and Future Work	85
A	Notational Conventions	91

1 Introduction

1.1 Overview

Timed computing systems are systems in which desirable correctness or performance properties of the system depend on the timing of events, not just on the order of their occurrence. A typical timed system consists of computer components, which operate in discrete steps, and timing-related components such as physical or logical clocks, whose behavior involve continuous transformation over time. Timed systems are employed in a wide range of domains including communications, embedded systems, real-time operating systems, and automated control. Many applications involving timed systems have strong safety, reliability and predictability requirements, which makes it important to have methods for systematic design of systems and rigorous analysis of timing-dependent behavior.

Modeling plays a key role in all stages in the design and analysis of systems. Models represent system designs at a level of abstraction that is suitable for isolating and focusing on their most crucial aspects. They can be modified and experimented with more easily than real implementations. Moreover, if the modeling is performed using the concepts provided by a formal framework, the modeling can be done more precisely, and analysis and verification methods supported by that framework can be applied. Timed systems, which combine discrete steps with continuous evolution of state over time, exhibit complex behaviors that are typically hard to describe and analyze in the absence of a carefully-developed modeling framework [11, 41, 42].

A modeling framework must support designing systems in structured ways, viewing them at multiple levels of abstraction and as compositions of interacting components. If a framework is to provide flexibility and generality, it must also support nondeterminism. A system designer might wish to allow several potential behaviors at certain points in the computation of a system, for example, to avoid making assumptions about how the environment will behave, or to allow several correct implementations for the same design. Such liberty in specification would not be possible to accommodate without nondeterminism. In addition to supporting all of these features, modeling frameworks for timed systems must provide mechanisms for representing continuously evolving components such as clocks and timers.

An interesting complication that arises in modeling timed systems is that time can progress in ways that conflict with our intuition about physical time. For example, we may force time to stop entirely to “urge” some discrete action to happen, or schedule infinitely many discrete actions to happen in a finite amount of time. A framework needs to provide concepts that identify the conditions under which a timed system behaves according to our intuitions, that is, the conditions under which time diverges as the system continues to run.

In this work, we introduce a basic mathematical framework – the *Timed Input/Output Automaton* modeling framework – to support description and analysis of timed systems.

In this framework, a system is represented as a *Timed I/O Automaton (TIOA)*, which is a kind of nondeterministic, possibly infinite-state, state machine. The state of a TIOA is described by a valuation of state variables that are internal to the automaton. The state of a TIOA can change in two ways: instantaneously by the occurrence of a *discrete transition*, which is labeled by a discrete action, or according a *trajectory*, which is a function that describes the evolution of the state variables over intervals of time. Trajectories may be continuous or discontinuous functions.

The TIOA framework supports decomposition of system description and analysis. A key to this decomposition is the rigorously-defined notion of *external behavior* for timed I/O automata. The external behavior of each TIOA is defined by a simple mathematical object called a *trace*—essentially, a sequence of actions interspersed with time-passage steps. *Abstraction* and *parallel composition* are other important notions for decomposition of system description and analysis.

For abstraction, the framework includes notions of *implementation* and *simulation*, which can be used to view timed systems at multiple levels of abstraction, starting from a high-level version that describes required properties, and ending with a low-level version that describes a detailed design or implementation. In particular, the TIOA framework defines what it means for one TIOA, $\mathcal{A}$, to *implement* another TIOA, $\mathcal{B}$, namely, any trace that can be exhibited by $\mathcal{A}$ is also allowed by $\mathcal{B}$. In this case, $\mathcal{A}$ might be more deterministic than $\mathcal{B}$, in terms of either discrete transitions or trajectories. For instance, $\mathcal{B}$ might be allowed to perform an output action at an arbitrary time before noon, whereas $\mathcal{A}$ produces the same output sometime between 10 and 11AM. The notion of a *simulation relation* from $\mathcal{A}$ to $\mathcal{B}$ provides a sufficient condition for demonstrating that $\mathcal{A}$ implements $\mathcal{B}$. A simulation relation is defined to satisfy three conditions, one relating start states, one relating discrete transitions, and one relating trajectories of $\mathcal{A}$ and $\mathcal{B}$.

For parallel composition, the framework provides a *composition operation*, by which TIOAs modeling individual timed system components can be combined to produce a model for a larger timed system. The model for the composed system can describe interactions among the components, which involves joint participation in discrete transitions. Composition requires certain “compatibility” conditions, namely, that each output action be controlled by at most one automaton, and that internal actions of one automaton cannot be shared by any other automaton. The composition operation respects traces, for example, if $\mathcal{A}_1$ implements $\mathcal{A}_2$ then the composition of $\mathcal{A}_1$ and $\mathcal{B}$ implements the composition of $\mathcal{A}_2$ and $\mathcal{B}$. Composition also satisfies *projection* and *pasting* results, which are fundamental for compositional design and verification of systems: a trace of a composition of TIOAs “projects” to give traces of the individual TIOAs, and traces of components are “pastable” to give behaviors of the composition.

If a TIOA approaches a finite point in time without quite reaching it, or by scheduling infinitely many discrete actions to happen in a finite amount of time, it is said to exhibit *Zeno behavior*, in reference to Zeno’s paradox [24]. The TIOA framework includes a notion of *receptiveness*, which is used to classify automata that do not contribute to producing

behavior, and which is preserved by composition. Receptiveness of a TIOA, $\mathcal{A}$, in the TIOA framework is defined in terms of the existence of a strategy, which is defined as a subautomaton of $\mathcal{A}$ that chooses some of the evolutions from each state of $\mathcal{A}$.

The TIOA framework presented in this work is purely mathematical. However, it constitutes a natural basis for computer support tools, which are currently under development [19].

1.2 Evolution of the TIOA framework

The TIOA modeling framework presented in this work has evolved from the *Hybrid Input/Output Automaton (HIOA)* modeling framework for hybrid systems [25] by Lynch, Segala and Vaandrager. Our approach is based on the assumption that a timed system can be viewed as a special kind of a hybrid system where the continuous transformation is limited to internal system components that determine the timing of events. Therefore, we define a TIOA as a restricted HIOA where the only essential difference between an HIOA and a TIOA is that an HIOA may have *external variables* to model the continuous information flowing into and out of the system, in addition to state variables. A major consequence of this definition is that the communication between TIOAs is restricted to shared-action communication only. The TIOA model does not impose any further restrictions on the expressive power of the HIOA model.

We have undertaken the project of developing this new modeling framework even though there are several timed automaton models that extend the basic I/O automaton model [32, 40, 30, 28], because we have observed that the new HIOA modeling framework of Lynch, Segala and Vaandrager offered a way of improving and simplifying previous work on timed I/O automaton models [40, 30, 28]. For example, the use of trajectories as first-class objects to represent the external behavior of a timed automaton, the definition of a strategy as an automaton rather than a two-player game, and the variable structure on states are all new features that were motivated by what we learned in developing the HIOA framework and that gave rise to more elegant definitions and simpler proofs for timed automata.

We intend the TIOA model to serve as a general semantic framework in which previous results for timed I/O automata [30, 32, 40, 28] and other related models [5, 31, 35, 9] can be re-cast in a style that is upwardly compatible with the new HIOA model. Limiting the communication to discrete interactions is an apt choice since the previous timed I/O automaton models also adopt this type of communication. On the other hand, by avoiding any further restrictions on the general hybrid model, we obtain an expressive model suitable for specifying complex timing behavior. For example, our model does not require variables to be either discrete or to evolve at the same rate as real-time as in some other models [5, 35]. Consequently, algorithms such as clock synchronization algorithms that use local clocks evolving at different and varying rates can be formalized naturally in our framework.

The fact that HIOAs subsume TIOAs as a special class does not eliminate the need for having a separate modeling framework for timed systems. First, having no external variables in the TIOA model gives rise to considerable simplifications in the theory. For example, proving that the composition of two timed automata is a well-defined automaton becomes simpler in the absence of external variables; no extra compatibility conditions as in the general HIOA framework are needed to obtain the desirable composition theorems for TIOAs.

Second, we believe that focusing on the TIOA model presented in this paper is compatible with our longer-term goal of developing a unified I/O automaton model that can address timing-dependent, probabilistic and general hybrid behavior in a common framework. We are planning to start out with a probabilistic model with discrete interactions only, and then extend the model to handle timing-dependent behavior, and only at later stages consider continuous interactions. It would be harder to integrate probabilistic mechanisms into the full hybrid model than it would be to integrate them into the TIOA model presented here.

1.3 Related work

There are several formalisms and tools for timed systems that are based on automata and state transition models. In this section, we briefly introduce those lines of work that we think are most closely related to ours. Note that we do not focus on the toolsets and their capabilities, but rather on the underlying formal models and languages.

One of the widely-used formal frameworks for timed systems is that of Alur-Dill timed automata [5, 3]. An Alur-Dill automaton is a finite directed multigraph augmented with a finite set of clock variables. The semantics of such a timed automaton are defined as a state transition system in which each state consists of a location and a clock valuation. Clocks are assumed to change with the same rate as real-time, that is with rate 1. Timed automata accept timed languages consisting of sequences of events tagged with their occurrence times. Decision problems such as universality and language inclusion are undecidable for timed automata. Recently, a version of timed automata called perturbed automata has been presented [7]. The clocks in perturbed timed automata can change at a rate within the interval $[1 - \epsilon, 1 + \epsilon]$, where ϵ is a given perturbation error. It has been shown that the language inclusion problem is decidable for systems modeled as products of perturbed automata each of which has a single clock.

The aim of facilitating automated verification seems to have motivated the restrictions on the expressive power of the model. The timed automaton model presented in this work is more expressive than the model of Alur-Dill automata. In our model, there are no finiteness assumptions and no restrictions imposed on the dynamic types of variables. Alur-Dill timed automata have been extensively studied with a formal language theoretic-view [6]. Our focus, on the other hand, has been to develop a general formal framework

with a well-defined notion of external behavior, parallel composition and abstraction that supports reasoning with simulation relations.

Uppaal [35, 23] is a widely-used modeling and verification tool for timed systems. It supports the description of systems as a network of Alur-Dill timed automata and enhances that model with CCS-style communication [33] along with other notions such as committed and urgent locations. Uppaal also supports communication via shared variables. Uppaal has a sophisticated model-checker that explores the whole state space of the modeled system to verify timing properties. Therefore, finiteness assumptions are built into the model to make such verification possible and the operations on clocks are restricted. Uppaal can be used as a model-checker for restricted TIOAs. We have done some preliminary work in this direction [39].

It would be interesting to work on formal semantics for Uppaal based on some variation of our restricted hybrid I/O automaton model. There are several small mismatches due to the style of communication and notions such as committed locations. It remains to be seen to what extent we can use the communication mechanisms of our automata to model these formally. We could, for example, allow a non-empty set of external variables with restricted dynamic types and seek restrictions on the use of shared variables in Uppaal, which would allow us to view these variables as external variables in the HIOA sense.

Kronos [46] is another verification tool for timed systems that uses Alur-Dill automata. This tool requires systems to be represented as timed automata and the correctness conditions to be expressed in the real-time temporal logic TCTL. Kronos, as Uppaal, can perform model-checking using a symbolic representation of the infinite state space by sets of linear constraints. It would be possible to use Kronos as a model-checker for restricted TIOAs.

The IF notation, which is the intermediate representation used in the IF toolset [8], is based on Alur-Dill automata extended with discrete data variables, communication primitives, dynamic process creation and destruction. This notation has been designed such that it can serve as a target for the translation of higher-level modeling languages, such as real-time extensions of SDL and UML. The support for dynamic process creation and destruction appears to be a distinguishing feature of the IF notation.

A slight generalization of Alur-Dill timed automata are the linear hybrid automata of [4]. In this model, apart from clocks that progress with rate 1, one can also use continuous variables whose derivatives are contained in some arbitrary interval. A well-known model checking tool for linear hybrid automata is HyTech [16], which uses symbolic manipulation techniques as in Uppaal and Kronos. The input language of HyTech can be translated into our TIOA model, to apply TIOA verification methods. Likewise, TIOAs whose continuous variables conform to the linearity conditions of HyTech could be verified using model-checking capabilities of HyTech.

The timed I/O automaton modeling framework presented in this paper can be used to express models that use lower and upper time bounds on tasks or actions [32, 31].

Our framework includes an operation for adding time bounds on a subset of the actions of a timed automaton. As a result of this operation, lower bounds are transformed to appropriate preconditions for transitions and upper bounds are transformed to stopping conditions for trajectories.

An interesting timed automaton model called “Clock GTA ” has been introduced in [9]. The model was used for describing algorithms that behave in accordance with their timing constraints in certain intervals but may exhibit timing failures for some other intervals. The possibility of expressing such an ability turns out to be crucial for performance and fault-tolerance analysis for practical algorithms [9, 29]. We are interested in finding a systematic way of describing such behavior with our new timed I/O automaton model.

1.4 Paper Organization

The rest of this paper is organized as follows. Section 2 contains mathematical preliminaries. Section 3 defines notions that are useful for describing the behavior of timed systems, most importantly, trajectories and timed sequences. Section 4 defines *timed automata (TAs)*, which contain all of the structure of TIOAs except for the classification of external actions as inputs or outputs. It also defines external behavior for TAs and implementation and simulation relationships between TAs. Section 5 presents composition and hiding operations for TAs, along with operations for adding bounds that relate TIOAs to other timed automaton models. Section 6 defines *timed I/O automata (TIOAs)* by adding an input/output classification to TAs, and extends the theory of TAs to TIOAs. It also defines special kinds of TIOAs such as progressive and receptive TIOAs. Section 7 presents compositionality results for TIOAs in general, and for the special classes of progressive and receptive TIOAs. Examples are included throughout.

2 Mathematical Preliminaries

In this section, we give basic mathematical definitions and notation that will be used as a foundation for our definitions of timed automata and timed I/O automata. These definitions involve functions, sequences, partial orders, and untimed automata.

2.1 Functions and Relations

If f is a function, then we denote the domain and range of f by $\text{dom}(f)$ and $\text{range}(f)$, respectively. If also S is a set, then we write $f \upharpoonright S$ for the restriction of f to S , that is, the function g with $\text{dom}(g) = \text{dom}(f) \cap S$ such that $g(c) = f(c)$ for each $c \in \text{dom}(g)$.

We say that two functions f and g are *compatible* if $f \upharpoonright \text{dom}(g) = g \upharpoonright \text{dom}(f)$. If f and g are compatible functions then we write $f \cup g$ for the unique function h with

$\text{dom}(h) = \text{dom}(f) \cup \text{dom}(g)$ satisfying the condition: for each $c \in \text{dom}(h)$, if $c \in \text{dom}(f)$ then $h(c) = f(c)$ and if $c \in \text{dom}(g)$ then $h(c) = g(c)$. More generally, if F is a set of pairwise compatible functions then we write $\bigcup F$ for the unique function h with $\text{dom}(h) = \bigcup \{\text{dom}(f) \mid f \in F\}$ satisfying the condition: for each $f \in F$ and $c \in \text{dom}(f)$, $h(c) = f(c)$.

If f is a function whose range is a set of functions and S is a set, then we write $f \downarrow S$ for the function g with $\text{dom}(g) = \text{dom}(f)$ such that $g(c) = f(c) \upharpoonright S$ for each $c \in \text{dom}(g)$. The restriction operation $\downarrow$ is extended to sets of functions by pointwise extension. Also, if f is a function whose range is a set of functions, all of which have a particular element d in their domain, then we write $f \downarrow d$ for the function g with $\text{dom}(g) = \text{dom}(f)$ such that $g(c) = f(c)(d)$ for each $c \in \text{dom}(g)$.

We say that two functions f and g whose ranges are sets of functions are *pointwise compatible* if for each $c \in \text{dom}(f) \cap \text{dom}(g)$, $f(c)$ and $g(c)$ are compatible. If f and g have the same domain and are pointwise compatible, then we denote by $f \dot{\cup} g$ the function h with $\text{dom}(h) = \text{dom}(f)$ such that $h(c) = f(c) \cup g(c)$ for each c .

A relation over sets X and Y is defined to be any subset of $X \times Y$. If R is a relation, then we denote the domain and range of R by $\text{dom}(R)$ and $\text{range}(R)$, respectively. A relation over X and Y is *total* over X if $\text{dom}(R) = X$. We say that a relation R over X and Y is *image-finite* if for each $x \in X$, $R(x)$ is finite.

2.2 Sequences

Let S be any set. A *sequence* over S is a function from a downward-closed subset of $\mathbb{Z}^{>0}$ to S . Thus, the domain of a sequence is either the set of all positive integers, or is of the form $\{1, \dots, k\}$ for some k . In the first case we say that the sequence is infinite, and in the second case finite. We use $|\sigma|$ to denote the cardinality of $\text{dom}(\sigma)$. The sets of finite and infinite sequences over S are denoted by S^* and S^ω , respectively. Concatenation of a finite sequence ρ with a finite or infinite sequence σ is denoted by $\rho \frown \sigma$. The empty sequence, that is, the sequence with the empty domain is denoted by λ . The sequence containing one element $c \in S$ is abbreviated as c . We say that a sequence σ is a *prefix* of a sequence ρ , denoted by $\sigma \leq \rho$, if $\sigma = \rho \upharpoonright \text{dom}(\sigma)$. Thus, $\sigma \leq \rho$ if either $\sigma = \rho$, or σ is finite and $\rho = \sigma \frown \sigma'$ for some sequence σ' . If σ is a nonempty sequence then $\text{head}(\sigma)$ denotes the first element of σ and $\text{tail}(\sigma)$ denotes σ with its first element removed. Moreover, if σ is finite, then $\text{last}(\sigma)$ denotes the last element of σ and $\text{init}(\sigma)$ denotes σ with its last element removed. Let σ and σ' be sequences over S . Then σ' is a subsequence of σ provided that there exists a monotone increasing function $f : \text{dom}(\sigma') \rightarrow \text{dom}(\sigma)$ such that $\sigma'(i) = \sigma(f(i))$ for all $i \in \text{dom}(\sigma')$. If $1 \leq j_1 \leq j_2 \leq |\sigma|$, then we define $\sigma(j_1 \dots j_2)$ to be the subsequence of σ obtained by extracting the elements in positions $j_1, \dots, j_2$; that is, σ' is the subsequence obtained from function f of length $j_2 - j_1 + 1$, where $f(i) = i + j_1 - 1$ for all $i \in \text{dom}(\sigma')$.

2.3 Partial Orders

We recall some basic definitions and results regarding partial orders, and in particular, complete partial orders (cpo) from [14, 15]. A *partial order* is a set S together with a binary relation $\sqsubseteq$ that is reflexive, antisymmetric, and transitive. In the sequel, we usually denote posets by the set S without explicit mention to the binary relation $\sqsubseteq$.

A subset $P \subseteq S$ is *bounded (above)* if there is a $c \in S$ such that $d \sqsubseteq c$ for each $d \in P$; in this case, c is an *upper bound* for P . A *least upper bound (lub)* for a subset $P \subseteq S$ is an upper bound c for P such that $c \leq d$ for every upper bound d for P . If P has a lub, then it is necessarily unique, and we denote it by $\sqcup P$. A subset $P \subseteq S$ is *directed* if every finite subset Q of P has an upper bound in P . A poset S is *complete*, and hence is a *complete partial order (cpo)* if every directed subset P of S has a lub in S .

We say that $P' \subseteq S$ *dominates* $P \subseteq S$, denoted by $P \sqsubseteq P'$, if for every $c \in P$ there is some $c' \in P'$ such that $c \sqsubseteq c'$. We use the following two simple lemmas, adapted from [15] [Lemmas 3.1.1 and 3.1.2].

Lemma 2.1 *If P, P' are directed subsets of a cpo S and $P \sqsubseteq P'$ then $\sqcup P \sqsubseteq \sqcup P'$.*

Lemma 2.2 *Let $P = \{c_{ij} \mid i \in I, j \in J\}$ be a doubly indexed subset of a cpo S . Let P_i denote the set $\{c_{ij} \mid j \in J\}$ for each $i \in I$. Suppose*

1. P is directed,
2. each P_i is directed with lub c_i , and
3. the set $\{c_i \mid i \in I\}$ is directed.

Then $\sqcup P = \sqcup \{c_i \mid i \in I\}$.

A finite or infinite sequence of elements, $c_0 c_1 c_2 \dots$, of a partially ordered set $(S, \sqsubseteq)$ is called a *chain* if $c_i \sqsubseteq c_{i+1}$ for each non-final index i . We define the *limit* of the chain, $\lim_{i \rightarrow \infty} c_i$, to be the lub of the set $\{c_0, c_1, c_2, \dots\}$ if S contains such a bound; otherwise, the limit is undefined. Since a chain is a special case of a directed set, each chain of a cpo has a limit.

A function $f : S \rightarrow S'$ between posets S and S' is *monotone* if $f(c) \sqsubseteq f(d)$ whenever $c \sqsubseteq d$. If f is monotone and P is a directed set, then the set $f(P) = \{f(c) \mid c \in P\}$ is directed as well. If f is monotone and $f(\sqcup P) = \sqcup f(P)$ for every directed P , then f is said to be *continuous*.

An element c of a cpo S is *compact* if, for every directed set P such that $c \sqsubseteq \sqcup P$, there is some $d \in P$ such that $c \sqsubseteq d$. We define $K(S)$ to be the set of compact elements of S . A cpo S is *algebraic* if every $c \in S$ is the lub of the set $\{d \in K(S) \mid d \sqsubseteq c\}$. A simple example of an algebraic cpo is the set of finite or infinite sequences over some given domain, equipped with the prefix ordering. Here the compact elements are the finite sequences.

2.4 A Basic Graph Lemma

Lemma 2.3 *Let G be an infinite directed graph that satisfies the following properties.*

1. *G has finitely many roots.*
2. *Each node of G has finite outdegree.*
3. *Each node of G is reachable from some root of G .*

Then, there is an infinite path in G starting from some root.

Proof: The proof is an extension of König's Lemma [22]. ■

3 Describing Timed System Behavior

In this section, we give basic definitions that are useful for describing discrete and continuous changes to the system's state. The key notions are *static* and *dynamic types* for variables, *trajectories*, and *hybrid sequences*. Most of the material in this section comes from the paper on the HIOA modeling framework [25]. The reader is referred to [25] for the proofs that are not included here.

3.1 Time

Throughout this paper, we fix a *time axis* $\mathbb{T}$, which is a subgroup of $(\mathbb{R}, +)$, the real numbers with addition. We assume that every infinite, monotone, bounded sequence of elements of $\mathbb{T}$ has a limit in $\mathbb{T}$. The reader may find it convenient to think of $\mathbb{T}$ as the set $\mathbb{R}$ of real numbers, but the set $\mathbb{Z}$ of integers and the singleton set $\{0\}$ are also examples of allowed time axes. We define $\mathbb{T}^{\geq 0} \triangleq \{t \in \mathbb{T} \mid t \geq 0\}$.

An *interval* J is a nonempty, convex subset of $\mathbb{T}$. We denote intervals as usual: $[t_1, t_2] = \{t \in \mathbb{T} \mid t_1 \leq t \leq t_2\}$, $[t_1, t_2) = \{t \in \mathbb{T} \mid t_1 \leq t < t_2\}$ etc. An interval J is *left-closed* (*right-closed*) if it has a minimum (resp., maximum) element, and *left-open* (*right-open*) otherwise. It is *closed* if it is both left-closed and right-closed. We write $\min(J)$ and $\max(J)$ for the minimum and maximum elements, respectively, of an interval J (if they exist), and $\inf(J)$ and $\sup(J)$ for the infimum and supremum, respectively, of J in $\mathbb{R} \cup \{-\infty, \infty\}$. For $K \subseteq \mathbb{T}$ and $t \in \mathbb{T}$, we define $K + t \triangleq \{t' + t \mid t' \in K\}$. Similarly, for a function f with domain K , we define $f + t$ to be the function with domain $K + t$ satisfying, for each $t' \in K + t$, $(f + t)(t') = f(t' - t)$.

In some definitions and theorems in the paper where we use $\mathbb{R}$ as the time domain we assume that the relation $\leq$ on $\mathbb{R}$ extends to a relation on $\mathbb{R} \cup \{\infty\}$ such that $\infty \leq \infty$ and for all $t \in \mathbb{R}$, $t < \infty$.

3.2 Static and Dynamic Types

We assume a universal set V of *variables*. A variable represents a location within the state of a system. For each variable v , we assume both a *(static) type*, which gives the set of values it may take on, and a *dynamic type*, which gives the set of trajectories it may follow. Formally, for each variable v we assume the following:

- $type(v)$, the *(static) type* of v . This is a nonempty set of values.
- $dtype(v)$, the *dynamic type* of v . This is a set of functions from left-closed intervals of T to $type(v)$ that satisfies the following properties:
 1. (*Closure under time shift*)
For each $f \in dtype(v)$ and $t \in T$, $f + t \in dtype(v)$.
 2. (*Closure under subinterval*)
For each $f \in dtype(v)$ and each left-closed interval $J \subseteq dom(f)$, $f \upharpoonright J \in dtype(v)$.
 3. (*Closure under pasting*)
Let $f_0 f_1 f_2, \dots$ be a sequence of functions in $dtype(v)$ such that, for each index i such that f_i is not the final function in the sequence, $dom(f_i)$ is right-closed and $\max(dom(f_i)) = \min(dom(f_{i+1}))$. Then the function f defined by $f(t) \triangleq f_i(t)$, where i is the smallest index such that $t \in dom(f_i)$, is in $dtype(v)$.

Example 3.1 (Discrete variables) Let v be any variable and let *Constant* be the set of constant functions from a left-closed interval of T to $type(v)$. Then *Constant* is closed under time shift and subinterval. If the dynamic type of v is obtained by closing *Constant* under the pasting operation, then v is called a *discrete* variable. This is essentially the same as the definition of a discrete variable in [31]. ■

Example 3.2 (Analog variables) Assume that $T = R$. Let v be any variable whose static type is an interval of R and *Continuous* be the set of continuous functions from a left-closed interval of T to $type(v)$. Then *Continuous* is closed under time shift and subinterval. If the dynamic type of v is obtained by closing *Continuous* under the pasting operation, then v is called an *analog* variable. ■

Example 3.3 (Standard real-valued function classes) If we take $T = R$ and $type(v) = R$, then other examples of dynamic types can be obtained by taking the pasting closure of standard function classes from real analysis, the set of differentiable functions, the set of functions that are differentiable k times (for any k), the set of smooth functions, the set of integrable functions, the set of L^p functions (for any p), the set of measurable locally essentially bounded functions [43], or the set of all functions. ■

Standard function classes are closed under time shift and subinterval, but not under pasting. A natural way of defining a dynamic type is as the pasting closure of a class of functions that is closed under time shift and subinterval. In such a case, it follows that the new class is closed under all three operations.

3.3 Trajectories

In this subsection, we define the notion of a *trajectory*, define operations on trajectories, and prove simple properties of trajectories and their operations. A trajectory is used to model the evolution of a collection of variables over an interval of time.

3.3.1 Basic Definitions

Let V be a set of variables, that is, a subset of $\mathbf{V}$. A *valuation* $\mathbf{v}$ for V is a function that associates with each variable $v \in V$ a value in $\text{type}(v)$. We write $\text{val}(V)$ for the set of valuations for V . Let J be a left-closed interval of $\mathbf{T}$ with left endpoint equal to 0. Then a *J-trajectory* for V is a function $\tau : J \rightarrow \text{val}(V)$, such that for each $v \in V$, $\tau \downarrow v \in \text{dtype}(v)$. A *trajectory* for V is a J -trajectory for V , for any J . We write $\text{trajs}(V)$ for the set of all trajectories for V .

A trajectory for V with domain $[0, 0]$ is called a *point* trajectory for V . If $\mathbf{v}$ is a valuation for V then $\wp(\mathbf{v})$ denotes the point trajectory for V that maps 0 to $\mathbf{v}$. We say that a J -trajectory is *finite* if J is a finite interval, *closed* if J is a (finite) closed interval, *open* if J is a right-open interval, and *full* if $J = \mathbf{T}^{\geq 0}$. If T is a set of trajectories, then $\text{finite}(T)$, $\text{closed}(T)$, $\text{open}(T)$, and $\text{full}(T)$ denote the subsets of T consisting of all the finite, closed, open, and full trajectories in T , respectively.

If τ is a trajectory then $\tau.\text{itime}$, the *limit time* of τ , is the supremum of $\text{dom}(\tau)$. We define $\tau.\text{fval}$, the *first valuation* of τ , to be $\tau(0)$, and if τ is closed, we define $\tau.\text{lval}$, the *last valuation* of τ , to be $\tau(\tau.\text{itime})$. For τ a trajectory and $t \in \mathbf{T}^{\geq 0}$, we define

$$\begin{aligned}\tau \sqsubseteq t &\triangleq \tau \upharpoonright [0, t], \\ \tau \triangleleft t &\triangleq \tau \upharpoonright [0, t), \\ \tau \sqsupseteq t &\triangleq (\tau \upharpoonright [t, \infty)) - t.\end{aligned}$$

Note that, since dynamic types are closed under time shift and subintervals, the result of applying the above operations is always a trajectory, except when the result is a function with an empty domain. By convention, we also write $\tau \sqsubseteq \infty \triangleq \tau$ and $\tau \triangleleft \infty \triangleq \tau$.

3.3.2 Prefix Ordering

Trajectory τ is a *prefix* of trajectory v , denoted by $\tau \leq v$, if τ can be obtained by restricting v to a subset of its domain. Formally, if τ and v are trajectories for V , then $\tau \leq v$ iff

$\tau = v \upharpoonright \text{dom}(\tau)$. Alternatively, $\tau \leq v$ iff there exists a $t \in \mathbb{T}^{\geq 0} \cup \{\infty\}$ such that $\tau = v \trianglelefteq t$ or $\tau = v \triangleleft t$. If $\tau \leq v$ then clearly $\text{dom}(\tau) \subseteq \text{dom}(v)$. If T is a set of trajectories for V , then $\text{pref}(T)$ denotes the *prefix closure* of T , defined by:

$$\text{pref}(T) \triangleq \{\tau \in \text{trajs}(V) \mid \exists v \in T : \tau \leq v\}.$$

We say that T is *prefix closed* if $T = \text{pref}(T)$.

The following lemma gives a simple domain-theoretic characterization of the set of trajectories over a given set V of variables:

Lemma 3.4 *Let V be a set of variables. The set $\text{trajs}(V)$ of trajectories for V , together with the prefix ordering $\leq$, is an algebraic cpo. Its compact elements are the closed trajectories.*

3.3.3 Concatenation

The concatenation of two trajectories is obtained by taking the union of the first trajectory and the function obtained by shifting the domain of the second trajectory until the start time agrees with the limit time of the first trajectory; the last valuation of the first trajectory, which may not be the same as the first valuation of the second trajectory, is the one that appears in the concatenation. Formally, suppose τ and τ' are trajectories for V , with τ closed. Then the *concatenation* $\tau \frown \tau'$ is the function given by

$$\tau \frown \tau' \triangleq \tau \cup (\tau' \upharpoonright (0, \infty) + \tau.\text{ftime}).$$

Because dynamic types are closed under time shift and pasting, it follows that $\tau \frown \tau'$ is a trajectory for V . Observe that $\tau \frown \tau'$ is finite (resp., closed, full) if and only if τ' is finite (resp., closed, full). Observe also that concatenation is associative.

The following lemma, which is easy to prove, shows the close connection between concatenation and the prefix ordering.

Lemma 3.5 *Let τ and v be trajectories for V with τ closed. Then*

$$\tau \leq v \iff \exists \tau' : v = \tau \frown \tau'.$$

Note that if $\tau \leq v$, then the trajectory τ' such that $v = \tau \frown \tau'$ is unique except that it has an arbitrary value for $\tau'.\text{fval}$. Note also that the “ $\Leftarrow$ ” implication in Lemma 3.5 would not hold if the first valuation of the second argument, rather than the last valuation of the first argument, were used in the concatenation.

We extend the definition of concatenation to any (finite or countably infinite) number of arguments. Let $\tau_0 \tau_1 \tau_2 \dots$ be a (finite or infinite) sequence of trajectories such that τ_i is closed for each nonfinal index i . Define trajectories $\tau'_0, \tau'_1, \tau'_2, \dots$ inductively by

$$\begin{aligned}\tau'_0 &\triangleq \tau_0, \\ \tau'_{i+1} &\triangleq \tau'_i \frown \tau_{i+1} \text{ for nonfinal } i.\end{aligned}$$

Lemma 3.5 implies that for each nonfinal i , $\tau'_i \leq \tau'_{i+1}$. We define the *concatenation* $\tau_0 \frown \tau_1 \frown \tau_2 \dots$ to be the limit of the chain $\tau'_0 \tau'_1 \tau'_2 \dots$; existence of this limit follows from Lemma 3.4.

3.4 Hybrid Sequences

In this subsection, we introduce the notion of a *hybrid sequence*, which is used to model a combination of changes that occur instantaneously and changes that occur over intervals of time. Our definition is parameterized by a set A of *actions*, which are used to model instantaneous changes and instantaneous synchronizations with the environment, and a set V of *variables*, which are used to model changes over intervals of time. We also define some special kinds of hybrid sequences and some operations on hybrid sequences, and give basic properties.

3.4.1 Basic Definitions

Fix a set A of actions and a set V of variables. An (A, V) -*sequence* is a finite or infinite alternating sequence $\alpha = \tau_0 a_1 \tau_1 a_2 \tau_2 \dots$, where

1. each τ_i is a trajectory in $\text{trajs}(V)$,
2. each a_i is an action in A ,
3. if α is a finite sequence then it ends with a trajectory, and
4. if τ_i is not the last trajectory in α then $\text{dom}(\tau_i)$ is closed.

A *hybrid sequence* is an (A, V) -sequence for some A and V .

Since the trajectories in a hybrid sequence can be point trajectories our notion of hybrid sequence allows a sequence of discrete actions to occur at the same real time, with corresponding changes of variable values. An alternative approach is described in [37], where state changes at a single real time are modeled using a notion of “superdense time”. Specifically, hybrid behavior is modeled in [37] using functions from an extended time domain, which includes countably many elements for each real time, to states.

If α is a hybrid sequence, with notation as above, then we define the *limit time* of α , $\alpha.\text{ltime}$, to be $\sum_i \tau_i.\text{ltime}$. A hybrid sequence α is defined to be:

- *time-bounded* if $\alpha.ltime$ is finite.
- *admissible* if $\alpha.ltime = \infty$.
- *closed* if α is a finite sequence and the domain of its final trajectory is a closed interval.
- *Zeno* if α is neither closed nor admissible, that is, if α is time-bounded and is either an infinite sequence, or else a finite sequence ending with a trajectory whose domain is right-open.
- *non-Zeno* if α is not Zeno.

For any hybrid sequence α , we define the *first valuation* of α , $\alpha.fval$, to be $head(\alpha).fval$. Also, if α is closed, we define the *last valuation* of α , $\alpha.lval$, to be $last(\alpha).lval$, that is, the last valuation in the final trajectory of α .

If α is a closed (A, V) -sequence, where $V = \emptyset$ and $\beta \in \text{trajs}(\emptyset)$, we call $\alpha \frown \beta$ a *time-extension* of α .

3.4.2 Prefix Ordering

We say that (A, V) -sequence $\alpha = \tau_0 a_1 \tau_1 \dots$ is a *prefix* of (A, V) -sequence $\beta = v_0 b_1 v_1 \dots$, denoted by $\alpha \leq \beta$, provided that (at least) one of the following holds:

1. $\alpha = \beta$.
2. α is a finite sequence ending in some τ_k ; $\tau_i = v_i$ and $a_{i+1} = b_{i+1}$ for every i , $0 \leq i < k$; and $\tau_k \leq v_k$.

Like the set of trajectories over V , the set of (A, V) -sequences is an algebraic cpo:

Lemma 3.6 *Let V be a set of variables and A a set of actions. The set of (A, V) -sequences, together with the prefix ordering $\leq$, is an algebraic cpo. Its compact elements are the closed (A, V) -sequences.*

3.4.3 Concatenation

Suppose α and α' are (A, V) -sequences with α closed. Then the *concatenation* $\alpha \frown \alpha'$ is the (A, V) -sequence given by

$$\alpha \frown \alpha' \triangleq \text{init}(\alpha) \ (\text{last}(\alpha) \frown \text{head}(\alpha')) \ \text{tail}(\alpha').$$

(Here, *init*, *last*, *head* and *tail* are ordinary sequence operations.)

Lemma 3.7 *Let α and β be (A, V) -sequences with α closed. Then*

$$\alpha \leq \beta \iff \exists \alpha' : \beta = \alpha \frown \alpha'.$$

Note that if $\alpha \leq \beta$, then the (A, V) -sequence α' such that $\beta = \alpha \frown \alpha'$ is unique except that it has an arbitrary value in $\text{val}(V)$ for $\alpha'.fval$.

As we did for trajectories, we extend the concatenation definition for (A, V) -sequences to any finite or infinite number of arguments. Let $\alpha_0 \alpha_1 \dots$ be a finite or infinite sequence of (A, V) -sequences such that α_i is closed for each nonfinal index i . Define (A, V) -sequences $\alpha'_0, \alpha'_1, \dots$ inductively by

$$\begin{aligned} \alpha'_0 &\triangleq \alpha_0, \\ \alpha'_{i+1} &\triangleq \alpha'_i \frown \alpha_{i+1} \text{ for nonfinal } i. \end{aligned}$$

Lemma 3.7 implies that for each nonfinal i , $\alpha'_i \leq \alpha'_{i+1}$. We define the *concatenation* $\alpha_0 \frown \alpha_1 \dots$ to be the limit of the chain $\alpha'_0 \alpha'_1 \dots$; existence of this limit is ensured by Lemma 3.6.

3.4.4 Restriction

Let A and A' be sets of actions and let V and V' be sets of variables. The (A', V') -restriction of an (A, V) -sequence α , denoted by $\alpha \upharpoonright (A', V')$, is obtained by first projecting all trajectories of α on the variables in V' , then removing the actions not in A' , and finally concatenating all adjacent trajectories. Formally, we define the (A', V') -restriction first for closed (A, V) -sequences and then extend the definition to arbitrary (A, V) -sequences using a limit construction. The definition for closed (A, V) -sequences is by induction on the length of those sequences:

$$\begin{aligned} \tau \upharpoonright (A', V') &= \tau \downarrow V' \text{ if } \tau \text{ is a single trajectory,} \\ \alpha a \tau \upharpoonright (A', V') &= \begin{cases} (\alpha \upharpoonright (A', V')) a (\tau \downarrow V') & \text{if } a \in A', \\ (\alpha \upharpoonright (A', V')) \frown (\tau \downarrow V') & \text{otherwise.} \end{cases} \end{aligned}$$

It is easy to see that the restriction operator is monotone on the set of closed (A, V) -sequences. Hence, if we apply this operation to a directed set, the result is again a directed set. Together with Lemma 3.6, this allows us to extend the definition of restriction to arbitrary (A, V) -sequences by:

$$\alpha \upharpoonright (A', V') = \sqcup \{ \beta \upharpoonright (A', V') \mid \beta \text{ is a closed prefix of } \alpha \}.$$

Lemma 3.8 *(A', V') -restriction is a continuous operation.*

Lemma 3.9 $(\alpha_0 \frown \alpha_1 \frown \dots) \upharpoonright (A, V) = \alpha_0 \upharpoonright (A, V) \frown \alpha_1 \upharpoonright (A, V) \frown \dots$

Lemma 3.10 $(\alpha \upharpoonright (A, V)) \upharpoonright (A', V') = \alpha \upharpoonright (A \cap A', V \cap V')$.

Lemma 3.11 *Let α be a hybrid sequence A a set of actions and V a set of variables.*

1. α is time-bounded if and only if $\alpha \upharpoonright (A, V)$ is time-bounded.
2. α is admissible if and only if $\alpha \upharpoonright (A, V)$ is admissible.
3. If α is closed then $\alpha \upharpoonright (A, V)$ is closed.
4. If α is non-Zeno then $\alpha \upharpoonright (A, V)$ is non-Zeno.

Example 3.12 (A Zeno execution with a closed (A, V) -restriction) In order to understand why we have an implication in only one direction in items 3 and 4, consider the Zeno sequence α of the form $\wp(\mathbf{v})a\wp(\mathbf{v})a\wp(\mathbf{v})\dots$. Let A be a set such that $a \notin A$ and let V consist of the variables in $\text{dom}(\mathbf{v})$. Obviously, $\alpha \upharpoonright (A, V)$, which is $\wp(\mathbf{v})$, is closed, and hence also non-Zeno. This shows that the fact that $\alpha \upharpoonright (A, V)$ is closed (resp., non-Zeno) does not imply that α is closed (resp., non-Zeno). ■

4 Timed Automata

In this section, as a preliminary step toward defining timed I/O automata, we define a slightly more general *timed automaton* model. In timed automata, actions are classified as external or internal, but external actions are not further classified as input or output; the input/output distinction is added in Section 6. We define how timed automata execute and define implementation and simulation relations between timed automata.

4.1 Definition of Timed Automata

A timed automaton is a state machine whose states are divided into *variables*, and that has a set of discrete *actions*, some of which may be internal and some external. The state of a timed automaton may change in two ways: by *discrete transitions*, which change the state atomically, and by *trajectories*, which describe the evolution of the state over intervals of time. The discrete transitions are labeled with actions; this will allow us to synchronize the transitions of different timed automata when we compose them in parallel. The evolution described by a trajectory may be described by continuous or discontinuous functions.

Formally, a *timed automaton* (TA) $\mathcal{A} = (X, Q, \Theta, E, H, \mathcal{D}, \mathcal{T})$ consists of:

- A set X of *internal variables*.
- A set $Q \subseteq \text{val}(X)$ of *states*.
- A nonempty set $\Theta \subseteq Q$ of *start states*.
- A set E of *external actions* and a set H of *internal actions*, disjoint from each other. We write $A \triangleq E \cup H$.
- A set $\mathcal{D} \subseteq Q \times A \times Q$ of *discrete transitions*.
We use $\mathbf{x} \xrightarrow{a}_{\mathcal{A}} \mathbf{x}'$ as shorthand for $(\mathbf{x}, a, \mathbf{x}') \in \mathcal{D}$. Here and elsewhere, we sometimes drop the subscript and write $\mathbf{x} \xrightarrow{a} \mathbf{x}'$, when we think $\mathcal{A}$ should be clear from the context. We say that a is *enabled* in $\mathbf{x}$ if $\mathbf{x} \xrightarrow{a} \mathbf{x}'$ for some $\mathbf{x}'$. We say that a set C of actions is enabled in a state $\mathbf{x}$ if some action in C is enabled in $\mathbf{x}$.
- A set $\mathcal{T}$ of trajectories for X such that $\tau(t) \in Q$ for every $\tau \in \mathcal{T}$ and $t \in \text{dom}(\tau)$. Given a trajectory $\tau \in \mathcal{T}$ we denote $\tau.fval$ by $\tau.fstate$ and, if τ is closed, we denote $\tau.lval$ by $\tau.lstate$. When $\tau.fstate = \mathbf{x}$ and $\tau.lstate = \mathbf{x}'$, we sometimes write $\mathbf{x} \xrightarrow{\tau}_{\mathcal{A}} \mathbf{x}'$. We require that the following axioms hold:

T0 (*Existence of point trajectories*)

If $\mathbf{x} \in Q$ then $\wp(\mathbf{x}) \in \mathcal{T}$.

T1 (*Prefix closure*)

For every $\tau \in \mathcal{T}$ and every $\tau' \leq \tau$, $\tau' \in \mathcal{T}$.

T2 (*Suffix closure*)

For every $\tau \in \mathcal{T}$ and every $t \in \text{dom}(\tau)$, $\tau \succeq t \in \mathcal{T}$.

T3 (*Concatenation closure*)

Let $\tau_0 \tau_1 \tau_2 \dots$ be a sequence of trajectories in $\mathcal{T}$ such that, for each nonfinal index i , τ_i is closed and $\tau_i.lstate = \tau_{i+1}.fstate$. Then $\tau_0 \frown \tau_1 \frown \tau_2 \dots \in \mathcal{T}$.

Thus, a timed automaton is essentially a hybrid automaton in the sense of [25] in which W , the set of external variables, is empty. (The only difference is the addition of the axiom **T0**, which does not affect any of the results of [25].) This definition differs from previous definitions of timed automata [28, 40] in two major respects. First, the states are structured using variables, which have dynamic types with specific closure properties. The variable structure is convenient for writing specifications and the dynamic types are useful in analyzing continuous evolution of the state. Second, the set of trajectories is defined as an explicit component of an automaton. In the previous definitions, time-passage was represented by special time-passage actions and trajectories were defined implicitly, as auxiliary functions describing the effects of time-passage actions on states.

Notation: We often denote the components of a TA $\mathcal{A}$ by $X_{\mathcal{A}}, Q_{\mathcal{A}}, \Theta_{\mathcal{A}}, E_{\mathcal{A}}$, etc., and the components of a TA $\mathcal{A}_i$ by X_i, Q_i, Θ_i, E_i , etc. We sometimes omit these subscripts, where no confusion seems likely. In examples we typically specify sets of trajectories using differential and algebraic equations and inclusions. Below we explain a few notational conventions that help us in doing this. Suppose the time domain $\mathbb{T}$ is $\mathbb{R}$, τ is a (fixed) trajectory over some set of variables V , and $v \in V$. With some abuse of notation, we use the variable name v to denote the function $\tau \downarrow v$ in $\text{dom}(\tau) \rightarrow \text{type}(v)$, which gives the value of v at all times during trajectory τ . Similarly, we view any expression e containing variables from V as a function with domain $\text{dom}(\tau)$. Suppose that v is a variable and e is a real-valued expression containing variables from V . Using these conventions we can say, for example, that τ satisfies the algebraic equation

$$v = e$$

which means that, for every $t \in \text{dom}(\tau)$, $v(t) = e(t)$, that is, the constraint on the variables expressed by the equation $v = e$ holds for each state on trajectory τ . Now suppose also that e , when viewed as a function, is integrable. Then we say that τ satisfies

$$d(v) = e$$

if, for every $t \in \text{dom}(\tau)$, $v(t) = v(0) + \int_0^t e(t') dt'$. Equivalently, for every $t_1, t_2 \in \text{dom}(\tau)$ such that $t_1 \leq t_2$, $v(t_2) = v(t_1) + \int_{t_1}^{t_2} e(t') dt'$. Note that this interpretation of the differential equation makes sense even at points where v is not differentiable. A similar interpretation of differential equations is used by Polderman and Willems [38], who call functions defined in this way “weak solutions”.

We generalize this notation to handle inequalities as well as equalities. Suppose that v is a variable and e is a real-valued expression containing variables from V . The inequality

$$e \leq v$$

means that, for every $t \in \text{dom}(\tau)$, $e(t) \leq v(t)$. That is, the constraint expressed by the inequality $e \leq v$ holds for each state of trajectory τ . Similarly, the inequality

$$v \leq e$$

means that, for every $t \in \text{dom}(\tau)$, $v(t) \leq e(t)$. Now suppose that e is integrable when viewed as a function. Then we say that τ satisfies

$$e \leq d(v)$$

if, for every $t_1, t_2 \in \text{dom}(\tau)$ such that $t_1 \leq t_2$, $v(t_1) + \int_{t_1}^{t_2} e(t') dt' \leq v(t_2)$, and τ satisfies

$$d(v) \leq e$$

if, for every $t_1, t_2 \in \text{dom}(\tau)$ such that $t_1 \leq t_2$, $v(t_2) \leq v(t_1) + \int_{t_1}^{t_2} e(t') dt'$.

Conventions for automata specifications: In all the examples of this monograph we assume the time axis T to be R and specify timed automata by using a variant of the language presented in [34, 20].

An automaton specification consists of four main parts: a signature, which lists the actions along with their kinds and parameter types, a state variables list, which declares the names and types of state variables, a collection of transition definitions and a trajectories definition.

Static types of variables are always declared explicitly in the state variables list. For example, we write $v:t$ for a variable v of static type t . Moreover, a variable can be initialized to a specific value allowed by its type. For example, in order to initialize the variable v above to the value val , we write $v:t := val$. If no initial value is specified it is assumed to be arbitrary. The state variables list in an automaton specification can be followed by an **initially** clause, which consists of a predicate that constrains the automaton parameters and initial values of state variables. All of the static types used in the examples have standard interpretations, except possibly for the type **AugmentedReal**, which denotes $R \cup \{\infty\}$.

The dynamic types of variables are specified implicitly. Variables of type **Real** are assumed to be analog and variables of types other than **Real** are assumed to be discrete. The definition of what it means for a variable to be discrete or analog is given in Examples 3.1 and 3.2. Note that according to these definitions, a variable of type **Real** is both analog and discrete if its dynamic type consists of piecewise constant functions only. The keyword **discrete** is used to qualify the type **Real** in the declaration of such variables. Although timed automata may contain variables that are neither discrete nor analog, none of our examples use such variables.

The transitions are specified in precondition-effect style. A **pre** clause specifies the enabling condition for an action. An **eff** clause contains a list of statements that specify the effect of performing that action on the state. All the statements in an effect clause are assumed to be executed sequentially in a single indivisible step. The absence of a specified precondition for an action means that the action is always enabled and the absence of a specified effect means that performing the action does not change the state.

The trajectories are specified using a combination of invariants, algebraic and differential equations and inequalities, and stopping conditions. A trajectory belongs to the set of legal trajectories of an automaton if it satisfies the predicate in the **invariant** clause, the stopping condition expressed by the **stop when** clause, and the equations or inequalities in the **evolve** clause. The stopping condition is satisfied by a trajectory if the only state in which the condition holds is the last state of that trajectory. That is, time cannot advance beyond the point where the stopping condition is true. The **evolve** clause specifies the algebraic and differential equations that must be satisfied by the trajectories. We write $d(v) = e$ for $d(v) = e$, $d(v) \leq e$ for $d(v) \leq e$ and $e \leq d(v)$ for $e \leq d(v)$. We assume that the value of a discrete variable is constant throughout a trajectory. That is, time-passage

does not change the value of discrete variables. If the evolution of a variable follows a continuous function throughout a trajectory then we write **continuous**(v). Note that our interpretation of differential inequalities allows “jumps” in trajectories. Therefore, extra caution is needed in the specification of **Real** variables that may evolve at a varying rate. If a variable is assumed to be continuous throughout each trajectory this must be explicitly stated in the trajectory definition.

Example 4.1 (Time-bounded channel) The automaton `TimedChannel(b:Real, M:Type)`

```

automaton TimedChannel(b: Real, M: Type)
  signature
    external send(m: M), receive(m: M)
  states
    queue: Queue[M] := {},
    now: Real := 0,
    initially b ≥ 0
  transitions
    external send(m)
      eff
        queue := append((m,now+b),queue)
    external receive(m)
      pre
        ∃u: Real (m,u) = head(queue)
      eff
        queue := tail(queue)
  trajectories
    stop when
      ∃(m,u): (M,Real) (m,u) ∈ queue ∧ (now = u)
    evolve
      d(now) = 1

```

Figure 1: Time-bounded channel

in Figure 1 is the specification of a reliable FIFO channel that delivers its messages within a certain time bound, represented by the automaton parameter `b` of type **Real** which is nonnegative. The other automaton parameter `M` is an arbitrary type parameter that represents the type of messages communicated by the channel.

The variable `queue` is used to hold pairs consisting of a message that has been sent and its delivery deadline. The variable `now` is used to describe real time.

Every `send(m)` transition adds to the queue a new pair whose first component is `m` and whose second component is the deadline `now + b`. A `receive(m)` transition can occur only when `m` is the first message in the queue and it results in the removal of the first message from the queue.

The trajectory specification shows that the variable `queue` is kept constant by trajectories and that the variable `now` increases with rate 1, that is, at the same rate as real time. The stopping condition implies that, within a trajectory, time cannot pass beyond the point where `now` becomes equal to the delivery deadline of some message in the queue. ■

Example 4.2 (Periodic sending process) The automaton `PeriodicSend(u:Real, M:Type)`

```

automaton PeriodicSend(u: Real, M: Type)
  signature
    external send(m: M)
  states
    clock: Real := 0
    initially u ≥ 0
  transitions
    external send(m)
      pre
        clock = u
      eff
        clock := 0
  trajectories
    stop when
      clock = u
    evolve
      d(clock) = 1

```

Figure 2: Periodic sending process

in Figure 2 is the specification of a process that sends messages periodically, every `u` time units, where `u` is an automaton parameter of type `Real` which is nonnegative. The type parameter `M` represents the type of the messages sent by the process.

The analog variable `clock` is a timer whose value records the amount of time that has elapsed since it was last reset to 0. A `send(m)` transition can occur only when `clock = u`, and it causes `clock` to be reset. The trajectory specification says that `clock` increases at the same rate as real time and time cannot pass beyond the point where `clock = u`. ■

Example 4.3 (Periodic sending process with failures) The specification of the `PeriodicSend(u:Real, M:Type)` process from Example 4.2 does not model failures. We now consider a variant of `PeriodicSend(u:Real, M:Type)` where the process may fail and stop doing any discrete actions. The specification of this new automaton is given in Figure 3.

```

automaton PeriodicSend2(u: Real, M: Type)
  signature
    external send(m: M), fail
  states
    failed: Bool := false,
    clock: Real := 0
    initially u ≥ 0
  transitions
    external send(m)
      pre
        ¬failed ∧ clock = u
      eff
        clock := 0
    external fail
      eff
        failed := true
  trajectories
    stop when
      ¬failed ∧ clock = u
    evolve
      d(clock) = 1

```

Figure 3: Periodic sending process with failures

The discrete variable `failed` in `automaton PeriodicSend2(u:Real, M:Type)` is a boolean flag that records whether the process is failed. It is initialized to `false` and is set to `true` when a `fail` action occurs. The trajectory specification of `PeriodicSend2(u:Real, M:Type)` shows that time can advance without any bound when the process is failed. ■

Example 4.4 (Timeout process) The automaton `Timeout(u:Real, M:Type)` in Figure 4 is the specification of a process that awaits the receipt of a message from another process. If `u` time units elapse without such a message arriving, `Timeout(u:Real, M:Type)` performs a `timeout` action, thereby “suspecting” the other process. When a message arrives it “unsuspects” the other process. `Timeout(u:Real, M:Type)` may suspect and unsuspect repeatedly.

The discrete variable `suspected` is a flag that shows whether `Timeout(u:Real, M:Type)` suspects that the other process is failed. The variable `clock` is a timer that records the amount of time that has elapsed since the receipt of the last message.

A `receive(m)` transition can occur at any time; this causes the variable `clock` to be reset and the flag `suspected` to be set to `false`. If `clock` reaches `u` before the arrival of a message then the `timeout` action becomes enabled. The process sets `suspected` to `true` as

```

automaton Timeout(u:Real, M: Type)
  signature
    external receive(m: M), timeout
  states
    suspected: Bool := false,
    clock Real := 0
    initially u > 0
  transitions
    external receive(m)
      eff
        clock:=0;
        suspected:= false
    external timeout
      pre
        ¬suspected ∧ clock = u
      eff
        suspected := true
  trajectories
    stop when
      clock = u and ¬suspected
    evolve
      d(clock) = 1

```

Figure 4: Timeout

a result of a timeout.

The discrete variable `suspected` remains constant throughout each trajectory. The trajectory specification also shows that `clock` increases at the same rate as real time and, if `suspected = false`, then time cannot go beyond the point where `clock = u`. Note that if `suspected = true`, there is no restriction on the amount of time that can elapse. ■

Example 4.5 (Fischer’s mutual exclusion algorithm) The automaton `FischerME(u_set, l_check: Real)` presented in Figures 5 and 6 is the specification of a shared memory mutual exclusion algorithm which uses a single shared variable that can be read and written by all the participants. We fix here the number of participants to be four, by defining `Index` to be an enumeration consisting of five elements, one of which represents a special value. Note, however, that this specification can be generalized to any finite number of participants. The automaton parameters `u_set` and `l_check` represent upper and lower time bounds for the `set(i)` and `check(i)` actions respectively. We assume that `u_set < l_check`.

The shared variable `x` can be assigned any value of type `Index` where `nil` is regarded as a special value. If a process is in the critical region, then the variable `x` contains the

```

type Index = enumeration of nil, p1, p2, p3, p4

type PcValue = enumeration of rem, test, set, check,
                leavetry, crit, leaveexit

automaton FischerME(u_set, l_check: Real)
  signature
    external try(i:Index), crit(i:Index), exit(i:Index), rem(i:Index)
    internal test(i:Index), set(i:Index),
              check(i:Index), reset(i:Index)

  states
    x: Index := nil,
    pc: Array[Index,PcValue] := constant(rem),
    lastset: Array[Index,discrete AugmentedReal] := constant(infty),
    firstcheck: Array[Index,discrete AugmentedReal] := constant(0),
    now: Real:=0
    initially u_set  $\geq$  0  $\wedge$  l_check  $\geq$  0  $\wedge$  u_set < l_check

```

Figure 5: Fischer’s mutual exclusion algorithm: Signature and states

index of that process. If all users are in the remainder region, then the variable x contains the value `nil`. The array variable `pc` records the program counters of all processes. The array variable `lastset` keeps track of the deadlines by which the processes’ `set` actions must occur. Similarly, the array variable `firstcheck` keeps track of the earliest time the processes’ `check` actions may occur. The analog variable `now` models real time.

The transition definitions for external actions `try(i)`, `test(i)`, `crit(i)`, and `exit(i)` are straightforward. When a process performs one of these actions, its program counter is updated to record the region entered by the process. The most interesting transition definitions are `test(i)`, `set(i)`, and `check(i)` since they are the ones that involve timing constraints of the algorithm. When a process i performs a `test` action and observes x to be `nil`, it sets `lastset[i]` to `now + u_set`. This sets the deadline for the performance of the `set(i)` action. Note that this deadline is enforced through the stopping condition in the trajectory specification. The transition `set(i)` sets `firstcheck[i]` to `now + l_check`. The value of `firstcheck[i]` determines the earliest time `check(i)` may occur. The `check(i)` action is enabled only when the current time has at least this value.

The trajectory specification says that the values of discrete variables are kept constant by trajectories. The stopping condition implies that if the value of `now` reaches the value of `lastset[i]` for some process i at some point in time, then that point must be the limit time of the trajectory.

■

```

transitions
  external try(i)
    pre
      pc[i] = rem
    eff
      pc[i] := test
  internal test(i)
    pre
      pc[i] = test
    eff
      if x = p0 then
        pc[i] := set;
        lastset[i] := now + u_set
  internal set(i)
    pre
      pc[i] = set
    eff
      x := i;
      pc[i] := check;
      lastset[i] := infty;
      firstcheck[i] := now + l_check
  internal check(i)
    pre
      pc[i] = check ∧
        now ≥ firstcheck[i]
    eff
      if x = i then pc[i] := leavetry
      else pc[i] := test
trajectories
  stop when
    ∃ i: Index now = lastset[i]
  evolve
    d(now) = 1

```

```

external crit(i)
  pre
    pc[i] = leavetry
  eff
    pc[i] := crit
external exit(i)
  pre
    pc[i] = crit
  eff
    pc[i] := reset

internal reset(i)
  pre
    pc[i] = reset
  eff
    x := i;
    pc[i] := leaveexit

external rem(i)
  pre
    pc[i] = leaveexit
  eff
    pc[i] := rem

```

Figure 6: Fischer's mutual exclusion algorithm: Transitions and trajectory definitions

```

automaton ClockSync(u,r: Real, i: Index)
  signature
    external send(m: Real, const i: Index),
      receive(m: Real, j: Index, const i: Index) where j  $\neq$  i
  states
    nextsend: Real := 0,
    maxother: Real := 0,
    physclock: Real := 0
    initially u > 0  $\wedge$  (0  $\leq$  r < 1)

  derived variables
    logclock = max(maxother, physclock)

  transitions
    external send(m,i)
      pre
        m = physclock  $\wedge$  physclock = nextsend
      eff
        nextsend := nextsend + u
    external receive(m,j,i)
      eff
        maxother := max(maxother,m)
  trajectories
    invariant
      continuous(physclock)
    stop when
      physclock = nextsend
    evolve
      (1 - r)  $\leq$  d(physclock)  $\leq$  (1 + r)

```

Figure 7: Clock synchronization

Example 4.6 (Clock synchronization)

The automaton `ClockSync(u,r:Real, i:Index)` in Figure 7 is the specification of a single process in a clock synchronization algorithm. Each process has a physical clock and generates a logical clock. The goal of the algorithm is to achieve “agreement” and “validity” among the logical clock values. Agreement means that the logical clocks are close to one another. Validity means that the logical clocks are within the range of the physical clocks.

The algorithm is based on the exchange of physical clock values between different processes in the system. The parameter `u` determines the frequency of sending messages. Processes in the system are indexed by the elements of the type `Index` which we assume to be pre-defined. `ClockSync(u,r:Real, i:Index)` has a physical clock `physclock`, which

may drift from the real time with a drift rate bounded by r . It uses the variable `maxother` to keep track of the largest physical clock value of the other processes in the system. The variable `nextsend` records when it is supposed to send its physical clock to the other processes. The logical clock, `logclock`, is defined to be the maximum of `maxother` and `physclock`. Formally `logclock` is a *derived variable*, which is a function whose value is defined in terms of the state variables.

A `send(m,i)` transition is enabled when `m = physclock` and `nextsend = physclock`. It causes the value of `nextsend` to be updated so that the next send can occur when `physclock` has advanced by `u` time units. The transition definition for `receive(m,j,i)` specifies the effect of receiving a message from another process `j` in the system. Upon the receipt of a message `m` from `j`, `i` sets `maxother` to the maximum of `m` and the current value of `maxother`, thereby updating its knowledge of the largest physical clock value of other processes in the system.

The trajectory specification is slightly different from that in the previous examples. In this example, the analog variable `physclock` does not change at the same rate as real time but it drifts with a rate that is bounded by r . The periodic sending of physical clocks to other processes is enforced through the stopping condition in the trajectory specification. Time is not allowed to pass beyond the point where `physclock = nextsend`.

■

4.2 Executions and Traces

We now define execution fragments, executions, trace fragments, and traces, which are used to describe automaton behavior. An *execution fragment* of a timed automaton $\mathcal{A}$ is an (A, V) -sequence $\alpha = \tau_0 a_1 \tau_1 a_2 \tau_2 \dots$, where (1) each τ_i is a trajectory in $\mathcal{T}$, and (2) if τ_i is not the last trajectory in α then $\tau_i.lstate \xrightarrow{a_{i+1}} \tau_{i+1}.fstate$. An execution fragment records what happens during a particular run of a system, including all the instantaneous, discrete state changes and all the changes to the state that occur while time advances. We write $frags_{\mathcal{A}}$ for the set of all execution fragments of $\mathcal{A}$.

If α is an execution fragment, with notation as above, then we define the *first state* of α , $\alpha.fstate$, to be $\alpha.fval$. An *execution fragment* of a timed automaton $\mathcal{A}$ *from* a state $\mathbf{x}$ of $\mathcal{A}$ is an execution fragment of $\mathcal{A}$ whose first state is $\mathbf{x}$. We write $frags_{\mathcal{A}}(\mathbf{x})$ for the set of execution fragments of $\mathcal{A}$ from $\mathbf{x}$. An execution fragment α is defined to be an *execution* if $\alpha.fstate$ is a start state, that is, $\alpha.fstate \in \Theta$. We write $execs_{\mathcal{A}}$ for the set of all executions of $\mathcal{A}$. If α is a closed (A, V) -sequence then we define the *last state* of α , $\alpha.lstate$, to be $\alpha.lval$.

A state of $\mathcal{A}$ is *reachable* if it is the last state of some closed execution of $\mathcal{A}$. A property that is true for all reachable states of an automaton is called an *invariant assertion*, or *invariant*, for short.

Lemma 4.7 *Let $\alpha_0 \alpha_1 \dots$ be a finite or infinite sequence of execution fragments of $\mathcal{A}$ such that, for each nonfinal index i , α_i is closed and $\alpha_i.lstate = \alpha_{i+1}.fstate$. Then $\alpha_0 \frown \alpha_1 \frown \dots$ is an execution fragment of $\mathcal{A}$.*

Proof: Follows easily from the definitions, using axiom **T3**. ■

Lemma 4.8 *Let α and β be execution fragments of $\mathcal{A}$ with α closed. Then*

$$\alpha \leq \beta \iff \exists \alpha' \in \text{frags}_{\mathcal{A}} : \beta = \alpha \frown \alpha'.$$

Proof: Implication “ $\Leftarrow$ ” follows directly from the corresponding implication in Lemma 3.7. Implication “ $\Rightarrow$ ” follows from the definitions and **T2**. ■

The external behavior of a timed automaton is captured by the set of “traces” of its execution fragments, which record external actions and the trajectories that describe the intervening passage of time. A trace consists of alternating external actions and trajectories over the empty set of variables, $\emptyset$; the only interesting information contained in these trajectories is the amount of time that elapses.

Formally, if α is an execution fragment, then the *trace* of α , denoted by $\text{trace}(\alpha)$, is the $(E, \emptyset)$ -restriction of α , $\alpha \upharpoonright (E, \emptyset)$. A *trace fragment* of a timed automaton $\mathcal{A}$ from a state $\mathbf{x}$ of $\mathcal{A}$ is the trace of an execution fragment of $\mathcal{A}$ whose first state is $\mathbf{x}$. We write $\text{tracefrags}_{\mathcal{A}}(\mathbf{x})$ for the set of trace fragments of $\mathcal{A}$ from $\mathbf{x}$. Also, we define a *trace* of $\mathcal{A}$ to be a trace fragment from a start state, that is, the trace of an execution of $\mathcal{A}$, and write $\text{traces}_{\mathcal{A}}$ for the set of traces of $\mathcal{A}$.

In the earlier timed automaton models [28, 40], execution fragments were defined in a similar style to the one presented here, that is, as an alternating sequence of trajectories and actions. However, the traces were not derived from execution fragments by a simple restriction to external actions and the empty set of variables. Rather, a trace was defined as a sequence consisting of actions paired with their time of occurrence together with a limit time. The new definition increases uniformity; the definitions, results and proof techniques for hybrid sequences apply to both execution fragments and traces.

We now revisit some of the automata presented earlier in this section and give sample executions and traces for these automata.

Example 4.9 (Periodic sending process) Consider the automaton `PeriodicSend(u:Real, M:Type)` from Example 4.2 where u is instantiated to the real number 3 and the message type parameter M is instantiated to the set $\{\mathbf{m1}, \mathbf{m2}, \dots\}$. The following sequence is an execution of the automaton:

$$\alpha = \tau_0 \text{ send}(\mathbf{m1}) \tau_1 \text{ send}(\mathbf{m2}) \tau_2 \text{ send}(\mathbf{m3}) \tau_3 \dots$$

where $\tau_i : [0, 3] \rightarrow \text{val}(\{\text{clock}\})$ are defined such that $\tau_i(t)(\text{clock}) = t$ for all $t \in [0, 3]$.

The functions τ_i are defined for closed intervals of length 3, starting at time 0. They describe the evolution of the variable `clock`, which is 0 at the start of each τ_i and increases with rate 1 for 3 time units. The discrete `send` events occur periodically, every 3 time units and reset the `clock` variable to 0.

The trace of the above execution fragment, $\text{trace}(\alpha)$, is the sequence

$$\alpha = \tau'_0 \text{ send(m1)} \tau'_1 \text{ send(m2)} \tau'_2 \text{ send(m3)} \tau'_3 \dots$$

where $\tau'_i : [0, 3] \rightarrow \text{val}(\emptyset)$.

Since the range of each function τ'_i contains only the function with the empty domain, $\text{trace}(\alpha)$ does not contain any information about what happens to the value of `clock` as time progresses. Since the domains of each τ_i and τ'_i are identical, α and $\text{trace}(\alpha')$ express the same information about the amount of time that elapses between discrete steps. ■

Example 4.10 (Timeout process) We now present an execution of the automaton $\text{Timeout}(u:\text{Real}, M:\text{Type})$ from Example 4.4 where the maximum waiting time u for a message is 5 and the message alphabet M is the set $\{\text{m1}, \text{m2}\}$. The following finite sequence is an execution of $\text{Timeout}(u:\text{Real}, M:\text{Type})$:

$$\alpha = \tau_0 \text{ receive(m1)} \tau_1 \text{ timeout} \tau_2 \text{ receive(m2)} \tau_3 \text{ timeout} \tau_4$$

where $\text{Val} = \text{val}(\{\text{suspected}, \text{clock}\})$ and the functions $\tau_0, \tau_1, \tau_2, \tau_3, \tau_4$ are defined as follows:

- $\tau_0 : [0, 2] \rightarrow \text{Val}$ where $\tau_0(t)(\text{suspected}) = \text{false}$ and $\tau_0(t)(\text{clock}) = t$ for all $t \in [0, 2]$.
- $\tau_1 : [0, 5] \rightarrow \text{Val}$ where $\tau_1(t)(\text{suspected}) = \text{false}$ and $\tau_1(t)(\text{clock}) = t$ for all $t \in [0, 5]$.
- $\tau_2 : [0, 1] \rightarrow \text{Val}$ where $\tau_2(t)(\text{suspected}) = \text{true}$ and $\tau_2(t)(\text{clock}) = 5 + t$ for all $t \in [0, 1]$.
- $\tau_3 : [0, 5] \rightarrow \text{Val}$ where $\tau_3(t)(\text{suspected}) = \text{false}$ and $\tau_3(t)(\text{clock}) = t$ for all $t \in [0, 5]$.
- $\tau_4 : [0, \infty) \rightarrow \text{Val}$ where $\tau_4(t)(\text{suspected}) = \text{true}$ and $\tau_4(t)(\text{clock}) = 5 + t$ for all $t \in [0, \infty)$.

In this sample execution, the first awaited message arrives at time 2. Since no other message arrives within the next 5 time units, the process performs a timeout. A new message arrives 1 time unit after the timeout and the variable `clock` is reset to 0. Since no new message arrives in the next 5 time units the process performs another timeout. The time elapses forever after this timeout since no further message arrives.

This example illustrates that the automaton $\text{Timeout}(u:\text{Real}, M:\text{Type})$ can perform multiple timeout transitions. Another point to note is that the sample execution consists of a finite (A, V) -sequence ending with a trajectory, as opposed to an infinite sequence as in Example 4.9. The final trajectory here is a trajectory whose domain is right open

and the execution is admissible and non-Zeno. Replacing τ_4 with a function on a closed interval would yield a non-Zeno execution that is not admissible.

The trace of the execution α can be obtained by letting the range of τ_i be the set consisting of the function with the empty domain, as we did in the previous example. That is, by hiding the values of the internal variables `clock` and `suspected` during trajectories. ■

Example 4.11 (Time-bounded channel) Consider the time-bounded channel automaton from Example 4.1. It is easy to observe that time cannot pass beyond any delivery deadline recorded in the message queue and that each deadline in the queue is less than or equal to the sum of the current time and the bound `b`. This property can be stated as an invariant assertion as follows.

Invariant 1 : In any reachable state $\mathbf{x}$ of automaton `TimedChannel(b:Real, M:Type)`, for all (m, u) in $\mathbf{x}(\text{queue})$, $\mathbf{x}(\text{now}) \leq u \leq \mathbf{x}(\text{now}) + b$.

Such an invariant can be proved by induction. Recall that reachable states are the final states of closed executions. Axioms **T1** and **T2** allow us to view any closed execution as a concatenation of closed execution fragments, $\alpha_0 \frown \alpha_1 \frown \dots \frown \alpha_k$, where every α_i is either a closed trajectory or a discrete action surrounded by point trajectories, and where $\alpha_i.\text{lstate} = \alpha_{i+1}.\text{fstate}$ for $0 \leq i \leq k-1$. The invariant can then be proved using induction on the length k of the sequence of execution fragments α_i . ■

Example 4.12 (Fischer's mutual exclusion) The main safety property that needs to be satisfied by the automaton `FischerME(u_set, l_check:Real)` from Example 4.5 is mutual exclusion. This safety property can be expressed as an invariant assertion:

Invariant 1 : In any reachable state $\mathbf{x}$ of `FischerME(u_set, l_check:Real)`, there do not exist $i:\text{Index}$ and $j:\text{Index}$ such that $\mathbf{x}(\text{pc})[i] = \text{crit}$ and $\mathbf{x}(\text{pc})[j] = \text{crit}$.

Even though the invariant does not refer to time, its proof depends on the timing constraints of the automaton. For example, the following auxiliary invariant can be used in proving Invariant 4.12:

Invariant 2 : In any reachable state $\mathbf{x}$ of `FischerME(u_set, l_check:Real)`, if $\mathbf{x}(\text{pc})[i] = \text{check}$, $\mathbf{x}(x) = i$, and $\mathbf{x}(\text{pc})[j] = \text{set}$, then $\mathbf{x}(\text{firstcheck})[i] > \mathbf{x}(\text{lastset})[j]$.

This invariant states that if the program counter of process i has the value `check`, the program counter of process j has the value `set`, and the variable x has the value i , then i will allow enough time for j to set x to j , before performing the check. If this timing constraint were not satisfied, it would be possible for i to check that $x = i$ before j sets x to j . Both of the processes would then observe x to contain their own index and enter the critical region. ■

Lemma 4.13 *If α is an execution of $\mathcal{A}$ then*

1. *α is time-bounded if and only if $\text{trace}(\alpha)$ is time-bounded.*
2. *α is admissible if and only if $\text{trace}(\alpha)$ is admissible.*
3. *If α is closed then $\text{trace}(\alpha)$ is closed.*
4. *If α is non-Zeno then $\text{trace}(\alpha)$ is non-Zeno.*

Proof: It follows directly from the restriction of (A,V)-sequences. ■

Lemma 4.14 *If β is a trace of $\mathcal{A}$ then*

1. *If β is closed then there exists an execution α of $\mathcal{A}$ such that $\text{trace}(\alpha) = \beta$ and α is closed.*
2. *If β is non-Zeno then there exists an execution α of $\mathcal{A}$ such that $\text{trace}(\alpha) = \beta$ and α is non-Zeno.*

Proof: For the first part of the theorem, let $\beta = \text{trace}(\alpha)$ be a closed trace of $\mathcal{A}$. By definition of a trace, we know that $\beta.\text{lttime} = \alpha.\text{lttime}$. We also know that α is either closed or has a suffix which is an infinite sequence of alternating point trajectories and internal actions. Now, let α' be the least closed prefix of α such that $\alpha'.\text{lttime} = \beta.\text{lttime}$. Clearly, α' is a closed execution of $\mathcal{A}$.

For the second part of the theorem, observe that a non-Zeno trace is either closed or admissible. Let $\beta = \text{trace}(\alpha)$. For the case where β is closed, we have already shown how we can find a closed execution. For the case where $\beta = \text{trace}(\alpha)$ is admissible, we know that $\alpha.\text{lttime} = \infty$. Hence, α is admissible, as needed. ■

Example 4.15 (Constructing a closed execution from a closed trace) Consider the Zeno hybrid sequence $\alpha = \wp(\mathbf{v}) a \wp(\mathbf{v}) a \wp(\mathbf{v}) \dots$ given in Example 3.12. Suppose that α is an execution of $\mathcal{A}$ and that a is an internal action of $\mathcal{A}$. Then, $\text{trace}(\alpha) = \wp(\mathbf{v}')$ where $\wp(\mathbf{v}')$ is a trajectory over the empty set of variables. However, the fact that $\text{trace}(\alpha)$ is closed does not imply that α is closed. Thus, we see why we have a one way implication in item 3 of Lemma 4.13. On the other hand, we can construct a closed execution of $\mathcal{A}$ with trace $\wp(\mathbf{v}')$ as explained in the proof of Lemma 4.14. The execution consisting of the point trajectory $\wp(\mathbf{v}')$ is a closed execution of $\mathcal{A}$ with trace $\wp(\mathbf{v}')$. ■

4.3 Special Kinds of Timed Automata

This section describes several restricted forms of timed automata and gives definitions that are needed for theorems that are presented in later sections.

Timed Automata with Finite Internal Nondeterminism: We are sometimes interested in bounding the amount of internal nondeterminism in a timed automaton. Thus, we say that a timed automaton $\mathcal{A}$ *has finite internal nondeterminism (FIN)* provided that:

1. The set Θ of start states is finite, and
2. For every state $\mathbf{x}$ of $\mathcal{A}$ and every trace fragment β of $\mathcal{A}$ from $\mathbf{x}$, the set $\{\alpha.lstate \mid \alpha \in frags_{\mathcal{A}}(\mathbf{x}) \wedge trace(\alpha) = \beta\}$ is finite.

Example 4.16 (Automata with FIN) The automata `TimedChannel(b:Real, M:Type)`, `PeriodicSend(u:Real, M:Type)`, `PeriodicSend2(u:Real, M:Type)`, and `Timeout(u:Real, M:Type)` given in Section 4.1 all have FIN. The first property of the definition of FIN is satisfied since each of these automata has a unique start state. The second property follows from the fact that in each automaton, for every state $\mathbf{x}$ and every trace fragment β from $\mathbf{x}$, there is a unique execution fragment α such that $trace(\alpha) = \beta$. ■

Example 4.17 (Automata without FIN) We show that `FischerME(u_set, l_check:Real)` and `ClockSync(a, r: Real, i:Index)` do not have FIN. For each automaton, we specify a trace, describe the set of all executions that have the specified trace, and argue that the second property in the definition of FIN fails for the chosen trace.

Let $\mathbf{x}$ be the start state of `FischerME(u_set, l_check:Real)` and $\beta = \tau_0 \text{ try}(i) \tau_1$ be a trace of the same automaton where the domains of the functions τ_0 and τ_1 are, respectively, the single point interval $[0, 0]$ and the interval $[0, u]$, and the range of both functions is the set consisting of the function with the empty domain. For any execution α , $trace(\alpha) = \beta$, if and only if $\alpha.ltime = u$, `try(i)` occurs at time 0, and all the actions in α that occur after `try(i)` are internal actions. There are infinitely many different times that the internal actions may occur, and infinitely many values `lastcheck` and `firstcheck` could have, by the time u . Therefore, the set $\{\alpha.lstate \mid \alpha \in frags_{\mathcal{A}}(\mathbf{x}) \wedge trace(\alpha) = \tau_0 \text{ try}(i) \tau_1\}$ is not finite and `FischerME(u_set, l_check:Real)` does not have FIN.

Now, let $\mathbf{x}$ be the start state of `ClockSync(a, r:Real, i:Index)` where $\mathbf{x}(\text{physclock}) = \mathbf{x}(\text{nextsend}) = \mathbf{x}(\text{maxother}) = 0$ and $\beta = \tau_0 \text{ send}(0) \tau_1$ be a trace of `ClockSync(a, r:Real, i:Index)` where the domains of functions τ_0 and τ_1 are, respectively, the interval $[0, 0]$ and the interval $[0, u]$, and the range of both functions is the set consisting of the function with the empty domain. For any α in which `send(0)` occurs at time 0 and is followed by a trajectory τ such that $\tau.ltime = u$, we have $trace(\alpha) = \beta$. For any such α , $\alpha.lstate(\text{physclock})$ can be any value in the interval $[u(1 - r), u(1 + r)]$. Therefore, the set $\{\alpha.lstate \mid \alpha \in frags_{\mathcal{A}}(\mathbf{x}) \wedge trace(\alpha) = \tau_0 \text{ send}(0) \tau_1\}$ is not finite and `ClockSync(a, r:Real, i:Index)` does not have FIN. ■

The following lemma states that if a timed automaton has FIN, then its set of traces is limit-closed.

Lemma 4.18 *Suppose that timed automaton $\mathcal{A}$ has FIN and $\mathbf{x} \in Q$. Suppose that $\beta_1 \beta_2 \dots$ is a chain of trace fragments of $\mathcal{A}$ from $\mathbf{x}$. Then the hybrid sequence $\lim_i \beta_i$ is a trace fragment of $\mathcal{A}$ from $\mathbf{x}$.*

Proof: This is analogous to the proof of Lemma 4.3 of [28]. Suppose that $\mathcal{A}$ is a timed automaton that has FIN, $\mathbf{x}$ is a state of $\mathcal{A}$, and $\beta_1 \beta_2 \dots$ is a chain of trace fragments of $\mathcal{A}$ from $\mathbf{x}$. We define a relation *after* between trace fragments from $\mathbf{x}$ and states of $\mathcal{A}$: $\text{after} = \{(\beta, \mathbf{y}) \mid \exists \alpha \in \text{frags}_{\mathcal{A}}(\mathbf{x}). \text{trace}(\alpha) = \beta \wedge \alpha.\text{lstate} = \mathbf{y}\}$.

We construct a directed graph G whose nodes are pairs $(\beta_i, \mathbf{y}) \in \text{after}$ where β_i is an element of the given chain. In G , there is an edge from $(\beta_i, \mathbf{y})$ to $(\beta_{i+1}, \mathbf{y}')$ exactly if $\beta_{i+1} = \beta_i \frown \gamma$ such that $\gamma = \text{trace}(\alpha)$ for some $\alpha \in \text{frags}_{\mathcal{A}}(\mathbf{y})$, and $\alpha.\text{lstate} = \mathbf{y}'$. By the definition of property FIN, there are finitely many roots of G . By the definition of FIN and the construction of G , each node of G has finite outdegree.

We claim that each node $(\beta_i, \mathbf{y})$ of G is reachable from some root $(\beta_1, \mathbf{z})$ for some $\mathbf{z}$. By definition of the node set, there exists $\alpha \in \text{frags}_{\mathcal{A}}(\mathbf{x})$ such that $\text{trace}(\alpha) = \beta_i$ and $\alpha.\text{lstate} = \mathbf{y}$. Choose $\alpha' \in \text{frags}_{\mathcal{A}}(\mathbf{x})$ to be a prefix of α such that $\text{trace}(\alpha') = \beta_1$ and let $\mathbf{z} = \alpha'.\text{lstate}$. By definition of the edge set of G , $(\beta_i, \mathbf{y})$ is reachable from $(\beta_1, \mathbf{z})$.

Hence, G satisfies the hypotheses of Lemma 2.3, which implies that there is an infinite execution fragment starting from $\mathbf{x}$ whose trace is $\lim_i \beta_i$. Lemma 2.3 is an extension of Konig's lemma. ■

There are two references to automata with FIN later in the paper. The first one is in Theorem 4.19, which lists some sufficient conditions for establishing an implementation relationship between two automata. The second reference appears in the discussion about the kinds of automata that satisfy the assumptions of Theorem 7.7.

Feasible Timed Automata: A timed automaton $\mathcal{A}$ is *feasible* provided that, for every state $\mathbf{x}$ of $\mathcal{A}$, there exists an admissible execution fragment of $\mathcal{A}$ from $\mathbf{x}$.

Feasibility is a basic requirement that any “reasonable” timed automaton should satisfy. Theorems 4.19, and 6.2 establish some results about feasible automata.

Timing-Independent Timed Automata: A timed automaton $\mathcal{A}$ is said to be *timing-independent* provided that all its state variables are discrete variables, and its set of trajectories is exactly the set of constant-valued functions over left-closed time intervals with left endpoint 0.

We refer to timing-independent automata later in Examples 5.12 and 7.9, and in our discussion about Corollary 7.8.

4.4 Implementation Relationships

Timed automata $\mathcal{A}_1$ and $\mathcal{A}_2$ are *comparable* if they have the same external interface, that is, if $E_1 = E_2$. If $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable then we say that $\mathcal{A}_1$ *implements* $\mathcal{A}_2$, denoted by $\mathcal{A}_1 \leq \mathcal{A}_2$, if the traces of $\mathcal{A}_1$ are included among those of $\mathcal{A}_2$, that is, if $\text{traces}_{\mathcal{A}_1} \subseteq \text{traces}_{\mathcal{A}_2}$.¹

Other preorders between timed automata could also be used as implementation relationships, for example, if $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable timed automata, we could consider:

- Every closed trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$.
- Every admissible trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$.
- Every non-Zeno trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$.

Theorem 4.19 *Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be comparable TAs.*

1. *If every closed trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$ and $\mathcal{A}_2$ has FIN, then $\mathcal{A}_1 \leq \mathcal{A}_2$.*
2. *If every admissible trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$ and $\mathcal{A}_1$ is feasible, then every closed trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$.*
3. *If every admissible trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$, $\mathcal{A}_1$ is feasible, and $\mathcal{A}_2$ has FIN, then $\mathcal{A}_1 \leq \mathcal{A}_2$.*

Proof: Part 1 follows from Lemma 4.18.

For Part 2, consider a closed trace β of $\mathcal{A}_1$. By feasibility of $\mathcal{A}_1$, we may extend β to an admissible trace β' of $\mathcal{A}_1$. Then by assumption, β' is also a trace of $\mathcal{A}_2$. By prefix closure of the set of traces, β is a trace of $\mathcal{A}_2$.

Part 3 follows from Parts 1 and 2. ■

4.5 Simulation Relations

In this section, we define simulation relations between timed automata. Simulation relations may be used to show that one TA implements another, in the sense of inclusion of sets of traces. We define two types of simulation relations: forward and backward simulations.

¹In [28, 13, 26, 27], definitions of the set of traces of an automaton and of one automaton implementing another are based on closed and admissible executions only. The results we obtain in this paper using the newer, more inclusive definition imply corresponding results for the earlier definition. For example, we have the following property: If $\mathcal{A}_1 \leq \mathcal{A}_2$ then the set of traces that arise from closed or admissible executions of $\mathcal{A}_1$ is a subset of the set of traces that arise from closed or admissible executions of $\mathcal{A}_2$. This follows from Lemmas 4.13 and 4.14.

Forward simulations are more commonly used than backward simulations because they are easier to think about and are general enough to cover most interesting situations that arise in practice. Backward simulations are sometimes necessary, in particular, when non-deterministic choices are resolved earlier in the specification than in the implementation. In proving implementation relations, we prefer to use forward simulation relations whenever they exist, since backward simulations are harder to think about.

4.5.1 Forward Simulations

Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs. A *forward simulation* from $\mathcal{A}$ to $\mathcal{B}$ is a relation $R \subseteq Q_{\mathcal{A}} \times Q_{\mathcal{B}}$ satisfying the following conditions, for all states $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$ of $\mathcal{A}$ and $\mathcal{B}$, respectively:

1. If $\mathbf{x}_{\mathcal{A}} \in \Theta_{\mathcal{A}}$ then there exists a state $\mathbf{x}_{\mathcal{B}} \in \Theta_{\mathcal{B}}$ such that $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$.
2. If $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$ and α is an execution fragment of $\mathcal{A}$ consisting of one action surrounded by two point trajectories, with $\alpha.fstate = \mathbf{x}_{\mathcal{A}}$, then $\mathcal{B}$ has a closed execution fragment β with $\beta.fstate = \mathbf{x}_{\mathcal{B}}$, $trace(\beta) = trace(\alpha)$, and $\alpha.lstate R \beta.lstate$.
3. If $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$ and α is an execution fragment of $\mathcal{A}$ consisting of a single closed trajectory, with $\alpha.fstate = \mathbf{x}_{\mathcal{A}}$, then $\mathcal{B}$ has a closed execution fragment β with $\beta.fstate = \mathbf{x}_{\mathcal{B}}$, $trace(\beta) = trace(\alpha)$, and $\alpha.lstate R \beta.lstate$.

Forward simulation relations induce a preorder between timed automata.

Theorem 4.20 *Let $\mathcal{A}, \mathcal{B}$ and $\mathcal{C}$ be comparable TAs. If R_1 is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ and R_2 is a forward simulation from $\mathcal{B}$ to $\mathcal{C}$, then $R_2 \circ R_1$ is a forward simulation from $\mathcal{A}$ to $\mathcal{C}$.*

The definition of a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ yields a correspondence for open trajectories of $\mathcal{A}$:

Lemma 4.21 *Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs and let R be a forward simulation from $\mathcal{A}$ to $\mathcal{B}$. Let $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$ be states of $\mathcal{A}$ and $\mathcal{B}$, respectively, such that $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$. Let α be an execution fragment of $\mathcal{A}$ from state $\mathbf{x}_{\mathcal{A}}$ consisting of a single open trajectory. Then $\mathcal{B}$ has an execution fragment β with $\beta.fstate = \mathbf{x}_{\mathcal{B}}$ and $trace(\beta) = trace(\alpha)$.*

Proof: Let τ be the single open trajectory in α . Using axioms **T1** and **T2**, we construct an infinite sequence $\tau_0 \tau_1 \dots$ of closed trajectories of $\mathcal{A}$ such that $\tau = \tau_0 \frown \tau_1 \frown \dots$. Then, working recursively, we construct a sequence $\beta_0 \beta_1 \dots$ of closed execution fragments of $\mathcal{B}$ such that $\beta_0.fstate = \mathbf{x}_{\mathcal{B}}$ and, for each i , $\tau_i.lstate R \beta_i.lstate$, $\beta_i.lstate = \beta_{i+1}.fstate$, and $trace(\tau_i) = trace(\beta_i)$. This construction uses induction on i , using Property 3 of the

definition of a forward simulation in the induction step. Now let $\beta = \beta_0 \frown \beta_1 \frown \dots$. By Lemma 4.7, β is an execution fragment of $\mathcal{B}$. Clearly, $\beta.fstate = \mathbf{x}_{\mathcal{B}}$. By Lemma 3.9 applied to both α and β , $trace(\beta) = trace(\alpha)$. Thus β has the required properties. ■

Theorem 4.22 *Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs and let R be a forward simulation from $\mathcal{A}$ to $\mathcal{B}$. Let $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$ be states of $\mathcal{A}$ and $\mathcal{B}$, respectively, such that $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$. Then $tracefrags_{\mathcal{A}}(\mathbf{x}_{\mathcal{A}}) \subseteq tracefrags_{\mathcal{B}}(\mathbf{x}_{\mathcal{B}})$.*

Proof: Suppose that δ is the trace of an execution fragment of $\mathcal{A}$ that starts from $\mathbf{x}_{\mathcal{A}}$; we prove that δ is also a trace of an execution fragment of $\mathcal{B}$ that starts from $\mathbf{x}_{\mathcal{B}}$. Let $\alpha = \tau_0 a_1 \tau_1 a_2 \tau_2 \dots$ be an execution fragment of $\mathcal{A}$ such that $\alpha.fstate = \mathbf{x}_{\mathcal{A}}$ and $\delta = trace(\alpha)$. We consider cases:

1. α is an infinite sequence.

Using axioms **T1** and **T2**, we can write α as an infinite concatenation $\alpha_0 \frown \alpha_1 \frown \alpha_2 \dots$, in which the execution fragments α_i with i even consist of a trajectory only, and the execution fragments α_i with i odd consist of a single discrete step surrounded by two point trajectories.

We define inductively a sequence $\beta_0 \beta_1 \dots$ of closed execution fragments of $\mathcal{B}$, such that $\beta_0.fstate = \mathbf{x}_{\mathcal{B}}$ and, for all i , $\beta_i.lstate = \beta_{i+1}.fstate$, $\alpha_i.lstate R \beta_i.lstate$, and $trace(\beta_i) = trace(\alpha_i)$. We use Property 3 of the definition of a simulation for the construction of the β_i 's with i even, and Property 2 for the construction of the β_i 's with i odd. Let $\beta = \beta_0 \frown \beta_1 \frown \beta_2 \dots$. By Lemma 4.7, β is an execution fragment of $\mathcal{B}$. Clearly, $\beta.fstate = \mathbf{x}_{\mathcal{B}}$. By Lemma 3.9, $trace(\beta) = trace(\alpha)$. Thus β has the required properties.

2. α is a finite sequence ending with a closed trajectory.

Similar to the first case.

3. α is a finite sequence ending with an open trajectory.

Similar to the first case, using Lemma 4.21. ■

Corollary 4.23 *Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs and let R be a forward simulation from $\mathcal{A}$ to $\mathcal{B}$. Then $traces_{\mathcal{A}} \subseteq traces_{\mathcal{B}}$.*

Proof: Suppose $\beta \in traces_{\mathcal{A}}$. Then $\beta \in tracefrags_{\mathcal{A}}(\mathbf{x}_{\mathcal{A}})$ for some start state $\mathbf{x}_{\mathcal{A}}$ of $\mathcal{A}$. Property 1 of the definition of simulation implies the existence of a start state $\mathbf{x}_{\mathcal{B}}$ of $\mathcal{B}$ such that $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$. Then Theorem 4.22 implies that $\beta \in tracefrags_{\mathcal{B}}(\mathbf{x}_{\mathcal{B}})$. Since $\mathbf{x}_{\mathcal{B}}$ is a start state of $\mathcal{B}$, this implies that $\beta \in traces_{\mathcal{B}}$, as needed. ■

Example 4.24 (Time-bounded channels) Consider two instances of the specification in Figure 1, $\text{TimedChannel}(b1, M)$ and $\text{TimedChannel}(b2, M)$ where $b1 \leq b2$. We define a forward simulation R from $\text{TimedChannel}(b1, M)$ to $\text{TimedChannel}(b2, M)$ below. If $\mathbf{x}$ is a state of $\text{TimedChannel}(b1, M)$ and $\mathbf{y}$ is a state of $\text{TimedChannel}(b2, M)$, then $\mathbf{x} R \mathbf{y}$ provided that the following conditions are satisfied:

1. $\mathbf{x}(\text{now}) = \mathbf{y}(\text{now})$.
2. $|\mathbf{x}(\text{queue})| = |\mathbf{y}(\text{queue})|$.
3. $\forall i. 1 \leq i \leq |\mathbf{x}(\text{queue})|$, if $\mathbf{x}(\text{queue})(i) = (m, u1)$ then $\mathbf{y}(\text{queue})(i) = (m, u2)$ and $u1 \leq u2$.

We can prove that R is a forward simulation from the automaton $\text{TimedChannel}(b1, M)$ to the automaton $\text{TimedChannel}(b2, M)$ by showing that R satisfies each of the three properties in the definition of a forward simulation relation. In each automaton there is a unique initial state that maps the variable `now` to 0 and `queue` to the empty sequence. It is obvious that the initial states, which are identical, are related by R and so the first property is satisfied.

For the rest of the proof, we let $\mathbf{x}$ and $\mathbf{y}$ be, respectively, states of $\text{TimedChannel}(b1, M)$ and $\text{TimedChannel}(b2, M)$ such that $\mathbf{x} R \mathbf{y}$. In order to show that the second property is satisfied, we need to consider two cases, one for each discrete action that may be performed by $\text{TimedChannel}(b1, M)$.

If $\text{TimedChannel}(b1, M)$ performs a `send(m)` action, and the state changes from $\mathbf{x}$ to $\mathbf{x}'$ then we need to find an execution fragment β of $\text{TimedChannel}(b2, M)$ from $\mathbf{y}$ ending in $\mathbf{y}'$, such that $\mathbf{x}' R \mathbf{y}'$ and $\text{trace}(\beta)$ is the same as the trace of $\wp(\mathbf{x}) \text{ send}(m) \wp(\mathbf{y})$. The execution fragment $\beta = \wp(\mathbf{y}) \text{ send}(m) \wp(\mathbf{y}')$ satisfies the required conditions. This follows from the hypothesis that $\mathbf{x} R \mathbf{y}$ and the definition of R , using the fact that the effect of a `send(m)` action of $\text{TimedChannel}(b1, M)$, $\text{TimedChannel}(b2, M)$ are, respectively, adding the entry $(m, \text{now} + b1)$ to $\mathbf{x}(\text{queue})$, and $(m, \text{now} + b2)$ to $\mathbf{y}(\text{queue})$ where $b1 \leq b2$.

If $\text{TimedChannel}(b1, M)$ performs a `receive(m)` action, and the state changes from $\mathbf{x}$ to $\mathbf{x}'$ then we need to show that `receive(m)` is also enabled in $\mathbf{y}$ and that there is an execution fragment with the required properties that ends in a state $\mathbf{y}'$ such that $\mathbf{x}' R \mathbf{y}'$. In order to show that `receive(m)` is enabled in $\mathbf{y}$, we use the hypothesis that $\mathbf{x} R \mathbf{y}$, which implies that the first element of $\mathbf{y}(\text{queue})$ is of the form (m, u) for some u . The execution fragment $\wp(\mathbf{y}) \text{ receive}(m) \wp(\mathbf{y}')$ of $\text{TimedChannel}(b1, M)$ can be shown to satisfy the required conditions.

For the third property, we consider a closed trajectory τ of $\text{TimedChannel}(b1, M)$ with $\tau.\text{fstate} = \mathbf{x}$ and show that there exists a closed execution fragment β of the automaton $\text{TimedChannel}(b2, M)$ with $\beta.\text{fstate} = \mathbf{y}$, $\text{trace}(\beta) = \text{trace}(\tau)$, and $\tau.\text{lstate} = \beta.\text{lstate}$. It is easy to check that the trajectory τ' of $\text{TimedChannel}(b2, M)$ with $\tau'.\text{fstate} = \mathbf{y}$ and $\tau'.\text{ltime} = \tau.\text{ltime}$ satisfies the required conditions. ■

Example 4.25 (Time-bounded channel that keeps all messages) In this example we define a variant of $\text{TimedChannel}(b, M)$ from Example 4.1 called $\text{TimedChannel2}(b, M)$. The main difference between $\text{TimedChannel}(b, M)$ and $\text{TimedChannel2}(b, M)$ is that the message queue in $\text{TimedChannel2}(b, M)$ is implemented using a finite sequence of (message, delivery deadline) pairs `queue` and a pointer `ptr` that points to the next element that is to be delivered. Hence, the internal variables of $\text{TimedChannel2}(b, M)$ consist of `queue`, `now` and `ptr`. The variable `ptr` initially has value 1, which indicates that it is pointing to the first element in the sequence. A `send(m)` action causes messages and deadlines to be added to the sequence as in $\text{TimedChannel}(b, M)$. A `receive(m)` causes `ptr` to be incremented to make it point to the next element in the sequence instead of removing the first element. The automaton $\text{TimedChannel}(b, M)$ can be viewed as an optimized implementation of $\text{TimedChannel2}(b, M)$.

We define below a forward simulation R from $\text{TimedChannel}(b, M)$ to $\text{TimedChannel2}(b, M)$. If $\mathbf{x}$ is a state of $\text{TimedChannel}(b, M)$ and $\mathbf{y}$ is a state of $\text{TimedChannel2}(b, M)$, then $\mathbf{x} R \mathbf{y}$ provided that the following conditions are satisfied:

1. $\mathbf{x}(\text{now}) = \mathbf{y}(\text{now})$.
2. $\mathbf{x}(\text{queue}) = \mathbf{y}(\text{queue})(\mathbf{y}(\text{ptr}) \dots |\mathbf{y}(\text{queue})|)$.

■

Example 4.26 (Clock synchronization)

In this example, we define a forward simulation from $\text{ClockSync}(u, r: \text{Real}, i: \text{Index})$ of Figure 7 to an automaton that sends multiples of u . The specification of this automaton, which is called $\text{SendVal}(u, r: \text{Real}, i: \text{Index})$ is given in Figure 8. We assume that the definitions of type Index in both automata are identical.

The variable `counter` keeps track of which multiple of u is to be sent next, and variable `now` contains the current time. The automaton parameter r is used in the precondition of the `send` and the stopping condition of the trajectory definition, to enforce bounds on the times of occurrence of `send`.

We now define a forward simulation R from the automaton $\text{ClockSync}(u, r, i)$ to the automaton $\text{SendVal}(u, r, i)$ where u, r and i are actual parameters. If $\mathbf{x}$ is a state of the automaton $\text{ClockSync}(u, r, i)$ and $\mathbf{y}$ is a state of $\text{SendVal}(u, r, i)$, then $\mathbf{x} R \mathbf{y}$ provided that the following conditions are satisfied:

1. $\mathbf{y}(\text{now})(1 - r) \leq \mathbf{x}(\text{physclock}) \leq \mathbf{y}(\text{now})(1 + r)$.
2. $\mathbf{y}(\text{counter}) = \mathbf{x}(\text{nextsend}) / u$.

■

```

automaton SendVal(u,r : Real, i: Index)
  signature
    external send(m: Real),
      receive(m:Real, j: Index, const i) where j  $\neq$  i
  states
    counter: discrete Real := 0,
    now: Real := 0,
    initially u > 0  $\wedge$  (0  $\leq$  r  $\leq$  1)
  transitions
    external send(m,i)
      pre
        m = counter * u  $\wedge$  counter * u / (1 + r)  $\leq$  now
      eff
        counter := counter + 1
    external receive(m,j,i)
  trajectories
    stop when
      now = counter * u / (1 - r)
    evolve
      d(now) = 1

```

Figure 8: Clock synchronization

4.5.2 Refinements

Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs. A *refinement* from $\mathcal{A}$ to $\mathcal{B}$ is a function $F \subseteq Q_{\mathcal{A}} \times Q_{\mathcal{B}}$, satisfying the following conditions, for all states $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$ of $\mathcal{A}$ and $\mathcal{B}$, respectively:

1. If $\mathbf{x}_{\mathcal{A}} \in \Theta_{\mathcal{A}}$ then $F(\mathbf{x}_{\mathcal{A}}) \in \Theta_{\mathcal{B}}$.
2. If α is an execution fragment of $\mathcal{A}$ consisting of one action surrounded by two point trajectories, with $\alpha.fstate = \mathbf{x}_{\mathcal{A}}$, then $\mathcal{B}$ has a closed execution fragment β with $\beta.fstate = F(\mathbf{x}_{\mathcal{A}})$, $trace(\beta) = trace(\alpha)$, and $\beta.lstate = F(\alpha.lstate)$.
3. If α is an execution fragment of $\mathcal{A}$ consisting of a single closed trajectory, with $\alpha.fstate = \mathbf{x}_{\mathcal{A}}$, then $\mathcal{B}$ has a closed execution fragment β with $\beta.fstate = F(\mathbf{x}_{\mathcal{A}})$, $trace(\beta) = trace(\alpha)$, and $\beta.lstate = F(\alpha.lstate)$.

Theorem 4.27 *Let $\mathcal{A}$ and $\mathcal{B}$ be two TAs and suppose $R \subseteq Q_{\mathcal{A}} \times Q_{\mathcal{B}}$. Then R is a refinement from $\mathcal{A}$ to $\mathcal{B}$ if and only if R is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ and R is a function.*

Theorem 4.28 *Let $\mathcal{A}, \mathcal{B}$ and $\mathcal{C}$ be comparable TAs. If R_1 is a refinement from $\mathcal{A}$ to $\mathcal{B}$ and R_2 is a refinement from $\mathcal{B}$ to $\mathcal{C}$, then $R_2 \circ R_1$ is a refinement from $\mathcal{A}$ to $\mathcal{C}$.*

An isomorphism from $\mathcal{A}$ to $\mathcal{B}$ is a refinement F from $\mathcal{A}$ to $\mathcal{B}$ such that F^{-1} is a refinement from $\mathcal{B}$ to $\mathcal{A}$. We say that two automata $\mathcal{A}$ and $\mathcal{B}$ are *isomorphic*, if there exists an isomorphism from $\mathcal{A}$ to $\mathcal{B}$ (or, equivalently from $\mathcal{B}$ to $\mathcal{A}$).

4.5.3 Backward Simulations

Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs. A *backward simulation* from $\mathcal{A}$ to $\mathcal{B}$ is a total relation $R \subseteq Q_{\mathcal{A}} \times Q_{\mathcal{B}}$ satisfying the following conditions, for all states $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$ of $\mathcal{A}$ and $\mathcal{B}$, respectively:

1. If $\mathbf{x}_{\mathcal{A}} \in \Theta_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$ then $\mathbf{x}_{\mathcal{B}} \in \Theta_{\mathcal{B}}$.
2. If $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$ and α is an execution fragment of $\mathcal{A}$ with $\alpha.lstate = \mathbf{x}_{\mathcal{A}}$, consisting of one discrete action surrounded by two point trajectories, then $\mathcal{B}$ has a closed execution fragment β with $\beta.lstate = \mathbf{x}_{\mathcal{B}}$, $trace(\beta) = trace(\alpha)$, and $\alpha.fstate R \beta.fstate$.
3. If $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$ and α is an execution fragment of $\mathcal{A}$ with $\alpha.lstate = \mathbf{x}_{\mathcal{A}}$, consisting of one trajectory, then $\mathcal{B}$ has a closed execution fragment β with $\beta.lstate = \mathbf{x}_{\mathcal{B}}$, $trace(\beta) = trace(\alpha)$, and $\alpha.fstate R \beta.fstate$.

Backward simulations induce a preorder between timed automata.

Theorem 4.29 *Let $\mathcal{A}, \mathcal{B}$ and $\mathcal{C}$ be comparable TAs. If R_1 is a backward simulation from $\mathcal{A}$ to $\mathcal{B}$ and R_2 is a backward simulation $\mathcal{B}$ to $\mathcal{C}$, then $R_2 \circ R_1$ is a backward simulation from $\mathcal{A}$ to $\mathcal{C}$.*

Theorem 4.30 *Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs and let R be a backward simulation from $\mathcal{A}$ to $\mathcal{B}$. Let $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{x}_{\mathcal{B}}$ be states of $\mathcal{A}$ and $\mathcal{B}$, respectively, such that $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$. Let β be the trace of a closed execution fragment of $\mathcal{A}$ from $\mathbf{y}_{\mathcal{A}}$ with last state $\mathbf{x}_{\mathcal{A}}$. Then there exists $\mathbf{y}_{\mathcal{B}}$ such that β is also the trace of a closed execution fragment of $\mathcal{B}$ from $\mathbf{y}_{\mathcal{B}}$ with last state $\mathbf{x}_{\mathcal{B}}$ and $\mathbf{y}_{\mathcal{A}} R \mathbf{y}_{\mathcal{B}}$.*

Proof: Fix some R , $\mathbf{x}_{\mathcal{A}}$, $\mathbf{x}_{\mathcal{B}}$ and β satisfying the conditions in the statement of the theorem. Let $\alpha \in frags_{\mathcal{A}}(\mathbf{y}_{\mathcal{A}})$ for some state $\mathbf{y}_{\mathcal{A}}$ of $\mathcal{A}$ with $trace(\alpha) = \beta$. By using the axioms **T1** and **T2**, we can write α as the concatenation of a sequence of closed execution fragments, $\alpha = \alpha_0 \frown \alpha_1 \frown \dots \frown \alpha_n$, where each α_i is either a closed trajectory or an action surrounded by two point trajectories, and $\alpha_i.lstate = \alpha_{i+1}.fstate$ for $0 \leq i \leq n$.

By using the definition of a backward simulation, working backwards from α_n , we can construct an execution fragment $\alpha' = \alpha'_0 \frown \alpha'_1 \frown \dots \frown \alpha'_n$ from a state $\mathbf{y}_{\mathcal{B}}$ of $\mathcal{B}$ such that (a) $\alpha'.lstate = \mathbf{x}_{\mathcal{B}}$, (b) for all i , $0 \leq i \leq n$, $\alpha_i.fstate R \alpha'_i.fstate$ and $trace(\alpha'_i) = trace(\alpha_i)$, (c) for all i , $0 \leq i \leq n-1$, $\alpha'_i.lstate = \alpha'_{i+1}.fstate$. Using Lemma 4.7, we can see that α' is an execution fragment of $\mathcal{B}$. By Lemma 3.9, $trace(\alpha) = trace(\alpha')$ as needed. ■

Corollary 4.31 *Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs and let R be a backward simulation from $\mathcal{A}$ to $\mathcal{B}$. Then every closed trace of $\mathcal{A}$ is a trace of $\mathcal{B}$.*

Proof: Suppose R is a backward simulation from $\mathcal{A}$ to $\mathcal{B}$ and β is a closed trace of $\mathcal{A}$. Then $\beta = \text{trace}(\alpha)$ for some closed execution α of $\mathcal{A}$. Let $\mathbf{x}_{\mathcal{A}}$ and $\mathbf{y}_{\mathcal{A}}$ be the first and last states of α respectively. By the totality of relation R , there exists some state $\mathbf{y}_{\mathcal{B}}$ of $\mathcal{B}$ such that $\mathbf{y}_{\mathcal{A}} R \mathbf{y}_{\mathcal{B}}$. By Theorem 4.30, there exists $\mathbf{x}_{\mathcal{B}}$ of $\mathcal{B}$ such that β is the trace of a closed execution fragment of $\mathcal{B}$ from $\mathbf{x}_{\mathcal{B}}$ with last state $\mathbf{y}_{\mathcal{B}}$ and $\mathbf{x}_{\mathcal{A}} R \mathbf{x}_{\mathcal{B}}$. Property 1 of the definition of a backward simulation relation implies that $\mathbf{x}_{\mathcal{B}}$ is a start state of $\mathcal{B}$. It follows that $\beta \in \text{traces}_{\mathcal{B}}$, as needed. ■

Theorem 4.32 *Let $\mathcal{A}$ and $\mathcal{B}$ be comparable TAs and let R be an image-finite backward simulation from $\mathcal{A}$ to $\mathcal{B}$. Then $\text{traces}_{\mathcal{A}} \subseteq \text{traces}_{\mathcal{B}}$.*

Proof: Let $\beta \in \text{traces}_{\mathcal{A}}$. If β is closed then Corollary 4.31 implies that β is a trace of $\mathcal{B}$. From now on we assume β is not closed.

Let $\alpha \in \text{execs}_{\mathcal{A}}$ with $\text{trace}(\alpha) = \beta$. Note that any such α is either an infinite sequence $\tau_0 a_1 \tau_1 \dots$ or a finite sequence $\tau_0 a_1 \tau_1 \dots \tau_n$ where the final trajectory τ_n is right open. In either case, using the axioms **T1** and **T2**, we can construct an infinite sequence $\alpha_0 \alpha_1 \dots$ of closed execution fragments such that $\alpha = \alpha_0 \frown \alpha_1 \frown \dots$ where α_0 is a point trajectory, each α_i is either a closed trajectory or an action surrounded by two point trajectories, and $\alpha_i.\text{lstate} = \alpha_{i+1}.\text{fstate}$ for each i , $0 \leq i$.

We construct a directed graph G whose nodes are pairs $(\mathbf{x}, i)$ consisting of a state of $\mathcal{B}$ and an index such that $(\alpha_i.\text{lstate}, \mathbf{x}) \in R$. In G , there is an edge from $(\mathbf{x}, i)$ to $(\mathbf{x}', j)$ exactly if $j = i + 1$ and there is an $\alpha' \in \text{frags}_{\mathcal{B}}(\mathbf{x})$ with $\text{trace}(\alpha') = \text{trace}(\alpha_{i+1})$ such that $\alpha'.\text{lstate} = \mathbf{x}'$. Since R is image-finite there are finitely many roots of G . By image-finiteness of R and the definition of the edge set, each node has finite outdegree. By using the definition of a backward simulation and the edge set of G , we can show that each node $(\mathbf{x}, i)$ is reachable from some root node $(\mathbf{z}, 0)$ for some start state $\mathbf{z}$ of $\mathcal{B}$.

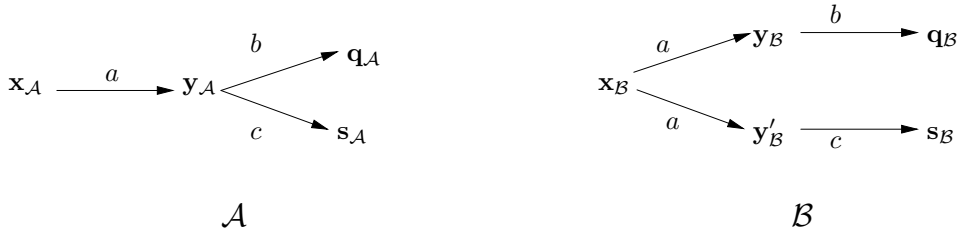
The directed graph G satisfies the hypotheses of Lemma 2.3, which implies that there is an infinite path in G starting from a root. An edge from a node $(\mathbf{x}, i)$ to $(\mathbf{x}', i + 1)$ along this infinite path corresponds to a closed execution fragment γ_{i+1} of $\mathcal{B}$ for i , $0 \leq i$ such that $\gamma_{i+1}.\text{fstate} = \mathbf{x}$, $\gamma_{i+1}.\text{lstate} = \mathbf{x}'$ and $\text{trace}(\gamma_{i+1}) = \text{trace}(\alpha_{i+1})$. By Lemma 4.7, $\gamma = \gamma_1 \frown \gamma_2 \frown \dots$ is an execution of $\mathcal{B}$ and by Lemma 3.9, $\text{trace}(\gamma) = \text{trace}(\gamma_1) \frown \text{trace}(\gamma_2) \frown \dots$. Since $\text{trace}(\gamma_{i+1}) = \text{trace}(\alpha_{i+1})$ for all i , $0 \leq i$, and α_0 is a point trajectory, by Lemma 3.9, we get $\text{trace}(\gamma) = \text{trace}(\alpha) = \beta$. ■

Example 4.33 (A backward simulation relation) This example illustrates the difference between forward and backward simulations. We consider two automata $\mathcal{A}$ and $\mathcal{B}$ and show that a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ does not exist while we exhibit a backward simulation from $\mathcal{A}$ to $\mathcal{B}$.

Let $\mathcal{A}$ and $\mathcal{B}$ be two comparable automata specified below. The trajectories consist of a set of point trajectories. This implies that the automaton does not allow time to pass — everything happens at time 0.

- $V_{\mathcal{A}} = \{stateA\}$ and $V_{\mathcal{B}} = \{stateB\}$ where:
 $stateA$ is a discrete variable with $type(stateA) = \{x_{\mathcal{A}}, y_{\mathcal{A}}, q_{\mathcal{A}}, s_{\mathcal{A}}\}$, and
 $stateB$ is a discrete variable with $type(stateB) = \{x_{\mathcal{B}}, y_{\mathcal{B}}, y'_{\mathcal{B}}, q_{\mathcal{B}}, s_{\mathcal{B}}\}$.
- $Q_{\mathcal{A}} = val(V_{\mathcal{A}})$ and $Q_{\mathcal{B}} = val(V_{\mathcal{B}})$. We write $\mathbf{x}_{\mathcal{A}}$ for the valuation that maps $stateA$ to $x_{\mathcal{A}}$, $\mathbf{y}_{\mathcal{A}}$ for the valuation that maps $stateA$ to $y_{\mathcal{A}}$, etc. Similarly, we write $\mathbf{x}_{\mathcal{B}}$ for the valuation that maps $stateB$ to $x_{\mathcal{B}}$, $\mathbf{y}_{\mathcal{B}}$ for the valuation that maps $stateB$ to $y_{\mathcal{B}}$, etc.
- $\Theta_{\mathcal{A}} = \{\mathbf{x}_{\mathcal{A}}\}$ and $\Theta_{\mathcal{B}} = \{\mathbf{x}_{\mathcal{B}}\}$.
- $E_{\mathcal{A}} = E_{\mathcal{B}} = \{a, b, c\}$ and $H_{\mathcal{A}} = H_{\mathcal{B}} = \emptyset$.
- $\mathcal{D}_{\mathcal{A}} = \{(\mathbf{x}_{\mathcal{A}}, a, \mathbf{y}_{\mathcal{A}}), (\mathbf{y}_{\mathcal{A}}, b, \mathbf{q}_{\mathcal{A}}), (\mathbf{y}_{\mathcal{A}}, c, \mathbf{s}_{\mathcal{A}})\}$, and
 $\mathcal{D}_{\mathcal{B}} = \{(\mathbf{x}_{\mathcal{B}}, a, \mathbf{y}_{\mathcal{B}}), (\mathbf{x}_{\mathcal{B}}, a, \mathbf{y}'_{\mathcal{B}}), (\mathbf{y}_{\mathcal{B}}, b, \mathbf{q}_{\mathcal{B}}), (\mathbf{y}'_{\mathcal{B}}, c, \mathbf{s}_{\mathcal{B}})\}$.
- $\mathcal{T}_{\mathcal{A}} = \{\wp(\mathbf{v}) \mid \mathbf{v} \in Q_{\mathcal{A}}\}$, and $\mathcal{T}_{\mathcal{B}} = \{\wp(\mathbf{v}) \mid \mathbf{v} \in Q_{\mathcal{B}}\}$

The following are representations of automata $\mathcal{A}$ and $\mathcal{B}$ as directed multigraphs. The nodes in the graph represent states and the edges represent discrete transitions where a label on an edge stands for the action involved in the transition.



An obvious candidate for a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ is the relation $R = \{(\mathbf{x}_{\mathcal{A}}, \mathbf{x}_{\mathcal{B}}), (\mathbf{y}_{\mathcal{A}}, \mathbf{y}_{\mathcal{B}}), (\mathbf{y}_{\mathcal{A}}, \mathbf{y}'_{\mathcal{B}}), (\mathbf{q}_{\mathcal{A}}, \mathbf{q}_{\mathcal{B}}), (\mathbf{s}_{\mathcal{A}}, \mathbf{s}_{\mathcal{B}})\}$. However, observe that even though $\mathbf{y}_{\mathcal{A}}$ and $\mathbf{y}_{\mathcal{B}}$ are related by R , the execution fragment $\wp(\mathbf{y}_{\mathcal{A}}) \ c \ \wp(\mathbf{s}_{\mathcal{A}})$ of $\mathcal{A}$ cannot be matched by any execution fragment of $\mathcal{B}$ starting with state $\mathbf{y}_{\mathcal{B}}$. Similarly, even though $\mathbf{y}_{\mathcal{A}}$ and $\mathbf{y}'_{\mathcal{B}}$ are related by R , the execution fragment $\wp(\mathbf{y}_{\mathcal{A}}) \ b \ \wp(\mathbf{q}_{\mathcal{A}})$ of $\mathcal{A}$ cannot be matched by any execution fragment of $\mathcal{B}$ starting with $\mathbf{y}'_{\mathcal{B}}$. Therefore, R is not a forward simulation. In fact, there is no forward simulation relation from $\mathcal{A}$ to $\mathcal{B}$: there are finitely

many possibilities for forward simulations from $\mathcal{A}$ to $\mathcal{B}$ and we see that none of them is a forward simulation by examining all the possibilities. The main reason for this is that while $\mathcal{A}$ makes the nondeterministic choice between performing b or c after performing a , $\mathcal{B}$ makes its choice earlier at the same time it performs a .

There is, however, a backward simulation from $\mathcal{A}$ to $\mathcal{B}$: the relation R defined above is a backward simulation. ■

4.5.4 History Relations

A relation $R \subseteq Q_{\mathcal{A}} \times Q_{\mathcal{B}}$ is a *history relation* from $\mathcal{A}$ to $\mathcal{B}$ if R is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ and R^{-1} is a refinement from $\mathcal{B}$ to $\mathcal{A}$. History relations induce a preorder between timed automata.

An automaton $\mathcal{B}$ is obtained from an automaton $\mathcal{A}$ by *adding history variables* if there exists a set of variables V such that

1. $V_{\mathcal{B}} = V_{\mathcal{A}} \cup V$ and $V_{\mathcal{A}} \cap V = \emptyset$,
2. $Q_{\mathcal{B}} \subseteq \text{val}(V_{\mathcal{B}})$ such that $Q_{\mathcal{B}} \upharpoonright V_{\mathcal{A}} \subseteq Q_{\mathcal{A}}$, and
3. The relation $\{(\mathbf{x}, \mathbf{y}) \mid \mathbf{y} \in Q_{\mathcal{B}} \text{ and } \mathbf{y} \upharpoonright V_{\mathcal{A}} = \mathbf{x}\}$ is a history relation from $\mathcal{A}$ to $\mathcal{B}$.

The method of adding history variables is typically used to make it possible to establish an implementation relationship using a refinement. If a refinement does not exist from a low-level automaton to a higher-level one, it can often be made to exist by adding history variables to the low-level automaton.

Example 4.34 (Adding history variables to obtain a refinement) We cannot show that $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ is an implementation of $\text{TimedChannel2}(\mathbf{b}, \mathbf{M})$ from Example 4.25 by using a refinement. This is because we have no way of specifying what the subsequence before the pointer should be in $\text{TimedChannel2}(\mathbf{b}, \mathbf{M})$ when relating the states of the two automata. This example shows how we can add history variables to $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ (actually, we add just one variable) to obtain a new automaton that is related to $\text{TimedChannel2}(\mathbf{b}, \mathbf{M})$ by a refinement.

Let log be a discrete variable whose static type is the same as the static type of queue in $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ and let the initial value of log be the empty sequence. We define a new automaton $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ whose set of variables consists of the variables of $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ and the variable log . The rest of the definition of $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ is the same as $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ except for the transition definition for $\text{receive}(\mathbf{m})$. A $\text{receive}(\mathbf{m})$ event in $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ not only removes the first message from the message queue but also appends this message to the sequence contained in log .

Let V_1, V_2 be the set of variables and Q_1, Q_2 be the set of states of $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ and $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ respectively. It is easy to verify that the relation $\{(\mathbf{x}, \mathbf{y}) \mid \mathbf{y} \in Q_2 \text{ and } \mathbf{y} \upharpoonright V_1 = \mathbf{x}\}$ is a history relation from $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ to $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$. This means that $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ is obtained from $\text{TimedChannel}(\mathbf{b}, \mathbf{M})$ by adding a history variable.

We now define a refinement F from $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ to $\text{TimedChannel2}(\mathbf{b}, \mathbf{M})$ as follows. In our definition we assume the following conventions. Concatenation on the left corresponds to putting an element on the front of a queue. Recall also that we use juxtaposition for concatenation of sequences. If $\mathbf{x}$ is a state of $\text{TimedChannelH}(\mathbf{b}, \mathbf{M})$ and $\mathbf{y}$ is a state of $\text{TimedChannel2}(\mathbf{b}, \mathbf{M})$, then $F(\mathbf{x}) = \mathbf{y}$ where:

1. $\mathbf{y}(\text{now}) = \mathbf{x}(\text{now})$.
2. $\mathbf{y}(\text{queue}) = \mathbf{x}(\text{log}) \frown \mathbf{x}(\text{queue})$ such that $|\mathbf{x}(\text{log})| = \mathbf{y}(\text{ptr}) - 1$.

■

Whenever an automaton $\mathcal{B}$ is obtained from $\mathcal{A}$ by adding history variables, then there exists a history relation from $\mathcal{A}$ to $\mathcal{B}$ by definition. Theorem 4.35 states that the converse also holds, if isomorphic automata are considered.

Theorem 4.35 *Let $\mathcal{A}$ and $\mathcal{B}$ be two comparable TAs such that $V_{\mathcal{A}}$ and $V_{\mathcal{B}}$ are disjoint. Suppose that there is a history relation from $\mathcal{A}$ to $\mathcal{B}$. Then, there exists an automaton $\mathcal{C}$ that is isomorphic to $\mathcal{B}$ and is obtained from $\mathcal{A}$ by adding history variables.*

Proof: Let R be a history relation from $\mathcal{A}$ to $\mathcal{B}$. Define automaton $\mathcal{C}$ as follows:

- $V_{\mathcal{C}} = V_{\mathcal{A}} \cup V_{\mathcal{B}}$.
- $Q_{\mathcal{C}} = \{\mathbf{x} \in \text{val}(V_{\mathcal{C}}) \mid (\mathbf{x} \upharpoonright V_{\mathcal{A}}, \mathbf{x} \upharpoonright V_{\mathcal{B}}) \in R\}$.
- $\Theta_{\mathcal{C}} = \{\mathbf{x} \in Q_{\mathcal{C}} \mid \mathbf{x} \upharpoonright V_{\mathcal{B}} \in \Theta_{\mathcal{B}}\}$.
- $E_{\mathcal{C}} = E_{\mathcal{B}}$ and $H_{\mathcal{C}} = H_{\mathcal{B}}$.
- $\mathbf{x} \xrightarrow{a}_{\mathcal{C}} \mathbf{y}$ if and only if $\mathbf{x} \upharpoonright V_{\mathcal{B}} \xrightarrow{a}_{\mathcal{B}} \mathbf{y} \upharpoonright V_{\mathcal{B}}$.
- $\mathbf{x} \xrightarrow{\tau}_{\mathcal{C}} \mathbf{y}$ if and only if $\mathbf{x} \upharpoonright V_{\mathcal{B}} \xrightarrow{\tau_1}_{\mathcal{B}} \mathbf{y} \upharpoonright V_{\mathcal{B}}$ where $\tau_1 = \tau \downarrow V_{\mathcal{B}}$.

Let $F : Q_{\mathcal{C}} \rightarrow Q_{\mathcal{B}}$ be defined such that $F(\mathbf{x}) = \mathbf{x} \upharpoonright V_{\mathcal{B}}$ for all $\mathbf{x} \in Q_{\mathcal{C}}$. The function F is an isomorphism from $\mathcal{C}$ to $\mathcal{B}$: It is easy to check that F is a refinement from $\mathcal{C}$ to $\mathcal{B}$. We can also easily verify that F^{-1} is a refinement from $\mathcal{B}$ to $\mathcal{C}$, by definition of $\mathcal{C}$ and the fact that R^{-1} is a function from the states of $\mathcal{B}$ to the states of $\mathcal{A}$.

Now, we verify that $\mathcal{C}$ is obtained from $\mathcal{A}$ by adding history variables. Let $V_{\mathcal{B}}$ be the variable set V required in the definition of a history variable and let $R' = \{(\mathbf{x}, \mathbf{y}) \mid \mathbf{y} \in Q_{\mathcal{C}} \wedge \mathbf{y} \upharpoonright V_{\mathcal{A}} = \mathbf{x}\}$. We need to show that R' is a history relation from $\mathcal{A}$ to $\mathcal{C}$.

1. R' is a forward simulation from $\mathcal{A}$ to $\mathcal{C}$.

By definitions of the relations F , R' and the automaton $\mathcal{C}$, $R' = F^{-1} \circ R$. Since F^{-1} is a refinement from $\mathcal{B}$ to $\mathcal{C}$, by Theorem 4.27, we know that it is a forward simulation from $\mathcal{B}$ to $\mathcal{C}$. Since R is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$, by Theorem 4.20 we have R' is a forward simulation from $\mathcal{A}$ to $\mathcal{C}$, as needed.

2. R'^{-1} is a refinement from $\mathcal{C}$ to $\mathcal{A}$.

By definitions of the relations F , R' and the automaton $\mathcal{C}$, $R'^{-1} = R^{-1} \circ F$. Since F is a refinement from $\mathcal{C}$ to $\mathcal{B}$ and R^{-1} is a refinement from $\mathcal{B}$ to $\mathcal{A}$, by Theorem 4.28, we have R'^{-1} is a refinement from $\mathcal{C}$ to $\mathcal{A}$, as needed.

■

The following theorem shows that forward simulations are essentially the same as history relations combined with refinements.

Theorem 4.36 *Let $\mathcal{A}$ and $\mathcal{B}$ be two comparable TAs such that $V_{\mathcal{A}}$ and $V_{\mathcal{B}}$ are disjoint. There is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$ if and only if there exists a TA $\mathcal{C}$ such that there is a history relation from $\mathcal{A}$ to $\mathcal{C}$ and a refinement from $\mathcal{C}$ to $\mathcal{B}$.*

Proof: To prove the implication $\Rightarrow$, suppose R is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$. Let $\mathcal{C}$ be an automaton defined as follows:

- $V_{\mathcal{C}} = V_{\mathcal{A}} \cup V_{\mathcal{B}}$.
- $Q_{\mathcal{C}} = \{\mathbf{x} \in \text{val}(V_{\mathcal{C}}) \mid (\mathbf{x} \upharpoonright V_{\mathcal{A}}, \mathbf{x} \upharpoonright V_{\mathcal{B}}) \in R\}$.
- $\Theta_{\mathcal{C}} = \{\mathbf{x} \in Q_{\mathcal{C}} \mid \mathbf{x} \upharpoonright V_{\mathcal{A}} \in \Theta_{\mathcal{A}} \wedge \mathbf{x} \upharpoonright V_{\mathcal{B}} \in \Theta_{\mathcal{B}}\}$.
- $E_{\mathcal{C}} = E_{\mathcal{A}}$ and $H_{\mathcal{C}} = H_{\mathcal{A}}$.
- $\mathbf{x} \xrightarrow{a}_{\mathcal{C}} \mathbf{y}$ if and only if both of the following conditions hold:
 1. $\mathbf{x} \upharpoonright V_{\mathcal{A}} \xrightarrow{a}_{\mathcal{A}} \mathbf{y} \upharpoonright V_{\mathcal{A}}$.
 2. There exists an execution fragment β of $\mathcal{B}$ such that $\beta.\text{fstate} = \mathbf{x} \upharpoonright V_{\mathcal{B}}$, $\beta.\text{lstate} = \mathbf{y} \upharpoonright V_{\mathcal{B}}$, and $\text{trace}(\beta) = \text{trace}(\wp(\mathbf{x}) \ a \ \wp(\mathbf{y}))$.
- $\mathbf{x} \xrightarrow{\tau}_{\mathcal{C}} \mathbf{y}$ if and only if both of the following conditions hold:
 1. $\tau_1 = \tau \downarrow V_{\mathcal{A}} \in \mathcal{T}_{\mathcal{A}}$ and $\mathbf{x} \upharpoonright V_{\mathcal{A}} \xrightarrow{\tau}_{\mathcal{A}} \mathbf{y} \upharpoonright V_{\mathcal{A}}$.
 2. $\tau_2 = \tau \downarrow V_{\mathcal{B}} \in \mathcal{T}_{\mathcal{B}}$ and $\mathbf{x} \upharpoonright V_{\mathcal{B}} \xrightarrow{a}_{\mathcal{B}} \mathbf{y} \upharpoonright V_{\mathcal{B}}$.

Let π_A and π_B be the functions that restrict states of C to, respectively, V_A and V_B . It follows from the definitions that π_A^{-1} is a history relation from $\mathcal{A}$ to $\mathcal{C}$ and π_B is a refinement from $\mathcal{C}$ to $\mathcal{B}$.

For the implication $\Leftarrow$, suppose that there is a history relation from $\mathcal{A}$ to $\mathcal{C}$ and that there is a refinement from $\mathcal{C}$ to $\mathcal{B}$. Then, by definition of a history relation, we know that there is a forward simulation from $\mathcal{A}$ to $\mathcal{C}$. We also know that there is a forward simulation from $\mathcal{C}$ to $\mathcal{B}$ by Theorem 4.27. It follows that there is a forward simulation from $\mathcal{A}$ to $\mathcal{B}$, as needed. ■

Example 4.37 (Theorem 4.36 applied to time-bounded channels) In Example 4.25, we demonstrated a forward simulation from the automaton `TimedChannel(b, M)` to the automaton `TimedChannel2((), b, M)`. Theorem 4.36 implies that there exists an automaton $\mathcal{A}$ such that there is a history relation from `TimedChannel(b, M)` to $\mathcal{A}$ and a refinement from $\mathcal{A}$ to `TimedChannelPointer(b, M)`. The automaton `TimedChannelH(b, M)` from Example 4.34 is a witness for $\mathcal{A}$. ■

4.5.5 Prophecy Relations

A relation $R \subseteq Q_A \times Q_B$ is a *prophecy relation* from $\mathcal{A}$ to $\mathcal{B}$ if R is a backward simulation from $\mathcal{A}$ to $\mathcal{B}$ and R^{-1} is a refinement from $\mathcal{B}$ to $\mathcal{A}$. Prophecy relations induce a preorder between timed automata.

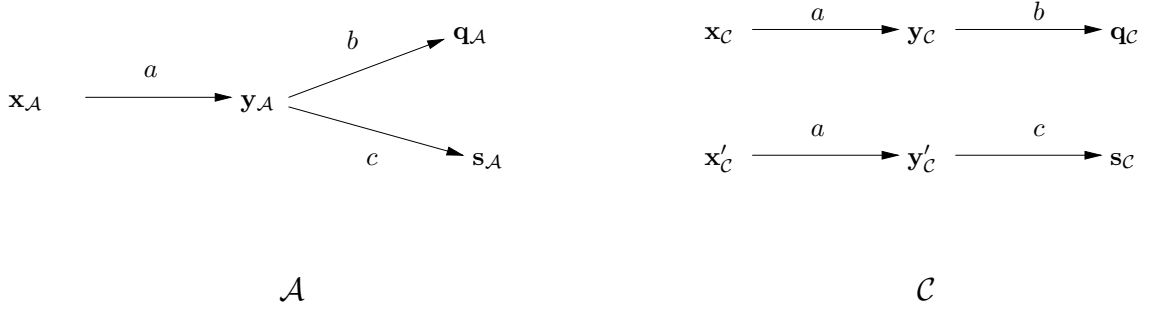
An automaton $\mathcal{B}$ is obtained from an automaton $\mathcal{A}$ by *adding prophecy variables* if there exists a set of variables V such that

1. $V_B = V_A \cup V$ and $V_A \cap V = \emptyset$,
2. $Q_B \subseteq \text{val}(V_B)$ such that $Q_B \upharpoonright V_A \subseteq Q_A$, and
3. The relation $\{(\mathbf{x}, \mathbf{y}) \mid \mathbf{y} \in Q_B \text{ and } \mathbf{y} \upharpoonright V_A = \mathbf{x}\}$ is a prophecy relation from $\mathcal{A}$ to $\mathcal{B}$.

Example 4.38 (Adding prophecy variables to obtain a refinement) In this example we consider adding a prophecy variable to the automaton $\mathcal{A}$ from Example 4.33. Let $\mathcal{C}$ be an automaton defined as follows:

- $V_C = V_A \cup \{v\}$ where v is a discrete variable with $\text{type}(v) = \{b, c\}$.

- $Q_C = \{\mathbf{x}_C, \mathbf{x}'_C, \mathbf{y}_C, \mathbf{y}'_C, \mathbf{q}_C, \mathbf{s}_C\}$ such that
 - $\mathbf{x}_C \models V_A = \mathbf{x}_A$ and $\mathbf{x}_C \models \{v\} = b$
 - $\mathbf{x}'_C \models V_A = \mathbf{x}_A$ and $\mathbf{x}'_C \models \{v\} = c$
 - $\mathbf{y}_C \models V_A = \mathbf{y}_A$ and $\mathbf{y}_C \models \{v\} = b$
 - $\mathbf{y}'_C \models V_A = \mathbf{y}_A$ and $\mathbf{y}'_C \models \{v\} = c$
 - $\mathbf{q}_C \models V_A = \mathbf{q}_A$ and $\mathbf{q}_C \models \{v\} = b$
 - $\mathbf{s}_C \models V_A = \mathbf{s}_A$ and $\mathbf{s}_C \models \{v\} = c$
- $\Theta_C = \{\mathbf{x}_C, \mathbf{x}'_C\}$.
- $E_C = \{a, b, c\}$.
- $\mathcal{D}_C = \{(\mathbf{x}_C, a, \mathbf{y}_C), (\mathbf{x}'_C, a, \mathbf{y}'_C), (\mathbf{y}_C, b, \mathbf{q}_C), (\mathbf{y}'_C, c, \mathbf{s}_C)\}$.
- $\mathcal{T}_C = \{\wp(\mathbf{v}) \mid \mathbf{v} \in Q_C\}$.



The relation $R = \{(\mathbf{x}_A, \mathbf{x}_C), (\mathbf{x}_A, \mathbf{x}'_C), (\mathbf{y}_A, \mathbf{y}_C), (\mathbf{y}_A, \mathbf{y}'_C), (\mathbf{q}_A, \mathbf{q}_C), (\mathbf{s}_A, \mathbf{s}_C)\}$ is a backward simulation from $\mathcal{A}$ to $\mathcal{C}$ and R^{-1} is a refinement. Therefore, $\mathcal{C}$ is obtained by adding a prophecy variable to $\mathcal{A}$. Note that there is no refinement from $\mathcal{A}$ to $\mathcal{B}$ defined in Example 4.33. However, the relation $F = \{(\mathbf{x}_C, \mathbf{x}_B), (\mathbf{x}'_C, \mathbf{x}_B), (\mathbf{y}_C, \mathbf{y}_B), (\mathbf{y}'_C, \mathbf{y}'_B), (\mathbf{q}_C, \mathbf{q}_B), (\mathbf{s}_C, \mathbf{s}_B)\}$ is a refinement from $\mathcal{C}$ to $\mathcal{B}$. ■

Theorem 4.39 *Let $\mathcal{A}$ and $\mathcal{B}$ be two comparable TAs such that V_A and V_B are disjoint. Suppose that there is a prophecy relation from $\mathcal{A}$ to $\mathcal{B}$. Then, there exists an automaton $\mathcal{C}$ that is isomorphic to $\mathcal{B}$ and is obtained from $\mathcal{A}$ by adding prophecy variables.*

Proof: The proof is analogous to the proof of Theorem 4.35. We assume a backward simulation relation R instead of a forward simulation relation. We construct the automaton $\mathcal{C}$ as in Theorem 4.35 and verify that it is obtained from $\mathcal{A}$ by adding a prophecy variable. ■

Theorem 4.40 *Let $\mathcal{A}$ and $\mathcal{B}$ be two comparable TAs such that $V_{\mathcal{A}}$ and $V_{\mathcal{B}}$ are disjoint. There is a backward simulation from $\mathcal{A}$ to $\mathcal{B}$ if and only if there exists a TA $\mathcal{C}$ such that there is a prophecy relation from $\mathcal{A}$ to $\mathcal{C}$ and a refinement from $\mathcal{C}$ to $\mathcal{B}$.*

Proof: The proof is analogous to the proof of Theorem 4.36. We assume a backward simulation relation R instead of a forward simulation. The construction of the automaton $\mathcal{C}$ and the reasoning that follows are similar. ■

Example 4.41 (Theorem 4.40 applied to Examples 4.33 and 4.38) In Example 4.33, we demonstrated a backward simulation from $\mathcal{A}$ to $\mathcal{B}$. Theorem 4.40 implies that there exists an automaton $\mathcal{C}$ such that there is a prophecy relation from $\mathcal{A}$ to $\mathcal{C}$ and a refinement from $\mathcal{C}$ to $\mathcal{B}$. The automaton $\mathcal{C}$ defined in Example 4.38 constitutes a witness for $\mathcal{C}$. ■

5 Operations on Timed Automata

In this section, we introduce four kinds of operations on timed automata: parallel composition, hiding, adding lower and upper bounds for tasks, and untiming.

5.1 Composition

5.1.1 Definitions and Basic Results

The composition operation for timed automata allows an automaton representing a complex system to be constructed by composing automata representing individual system components. Our composition operation identifies external actions with the same name in different component automata. When any component automaton performs a discrete step involving an action a , so do all component automata that have a as an external action. The composition operator for timed automata is simpler than it is for general hybrid automata since all the variables in a timed automaton are internal.²

Formally, we say that timed automata $\mathcal{A}_1$ and $\mathcal{A}_2$ are *compatible* if $H_1 \cap A_2 = H_2 \cap A_1 = \emptyset$ and $X_1 \cap X_2 = \emptyset$. If $\mathcal{A}_1$ and $\mathcal{A}_2$ are compatible then their *composition* $\mathcal{A}_1 \parallel \mathcal{A}_2$ is defined to be the structure $\mathcal{A} = (X, Q, \Theta, E, H, \mathcal{D}, \mathcal{T})$ where

- $X = X_1 \cup X_2$.
- $Q = \{\mathbf{x} \in \text{val}(X) \mid \mathbf{x} \upharpoonright X_i \in Q_i, i \in \{1, 2\}\}$.

²The composition operation for general hybrid automata requires external variables to be identified as well as external actions. When any component automaton follows a particular trajectory for an external variable v , then so do all component automata of which v is an external variable.

- $\Theta = \{\mathbf{x} \in Q \mid \mathbf{x} \upharpoonright X_i \in \Theta_i, i \in \{1, 2\}\}$.
- $E = E_1 \cup E_2$ and $H = H_1 \cup H_2$.
- For each $\mathbf{x}, \mathbf{x}' \in Q$ and each $a \in A$, $\mathbf{x} \xrightarrow{a}_{\mathcal{A}} \mathbf{x}'$ iff for $i \in \{1, 2\}$, either (1) $a \in A_i$ and $\mathbf{x} \upharpoonright X_i \xrightarrow{a}_i \mathbf{x}' \upharpoonright X_i$, or (2) $a \notin A_i$ and $\mathbf{x} \upharpoonright X_i = \mathbf{x}' \upharpoonright X_i$.
- $\mathcal{T} \subseteq \text{trajs}(X)$ is given by $\tau \in \mathcal{T} \Leftrightarrow \tau \downarrow X_i \in \mathcal{T}_i, i \in \{1, 2\}$.

Theorem 5.1 *If $\mathcal{A}_1$ and $\mathcal{A}_2$ are timed automata then $\mathcal{A}_1 \parallel \mathcal{A}_2$ is a timed automaton.*

Lemma 5.2 *Let $\mathcal{A} = \mathcal{A}_1 \parallel \mathcal{A}_2$ and let α be an execution fragment of $\mathcal{A}$. Then $\alpha \upharpoonright (A_1, X_1)$ and $\alpha \upharpoonright (A_2, X_2)$ are execution fragments of $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively. Furthermore,*

1. α is time-bounded iff both $\alpha \upharpoonright (A_1, X_1)$ and $\alpha \upharpoonright (A_2, X_2)$ are time-bounded.
2. α is admissible iff both $\alpha \upharpoonright (A_1, X_1)$ and $\alpha \upharpoonright (A_2, X_2)$ are admissible.
3. α is closed iff both $\alpha \upharpoonright (A_1, X_1)$ and $\alpha \upharpoonright (A_2, X_2)$ are closed.
4. α is non-Zeno iff both $\alpha \upharpoonright (A_1, X_1)$ and $\alpha \upharpoonright (A_2, X_2)$ are non-Zeno.
5. α is an execution iff both $\alpha \upharpoonright (A_1, X_1)$ and $\alpha \upharpoonright (A_2, X_2)$ are executions.

Lemma 5.3 *Let $\mathcal{A} = \mathcal{A}_1 \parallel \mathcal{A}_2$, and let α be an execution fragment of $\mathcal{A}$. Then, for $i = 1, 2$, $\text{trace}(\alpha) \upharpoonright (E_i, \emptyset) = \text{trace}(\alpha \upharpoonright (A_i, X_i))$.*

The following theorem is a fundamental theorem that relates the set of traces of a composed automaton to the sets of traces of its components. Set inclusion in one direction expresses the idea that a trace of a composition “projects” to yield traces of the components. Set inclusion in the other direction expresses the idea that traces of components can be “pasted” to yield a trace of the composition.

Theorem 5.4 *Let $\mathcal{A} = \mathcal{A}_1 \parallel \mathcal{A}_2$. Then $\text{traces}_{\mathcal{A}}$ is exactly the set of $(E, \emptyset)$ -sequences whose restrictions to $\mathcal{A}_1$ and $\mathcal{A}_2$ are traces of $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively. That is, $\text{traces}_{\mathcal{A}} = \{\beta \mid \beta \text{ is an } (E, \emptyset)\text{-sequence and } \beta \upharpoonright (E_i, \emptyset) \in \text{traces}_{\mathcal{A}_i}, i \in \{1, 2\}\}$.*

Notation: The compatibility conditions for composition require the set of internal variables of each automaton to be disjoint from the set of internal variables of all the other automata in the composition. We use a general scheme to disambiguate the internal variables of components in order to avoid possible name clashes that can violate the compatibility conditions. If $\mathcal{A}$ is the name of an automaton and v is an internal variable of $\mathcal{A}$, then we refer to this variable as $\mathcal{A}.v$ in the composite automaton.

Example 5.5 (Periodic sending process with timeouts) Let $\mathcal{C}$ be the composition of three automata from Examples 4.1, 4.2 and 4.4:

$$\mathcal{C} = \text{PeriodicSend}(u1, M) \parallel \text{TimedChannel}(b, M) \parallel \text{Timeout}(u2, M)$$

where $M = \{m1, \dots, mn\}$ and $b + u1 < u2$. The following sequence is a trace of $\mathcal{C}$.

$$\alpha = \tau_0 \text{ send}(m1) \tau_1 \text{ receive}(m1) \tau_2 \text{ send}(m2) \tau_3 \text{ receive}(m2) \tau_4 \dots$$

where e is the set consisting of the function with the empty domain and

$$\tau_0 : [0, u1] \rightarrow e \quad \tau_1 : [0, b] \rightarrow e \quad \tau_2 : [0, u1 - b] \rightarrow e \quad \tau_3 : [0, b] \rightarrow e \quad \tau_4 : [0, u1 - b] \rightarrow e$$

The following invariant states that $\mathcal{C}$ never performs a `timeout` action.

Invariant 1 : In any reachable state x of $\mathcal{C}$, $x(\text{Timeout.suspected}) = \text{false}$.

In order to prove this invariant we can use an auxiliary invariant such as the one below, which establishes the fact that every message is delivered before the variable `now`, which keeps track of real-time, reaches the point at which a `timeout` action occurs.

Invariant 2 :

1. if $x(\text{TimedChannel.queue})$ is not empty then
 $x(\text{TimedChannel.queue})[1] < x(\text{TimedChannel.now} + u2) - x(\text{Timeout.clock})$.
2. if $x(\text{TimedChannel.queue})$ is empty then
 $u1 - x(\text{PeriodicSend.clock}) + b < u2 - x(\text{Timeout.clock})$.

■

Example 5.6 (Periodic sending process with failures and timeouts) In this example, we consider a composite automaton defined exactly like the one in Example 5.5 except that the automaton `PeriodicSend(u1, M)` is replaced with `PeriodicSend2(u1, M)`. Let $\mathcal{C} = \text{PeriodicSend2}(u1, M) \parallel \text{TimedChannel}(b, M) \parallel \text{Timeout}(u2, M)$. The following sequence is a trace of $\mathcal{C}$.

$$\tau_0 \text{ send}(m1) \tau_1 \text{ receive}(m1) \tau_2 \text{ fail} \tau_3 \text{ timeout} \tau_4$$

where e is the set consisting of the function with the empty domain and

$$\tau_0 : [0, u1] \rightarrow e \quad \tau_1 : [0, b] \rightarrow e \quad \tau_2 : [0, b] \rightarrow e \quad \tau_3 : [0, u2-b] \rightarrow e \quad \tau_4 : [0, \infty) \rightarrow e$$

According to this sample trace, the first message sent by the periodic sending process is received exactly b time units after it is sent. The periodic sending process fails $2 \times b$ time units after sending its first message. The timeout process performs a timeout since no second message arrives within the next u_2 time units after the receipt of the first message.

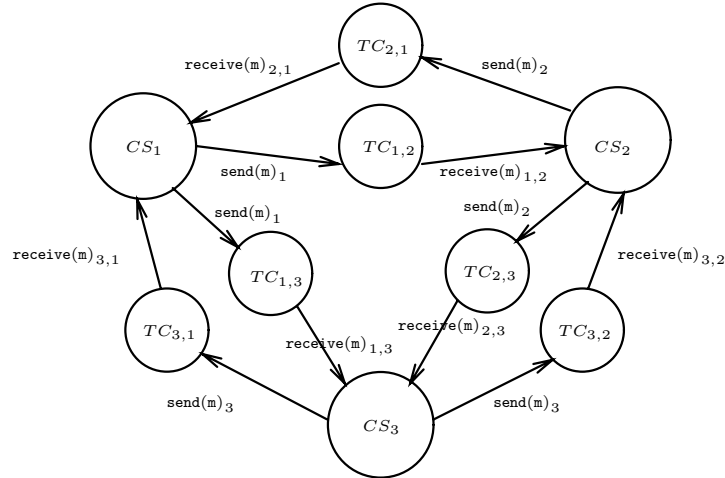
The following invariant states that a timeout performed by $\mathcal{C}$ can be used to conclude that the sender process has failed.

Invariant 1 : Let $\mathcal{C} = \text{PeriodicSend2}(u_1, M) \parallel \text{TimedChannel}(b, M) \parallel \text{Timeout}(u_2, M)$ and assume that $b + u_1 < u_2$. In any reachable state $\mathbf{x}$ of $\mathcal{C}$, if $\mathbf{x}(\text{Timeout.suspected}) = \text{true}$ then $\mathbf{x}(\text{PeriodicSend2.failed}) = \text{true}$.

The automaton $\mathcal{C}$ is guaranteed to perform a timeout to signal the failure of a process, within a specified amount of time after the occurrence of a fail event. The following is a formal statement of this property.

Let α be an execution of $\mathcal{C}$ and let t be the point in time at which a fail event occurs in α . Then α includes a timeout event that occurs in the interval $(t + b, t + b + u_2]$. ■

Example 5.7 (Clock synchronization) In this example we consider the composition of three clock synchronization automata with six time-bounded channel automata. A graphical representation of the composite automaton is given below. The abbreviation CS_i represents the automaton $\text{ClockSync}(u, r, i)$. The abbreviation $TC_{i,j}$ represents the timed channel with maximum delay b that communicates real-valued messages from $\text{ClockSync}(u, r, i)$ to $\text{ClockSync}(u, r, j)$. We assume that the time-bounded channel automata used in this composition are defined as in Example 4.1 where receive and send actions in each instance are renamed such that they can be shared with clock synchronization automata. Let $\mathcal{C} = CS_1 \parallel CS_2 \parallel CS_3 \parallel TC_{1,2} \parallel TC_{2,1} \parallel TC_{1,3} \parallel TC_{3,1} \parallel TC_{2,3} \parallel TC_{3,2}$.



A physical clock diverges from real time at the largest rate when it evolves with rate $(1 + r)$ or $(1 - r)$. For example, if a physical clock evolves with rate $1 + r$, then at time t , its value is $t \times (1 + r)$. Hence, the largest possible difference between a physical clock and the real time is $(t \times r)$. This property is stated by the invariant below.

Invariant 1 : In any reachable state x of $\mathcal{C}$, at any time $t \in T$, for any $i \in \{1, 2, 3\}$, $|x(\text{ClockSync}(u, r, i).\text{physclock}) - t| \leq (t \times r)$.

Two physical clocks in $\mathcal{C}$ diverge at the largest rate when one evolves with rate $(1 + r)$ and the other with $(1 - r)$. It follows from Invariant 1 that, at any time t the largest possible difference between the physical clock values for two processes is $(2 \times t \times r)$. This property is formalized by the following invariant.

Invariant 2 : In any reachable state x of $\mathcal{C}$, at any time $st \in T$, for any $i, j \in \{1, 2, 3\}$, $|x(\text{ClockSync}(u, r, i).\text{physclock}) - x(\text{ClockSync}(u, r, j).\text{physclock})| \leq (2 \times t \times r)$ where $i, j \in \{1, 2, 3\}$.

The following invariant states that in any reachable state there exists a process j such that the logical clock of each other process in the system is smaller than or equal to the physical clock of j . This follows from the definition of a logical clock and the fact that physical clocks always increase.

Invariant 3 : In any reachable state x of $\mathcal{C}$, there exists $j \in \{1, 2, 3\}$ such that for all $i \in \{1, 2, 3\}$, $x(\text{ClockSync}(u, r, i).\text{logclock}) \leq x(\text{ClockSync}(u, r, j).\text{physclock})$.

The following invariant states that in any reachable state there exists a process j such that the logical clock of each other process in the system is larger than or equal to the physical clock of j . This follows from the definition of a logical clock.

Invariant 4 : In any reachable state x of $\mathcal{C}$, there exists $j \in \{1, 2, 3\}$ such that for all $i \in \{1, 2, 3\}$, $x(\text{ClockSync}(u, r, i).\text{logclock}) \geq x(\text{ClockSync}(u, r, j).\text{physclock})$.

Invariants 3 and 4 together are called validity properties. They express the condition that all the logical clocks remain in an envelope bounded by the maximum and minimum physical clock values in the system.

The following invariant formalizes the property that all the logical clocks at a given time lie within the envelope formed by the largest and the smallest physical clock values in the system. It follows from Invariants 1, 3 and 4 that any point in this envelope can diverge from real time t by at most $(t \times r)$ time units.

Invariant 5 : In any reachable state x of $\mathcal{C}$, at any time $t \in T$, for any $i \in \{1, 2, 3\}$ $|x(\text{ClockSync}(u, r, i).\text{logclock}) - t| \leq (t \times r)$.

Finally, we state a property about the agreement of logical clocks in $\mathcal{C}$.

Invariant 6 : In any reachable state $\mathbf{x}$ of $\mathcal{C}$, for $i, j \in \{1, 2, 3\}$,
 $|\mathbf{x}(\text{ClockSync}(u, r, i).\text{logclock}) - \mathbf{x}(\text{ClockSync}(u, r, j).\text{logclock})| \leq u + (b \times (1 + r)).$

To see why Invariant 6 holds, fix j to be a process with the largest physical clock in $\mathbf{x}$, and fix i to be any other process. Let v_j, v_i be the logical clock values of j and i respectively in state $\mathbf{x}$. Note that v_j is also the physical clock value of j in $\mathbf{x}$. By Invariant 3, we know that $v_i \leq v_j$. To show Invariant 6, it suffices to show that $v_j - v_i \leq u + (b \text{ times } (1 + r))$.

Let α be a finite execution that leads to state $\mathbf{x}$. There are two cases to consider.

1. Some message sent by j arrives at i in α .

Consider the last such message and let v_1 be the value that it contains. Let v_2 be the newly adjusted logical clock value of i immediately after the message arrives. We know that $v_i \geq v_2 \geq v_1$.

If j sends a later message to i in α , then it sends the next later message when its physical clock has value $v_1 + u$. By assumption, this message does not arrive at i . Therefore, the real time that elapses after sending it must be at most b . It follows that the physical clock increase of j since sending this message is at most $b \times (1 + r)$ and so $v_j \leq v_1 + u + b \times (1 + r)$. On the other hand, if j does not send a later message to i in α , then $v_j \leq v_1 + u$. In either case, we have $v_j \leq v_1 + u + b \times (1 + r)$. Since $v_i \geq v_1$, we have $v_j - v_i \leq u + b \times (1 + r)$, as needed for Invariant 6.

2. No message sent by j arrives at i in α .

Since the first send occurs at time 0 and sb is the largest possible communication delay, the fact that i has not received the first message sent by j at time 0 implies that $t \leq b$. Since both clocks start at 0, we have $v_j \leq b \times (1 + r)$ and $v_i \geq 0$. Therefore, $v_j - v_i \leq u + b \times (1 + r)$, which suffices for Invariant 6.

■

5.1.2 Substitutivity Results

Theorem 5.4, which relates the set of traces of a composed automaton to the set of traces of component automata, is fundamental for compositional reasoning. We now introduce another important class of results, substitutivity results, that are useful for decomposing verification of composite automata. These results are best understood by viewing one of the components of a composition as the system and the other as the environment with which the system interacts.

The following result states that if a TA $\mathcal{A}_1$ can be shown to implement another one $\mathcal{A}_2$, with no assumptions about their environments, then $\mathcal{A}_1$ can be shown to implement $\mathcal{A}_2$ in a given environment $\mathcal{B}$.

Theorem 5.8 *Suppose $\mathcal{A}_1$, $\mathcal{A}_2$ and $\mathcal{B}$ are TAs, $\mathcal{A}_1$ and $\mathcal{A}_2$ have the same external actions, and each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with $\mathcal{B}$. If $\mathcal{A}_1 \leq \mathcal{A}_2$ then $\mathcal{A}_1 \parallel \mathcal{B} \leq \mathcal{A}_2 \parallel \mathcal{B}$.*

Corollary 5.9 *Suppose $\mathcal{A}_1$, $\mathcal{A}_2$, $\mathcal{B}_1$, and $\mathcal{B}_2$ are TAs, $\mathcal{A}_1$ and $\mathcal{A}_2$ have the same external actions, $\mathcal{B}_1$ and $\mathcal{B}_2$ have the same external actions, and each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with each of $\mathcal{B}_1$ and $\mathcal{B}_2$. If $\mathcal{A}_1 \leq \mathcal{A}_2$ and $\mathcal{B}_1 \leq \mathcal{B}_2$ then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.*

We can strengthen Corollary 5.9 slightly by the following corollary: if $\mathcal{A}_1$ implements $\mathcal{A}_2$ in an environment $\mathcal{B}_2$, then $\mathcal{A}_1$ composed with an environment that is more restrictive than $\mathcal{B}_2$ (whose set of external behaviors is smaller than that of $\mathcal{B}_2$), implements $\mathcal{A}_2$ composed with $\mathcal{B}_2$.

Corollary 5.10 *Suppose $\mathcal{A}_1$, $\mathcal{A}_2$, $\mathcal{B}_1$, and $\mathcal{B}_2$ are TAs, $\mathcal{A}_1$ and $\mathcal{A}_2$ have the same external actions, $\mathcal{B}_1$ and $\mathcal{B}_2$ have the same external actions, and each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with each of $\mathcal{B}_1$ and $\mathcal{B}_2$. If $\mathcal{A}_1 \parallel \mathcal{B}_2 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ and $\mathcal{B}_1 \leq \mathcal{B}_2$ then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.*

Proof: Let $\beta \in \text{traces}_{\mathcal{A}_1 \parallel \mathcal{B}_1}$. By Theorem 5.4, $\beta \upharpoonright (E_{\mathcal{A}_1}, \emptyset) \in \text{traces}_{\mathcal{A}_1}$ and $\beta \upharpoonright (E_{\mathcal{B}_1}, \emptyset) \in \text{traces}_{\mathcal{B}_1}$. Since $\mathcal{B}_1 \leq \mathcal{B}_2$, $\beta \upharpoonright (E_{\mathcal{B}_1}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. Since $\mathcal{B}_1$ and $\mathcal{B}_2$ have the same external actions, it follows that $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. We have $\beta \upharpoonright (E_{\mathcal{A}_1}, \emptyset) \in \text{traces}_{\mathcal{A}_1}$ and $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. By Theorem 5.4, $\beta \in \text{traces}_{\mathcal{A}_1 \parallel \mathcal{B}_2}$. Since $\mathcal{A}_1 \parallel \mathcal{B}_2 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ by assumption, $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$, as needed. ■

For other preorders, we also get substitutivity results, for example:

Theorem 5.11 *Suppose $\mathcal{A}_1$, $\mathcal{A}_2$ and $\mathcal{B}$ are TAs, $\mathcal{A}_1$ and $\mathcal{A}_2$ have the same external actions, and each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with $\mathcal{B}$.*

1. *If every closed trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$ then every closed trace of $\mathcal{A}_1 \parallel \mathcal{B}$ is a trace of $\mathcal{A}_2 \parallel \mathcal{B}$.*
2. *If every admissible trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$ then every admissible trace of $\mathcal{A}_1 \parallel \mathcal{B}$ is a trace of $\mathcal{A}_2 \parallel \mathcal{B}$.*
3. *If every non-Zeno trace of $\mathcal{A}_1$ is a trace of $\mathcal{A}_2$ then every non-Zeno trace of $\mathcal{A}_1 \parallel \mathcal{B}$ is a trace of $\mathcal{A}_2 \parallel \mathcal{B}$.*

Example 5.12 (A counterexample for a desirable substitutivity theorem)

Suppose $\mathcal{A}_1$ and $\mathcal{A}_2$ have the same external actions, $\mathcal{B}_1$ and $\mathcal{B}_2$ have the same external actions, and that each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with each of $\mathcal{B}_1$ and $\mathcal{B}_2$. If we view

```

automaton CatchUpA
  signature
    external a, b
  states
    counta: Nat := 0, countb: Nat := 0,
    now: Real := 0, next: Real := 0
  transitions
    external a
      pre
        (counta ≤ countb)
        ∧ (now = next)
      eff
        counta := counta + 1;
        next := now + 1
    external b
      eff
        countb := countb + 1;
        next := now + 1
  trajectories
    stop when
      now = next
    evolve
      d(now) = 1

```

```

automaton CatchUpB
  signature
    external a, b
  states
    counta: Nat := 0, countb: Nat := 0,
    now: Real := 0, next: Real := 0
  transitions
    external a
      eff
        counta := counta + 1
    external b
      pre
        (countb + 1) ≤ counta
        ∧ now = next
      eff
        countb := countb + 1;
        next := now + 1
  trajectories
    stop when
      now = next
    evolve
      d(now) = 1

```

Figure 9: CatchUpA and CatchUpB

```

automaton BoundedAlternateA
  signature
    external a, b
  states
    myturn: Bool := true,
    maxout: Nat
  transitions
    external a
      pre
        myturn  $\wedge$  (maxout > 0)
      eff
        myturn := false;
        maxout := maxout - 1

    external b
      eff
        myturn := true

```

```

automaton BoundedAlternateB
  signature
    external a, b
  states
    myturn: Bool := false,
    maxout: Nat
  transitions
    external a
      eff
        myturn := true

    external b
      pre
        myturn  $\wedge$  (maxout > 0)
      eff
        myturn := false;
        maxout := maxout - 1

```

Figure 10: BoundedAlternateA and BoundedAlternateB

$\mathcal{A}_2$ and $\mathcal{B}_2$ as specifications and want to prove that $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$, it would be useful to have a theorem that says if $\mathcal{A}_1 \parallel \mathcal{B}_2 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ and $\mathcal{A}_2 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$. That is, if $\mathcal{A}_1$ implements $\mathcal{A}_2$ in the context of $\mathcal{B}_2$ and $\mathcal{B}_1$ implements $\mathcal{B}_2$ in the context of $\mathcal{A}_2$, we would like to conclude that $\mathcal{A}_1 \parallel \mathcal{B}_1$ implements $\mathcal{A}_2 \parallel \mathcal{B}_2$. We show by means of a counterexample that it is impossible to prove such a theorem. The problem arises with the infinite behaviors of $\mathcal{A}_1 \parallel \mathcal{B}_2$.

As examples for $\mathcal{A}_1, \mathcal{B}_1, \mathcal{A}_2$, and $\mathcal{B}_2$, consider, respectively, the automata **CatchUpA**, **CatchUpB**, **BoundedAlternateA**, **BoundedAlternateB** in Figures 9 and 10. All automata have the same set of actions, consisting of the external actions **a** and **b**. **CatchUpA** can perform an arbitrary number of **b** actions, and can perform an **a** provided that $\text{counta} \leq \text{countb}$ and one time unit has elapsed since the occurrence of the last action. **CatchUpA** allows

`counta` to increase to one more than `countb`. `CatchUpB` can perform an arbitrary number of `a` actions, and can perform a `b` provided that `counta` is at least one more than `countb`. `CatchUpB` allows `countb` to reach `counta`.

`BoundedAlternateA` has an infinite number of start states, each giving a different finite bound on the number of `a` actions it can perform. Similarly, `BoundedAlternateB` has an infinite number of start states, each giving a different finite bound on the number of `b` actions it can perform. Note that the absence of trajectory definitions in the specifications of these automata imply that they are timing-independent. That is, there is no constraint on the timing of actions.

The automata `CatchUpA` and `CatchUpB` strictly alternate their inputs and outputs until a maximum count is reached for outputs, when put in the context of, respectively, `BoundedAlternateA` and `BoundedAlternateB`. Hence, $(\text{CatchUpA} \parallel \text{BoundedAlternateB}) \leq (\text{BoundedAlternateA} \parallel \text{BoundedAlternateB})$, and $(\text{BoundedAlternateA} \parallel \text{CatchUpB}) \leq (\text{BoundedAlternateA} \parallel \text{BoundedAlternateB})$. On the other hand, $(\text{CatchUpA} \parallel \text{CatchUpB})$ can perform an infinite sequence of alternating `a` and `b` actions, which is not allowed by $(\text{BoundedAlternateA} \parallel \text{BoundedAlternateB})$. Hence, $(\text{CatchUpA} \parallel \text{CatchUpB})$ does not implement $(\text{BoundedAlternateA} \parallel \text{BoundedAlternateB})$. ■

In Section 7, we revisit the substitutivity issue and prove Theorem 7.8, a variant of the desirable theorem considered in the above example, by assuming certain conditions on the environments $\mathcal{A}_2$ and $\mathcal{B}_2$.

5.2 Hiding

We define one hiding operation for timed automata, which hides external actions: if $E \subseteq E_{\mathcal{A}}$, then $\text{ActHide}(E, \mathcal{A})$ is the TA $\mathcal{B}$ that is equal to $\mathcal{A}$ except that $E_{\mathcal{B}} = E_{\mathcal{A}} - E$ and $H_{\mathcal{B}} = H_{\mathcal{A}} \cup E$.

Lemma 5.13 *If $E \subseteq E_{\mathcal{A}}$ then $\text{ActHide}(E, \mathcal{A})$ is a TA.*

Lemma 5.14 *If $\mathcal{A}$ is a TA and $E \subseteq E_{\mathcal{A}}$ then $\text{traces}_{\text{ActHide}(E, \mathcal{A})} = \{\beta \upharpoonright (E_{\mathcal{A}} - E, \emptyset) \mid \beta \in \text{traces}_{\mathcal{A}}\}$.*

The following theorem states that the hiding operation respects the implementation relation.

Theorem 5.15 *Suppose $\mathcal{A}$ and $\mathcal{B}$ are TAs with $\mathcal{A} \leq \mathcal{B}$, and suppose $E \subseteq E_{\mathcal{A}}$. Then $\text{ActHide}(E, \mathcal{A}) \leq \text{ActHide}(E, \mathcal{B})$.*

5.3 Extending Timed Automata with Bounds

In this section, we define a new class of automata, “TA with bounds” where the basic definition of a timed automaton is extended with the notion of a task and a pair of bounds (a lower and an upper bound) for each task. We then define an operation that transforms a given TA with bounds to another TA. This operation supports specifying a system by thinking in terms of tasks and bounds as in the timed automata of Merritt, Modugno, and Tuttle [32] and the phase transition systems of Maler, Manna and Pnueli [31].

In defining the operation for extending timed automata with bounds, we restrict attention to a class of automata where the enabling and disabling of actions during trajectories follow certain rules. Specifically, our operation is defined on automata in which each action is enabled or disabled throughout an entire trajectory, or becomes enabled once during a trajectory and remains so until the end of that trajectory. The given restrictions ensure that the result of applying the operation to a TA is another TA and that the resulting TA satisfies the restrictions.

Let $\mathcal{A}$ be a TA, C a set of actions of $\mathcal{A}$, and $\mathcal{T}$ the set of trajectories of $\mathcal{A}$. We say that $\mathcal{T}$ is *well-formed* with respect to C if each $\tau \in \mathcal{T}$ satisfies one of the following conditions:

1. For all $t \in \text{dom}(\tau)$, C is enabled in $\tau(t)$.
2. For all $t \in \text{dom}(\tau)$, C is disabled in $\tau(t)$.
3. There exists $t \in \text{dom}(\tau)$ such that for all $t' \in [0, t)$, C is disabled in $\tau(t')$ and for all $t' \in \text{dom}(\tau) - [0, t)$, C is enabled in $\tau(t')$.

A TA with bounds, $\mathcal{A} = (\mathcal{B}, C, l, u)$ consists of:

- A timed automaton $\mathcal{B} = (X, Q, \Theta, E, H, \mathcal{D}, \mathcal{T})$.
- A set $C \subseteq E \cup H$ of actions called a *task*; we assume that $\mathcal{T}$ is well-formed with respect to C .
- A lower time bound l and an upper time bound u for C . We require that the following axioms are satisfied for l and u :

B1 $l \in \mathbb{R}^{\geq 0}$ and $u \in \mathbb{R}^{\geq 0} \cup \{\infty\}$.

B2 $l \leq u$.

Lower and upper bounds are used to specify how much time is allowed to pass between the enabling and the performance of an action. If l is the lower bound for a task C , then an action in C must remain enabled at least for l time units before being performed. If u is the upper bound for a task C , then an action in C can remain enabled at most u time

units without being performed: it must either be performed or become disabled within u time units.

We now define an operation **Extend**, which transforms a TA $\mathcal{A}$ with bounds to another TA $\mathcal{A}'$ that incorporates the new bounds, in addition to the timing constraints already present in $\mathcal{A}$. Let $\mathcal{A} = (\mathcal{B}, C, l, u)$ be a TA with bounds where $\mathcal{B} = (X, Q, \Theta, E, H, \mathcal{D}, \mathcal{T})$. Then **Extend**($\mathcal{A}$) is the TA $\mathcal{A}' = (X', Q', \Theta', E', H', \mathcal{D}', \mathcal{T}')$ such that the components of $\mathcal{A}'$ consist of:

- $X' = X \cup \{now, first, last\}$ where:
 1. $now, first$, and $last$ are new variables that do not appear in X .
 2. now is an analog variable such that $type(now) = R$.
 3. $first$ and $last$ are discrete variables where $type(first) = R$ and $type(last) = R \cup \{\infty\}$.
- $Q' = \{\mathbf{x} \in val(X') \mid \mathbf{x} \restriction X \in Q\}$.
- Θ' consists of all the states $\mathbf{x} \in Q'$ that satisfy the following conditions:
 1. $\mathbf{x} \restriction X \in \Theta$.
 2. $\mathbf{x}(now) = 0$.
 3. $\mathbf{x}(first) = \begin{cases} l & \text{if } C \text{ is enabled in } \mathbf{x} \restriction X, \\ 0 & \text{otherwise.} \end{cases}$
 $\mathbf{x}(last) = \begin{cases} u & \text{if } C \text{ is enabled in } \mathbf{x} \restriction X, \\ \infty & \text{otherwise.} \end{cases}$
- $E' = E$ and $H' = H$. We write $A' \triangleq E' \cup H'$.
- If $a \in (E \cup H)$ then $(\mathbf{x}, a, \mathbf{x}') \in \mathcal{D}'$ exactly if all of the following conditions hold:
 1. $(\mathbf{x} \restriction X) \xrightarrow{a}_{\mathcal{A}} (\mathbf{x}' \restriction X)$.
 2. $\mathbf{x}'(now) = \mathbf{x}(now)$.
 3. (a) If $a \in C$, then $\mathbf{x}(first) \leq \mathbf{x}(now)$.
 (b) If C is enabled both in $\mathbf{x} \restriction X$ and $\mathbf{x}' \restriction X$ and $a \notin C$, then $\mathbf{x}(first) = \mathbf{x}'(first)$ and $\mathbf{x}(last) = \mathbf{x}'(last)$.
 (c) If C is enabled in $\mathbf{x}' \restriction X$ and either C is not enabled in $\mathbf{x} \restriction X$ or $a \in C$, then $\mathbf{x}'(first) = \mathbf{x}(now) + l$ and $\mathbf{x}'(last) = \mathbf{x}(now) + u$.
 (d) If C is not enabled in $\mathbf{x}' \restriction X$, then $\mathbf{x}'(first) = 0$ and $\mathbf{x}'(last) = \infty$.
- $\mathcal{T}'$ is a set that consists of all $\tau \in trajs(X')$ that satisfy the following conditions:
 1. $(\tau \downarrow X) \in \mathcal{T}$.

2. $d(now) = 1$.
3. (a) If for all $t \in dom(\tau)$, C is enabled in $\tau \downarrow X(t)$ then *first* and *last* are constant throughout τ .
- (b) If for all $t \in dom(\tau)$, C is disabled in $\tau \downarrow X(t)$ then *first* and *last* are constant throughout τ .
- (c) If for all $t' \in [0, t)$, C is disabled in $\tau(t')$ and for all $t' \in dom(\tau) - [0, t)$, C is enabled in $\tau(t')$ then
 - i. *first* and *last* are constant in $[0, t)$.
 - ii. $\tau(t)(first) = \tau(t)(now) + l$ and $\tau(t)(last) = \tau(t)(now) + u$.
 - iii. *first* and *last* are constant in $dom(\tau) - [0, t)$.
- (d) $now \leq last$.

The transformation is based on the idea of augmenting the state of the original automaton with a variable to represent current time (*now*) and the earliest time (*first*) and the latest time (*last*) a task can be performed. All these variables represent time in absolute terms. Item 3(a) in the definition of $\mathcal{D}'$ expresses the new lower bound constraint and Item 3(d) in the definition of $\mathcal{T}'$ the new upper bound constraint.

Let $\mathcal{A}$ be a TA with bounds $(\mathcal{B}, C, l, u)$. In a start state $\mathbf{x}$ of $\text{Extend}(\mathcal{A})$, the variables *first* and *last* are initialized to l and u respectively, if C is enabled in $\mathbf{x}$. If C is not enabled in $\mathbf{x}$, then *first* is set to 0 and *last* is set to ∞ . Items 3(a) in the definition of $\mathcal{D}'$ and 3(c) in the definition of $\mathcal{T}'$ show how the variables *first* and *last* are updated. When C becomes newly enabled by a discrete transition or when a C action leads to a state in which C is enabled, *first* is set to $now + l$ and *last* is set to $now + u$. The variables *first* and *last* are updated similarly when C becomes newly enabled in the course of a trajectory.

Theorem 5.16 *Suppose that $\mathcal{A} = (\mathcal{B}, C, l, u)$ is a TA with bounds. Then $\text{Extend}(\mathcal{A})$ is a TA with a set of trajectories that is well-formed with respect to C .*

Proof: The proof follows from the definitions of TA and the operation Extend . Step 3(a) in the definition of $\mathcal{D}'$ adds a new lower bound constraint, which makes enabling start at some particular time. Step 3(b) in the definition of $\mathcal{T}'$, adds a new upper bound constraint, which stops trajectories at a particular time and which does not add any enabling or disabling to trajectories. ■

In the rest of this section, we sometimes speak of variables, states and traces of a TA with bounds. If $\mathcal{A} = (\mathcal{B}, C, l, u)$ is a TA with bounds, variables, states and traces of $\mathcal{A}$ refer to, respectively, the states and the traces of the underlying automaton $\mathcal{B}$.

Theorem 5.17 *Suppose $\mathcal{A} = (\mathcal{B}, C, l, u)$ is a TA with bounds. Then $\text{traces}_{\text{Extend}(\mathcal{A})} \subseteq \text{traces}_{\mathcal{A}}$.*

Proof: Let $F : Q' \rightarrow Q$ be defined as follows: $F(\mathbf{x}) = \mathbf{x} \upharpoonright X$ where X is the set of internal variables of $\mathcal{A}$. It is easy to check that F is a refinement from $\text{Extend}(\mathcal{A})$ to $\mathcal{A}$. By Theorem 4.27 and Corollary 4.23, we conclude that $\text{traces}_{\text{Extend}(\mathcal{A})} \subseteq \text{traces}_{\mathcal{A}}$. ■

Lemma 5.18 *Suppose that $\mathcal{A}$ is a TA with bounds. For any reachable state $\mathbf{x}$ of $\text{Extend}(\mathcal{A})$, if C is enabled in $\mathbf{x} \upharpoonright X$ in $\mathcal{A}$, then $\mathbf{x}(\text{last}) \leq \mathbf{x}(\text{now}) + u$.*

Proof: Consider a closed execution α of $\text{Extend}(\mathcal{A})$. Using the axioms **T1** and **T2** for trajectories, we can write α as a concatenation of closed execution fragments $\alpha_0 \frown \alpha_1 \frown \dots \frown \alpha_k$ where α_0 is a point trajectory, and each α_i for $i \geq 1$ is either a trajectory or a discrete action surrounded by two point trajectories such that for all $0 \leq i \leq k-1$, $\alpha_i.\text{lstate} = \alpha_{i+1}.\text{fstate}$. We prove the invariant by induction on the length k of the sequence of execution fragments.

For the base case, suppose that C is enabled in $\alpha_0.\text{fstate} \upharpoonright X$. Since α is an execution, we know that $\alpha_0.\text{fstate}$ is a start state of $\text{Extend}(\mathcal{A})$. By definition of $\text{Extend}(\mathcal{A})$, $\alpha_0.\text{fstate}(\text{last}) = u$. Since $\alpha_0.\text{fstate}(\text{now}) = 0$, $\alpha_0.\text{fstate}(\text{last}) \leq \alpha_0.\text{fstate}(\text{now}) + u$, as required.

For the inductive step, we assume that the property is true for the sequence $\alpha_0 \frown \alpha_1 \frown \dots \frown \alpha_k$ and show that it is true in the sequence α_{k+1} in $\alpha_0 \frown \alpha_1 \frown \dots \frown \alpha_k \frown \alpha_{k+1}$. There are two cases to consider depending on whether α_{k+1} is a discrete action surrounded by two point trajectories or a trajectory.

1. α_{k+1} is an action a surrounded by two point trajectories. Suppose that C is enabled in $\alpha_{k+1}.\text{lstate}$. There are two subcases to consider:
 - (a) C is enabled in $\alpha_k.\text{lstate} \upharpoonright X$ and $a \notin C$.
Then, $\alpha_{k+1}.\text{fstate}(\text{last}) = \alpha_k.\text{fstate}(\text{last})$ and $\alpha_{k+1}.\text{fstate}(\text{now}) = \alpha_k.\text{fstate}(\text{now})$. By inductive hypothesis, $\alpha_k.\text{lstate}(\text{last}) \leq \alpha_k.\text{lstate}(\text{now}) + u$. Therefore, $\alpha_{k+1}.\text{lstate}(\text{last}) \leq \alpha_{k+1}.\text{lstate}(\text{now}) + u$, as needed.
 - (b) C is disabled in $\alpha_k.\text{lstate} \upharpoonright X$ or $a \in C$.
Then, by definition of $\text{Extend}(\mathcal{A})$, $\alpha_{k+1}.\text{lstate}(\text{last}) = \alpha_{k+1}.\text{lstate}(\text{now}) + u$, which suffices.
2. α_{k+1} is a trajectory.
Suppose that C is enabled in $\alpha_{k+1}.\text{lstate} \upharpoonright X$ in $\mathcal{A}$. There are two subcases to consider:
 - (a) C is enabled in $\alpha_{k+1}.\text{fstate} \upharpoonright X$ in $\mathcal{A}$.
By inductive hypothesis $\alpha_{k+1}.\text{fstate}(\text{last}) \leq \alpha_{k+1}.\text{fstate}(\text{now}) + u$. By the well-formedness assumption, we know that C must be enabled throughout α_{k+1} and by definition of $\text{Extend}(\mathcal{A})$ last is constant throughout α_{k+1} . Since the value of now increases, it is easy to see that $\alpha_{k+1}.\text{lstate}(\text{last}) \leq \alpha_{k+1}.\text{lstate}(\text{now}) + u$.

- (b) C is disabled in $\alpha_{k+1}.fstate \upharpoonright X$ in $\mathcal{A}$.

Then, since it is enabled in $\alpha_{k+1}.lstate \upharpoonright X$ by the well-formedness assumption, it becomes enabled at some point t in the domain of α_{k+1} and remains enabled thereafter. Therefore, $\alpha_{k+1}(t)(last) = \alpha_{k+1}(t)(now) + u$, by definition of $\text{Extend}(\mathcal{A})$. Since $last$ remains constant after it is set and the value of now increases, $\alpha_{k+1}.lstate(last) \leq \alpha_{k+1}.lstate(now) + u$ holds. ■

The theorem below shows that the executions of an automaton obtained by applying the transformation Extend to a TA with bounds respect the time bounds specified by the lower bound l and the upper bound u .

Theorem 5.19 *Let $\mathcal{A} = (\mathcal{B}, C, l, u)$ be a TA with bounds. Then,*

1. *There does not exist a closed execution fragment α of $\text{Extend}(\mathcal{A})$ from a reachable state, where $\alpha.ltime > u$, C is enabled in $\mathcal{A}$ in all the states of $\alpha \upharpoonright (A, X)$ and no action in C occurs in α .*
2. *There does not exist a closed execution fragment α of $\text{Extend}(\mathcal{A})$ from a reachable state, where $\alpha.ltime < l$, such that C is not enabled in $\mathcal{A}$ in the first state of $\alpha \upharpoonright (A, X)$ and an action in C occurs in α .*

Proof:

1. Suppose, for the sake of contradiction, that there exists a closed execution fragment $\alpha = \tau_0 a_1 \tau_1 a_2 \dots \tau_n$ of $\text{Extend}(\mathcal{A})$ from a reachable state, where $\alpha.ltime > u$, C is enabled in $\mathcal{A}$ in all the states of $\alpha \upharpoonright (A, X)$ and none of the a_i in α is in C . By definition of trajectories for $\text{Extend}(\mathcal{A})$ it must be the case that $\alpha.lstate(now) \leq \alpha.lstate(last)$. Since C is enabled in $\mathcal{A}$ in all states in α , by Lemma 5.18 we have $\alpha.fstate(last) \leq \alpha.fstate(now) + u$. By definition of $\text{Extend}(\mathcal{A})$, $last$ remains constant throughout α ; therefore, $\alpha.lstate(last) = \alpha.fstate(last)$. Since $\alpha.fstate(last) \leq \alpha.fstate(now) + u$, it follows that $\alpha.lstate(last) \leq \alpha.fstate(now) + u$. By definition of α , we have $\alpha.lstate(now) = \alpha.fstate(now) + \alpha.ltime$. It follows that $\alpha.fstate(now) + \alpha.ltime \leq \alpha.fstate(now) + u$. This implies $\alpha.ltime \leq u$. But this gives us the needed contradiction since $\alpha.ltime > u$.
2. We assume that α is a closed execution fragment of $\text{Extend}(\mathcal{A})$ from a reachable state where $\alpha.ltime < l$, such that C is not enabled in $\mathcal{A}$ in the first state of α and an action in C occurs in α . Let $(\mathbf{x}, a, \mathbf{x}')$ be the first discrete transition of $\text{Extend}(\mathcal{A})$ in α such that $a \in C$. We show that the condition $\mathbf{x}(first) \leq \mathbf{x}(now)$, which has to hold for the discrete transition to occur, cannot be true, hence arrive at a contradiction.

By Theorem 5.16, the set of trajectories of $\text{Extend}(\mathcal{A})$ is well-formed with respect to C . Therefore, C can become enabled by either a discrete transition or during a trajectory, and remains enabled until the occurrence of $(\mathbf{x}, a, \mathbf{x}')$.

- (a) C becomes enabled by a discrete transition and remains enabled in $\mathcal{A}$ until the occurrence of $(\mathbf{x}, a, \mathbf{x}')$.

Let $(\mathbf{y}, b, \mathbf{y}')$ be the discrete transition of $\mathcal{A}$ that enables C . By item 3(c) in the definition of $\mathcal{D}'$ we know that first is set to $\mathbf{y}(\text{now}) + l$ when C becomes enabled. By item 3(b) in the definition of $\mathcal{D}'$ and 3(a) in the definition of $\mathcal{T}'$, we know that it remains constant so that $\mathbf{x}(\text{first}) = \mathbf{y}(\text{now}) + l$. Since $(\mathbf{x}, a, \mathbf{x}')$ is a discrete transition of $\text{Extend}(\mathcal{A})$, it must be the case that $\mathbf{x}(\text{first}) \leq \mathbf{x}(\text{now})$. Since $\mathbf{x}(\text{now}) \leq \mathbf{y}(\text{now}) + \alpha.\text{itime}$ and $\mathbf{x}(\text{first}) = \mathbf{y}(\text{now}) + l$ it follows that $\mathbf{y}(\text{now}) + l \leq \mathbf{y}(\text{now}) + \alpha.\text{itime}$. But we know by assumption that $\alpha.\text{itime} < l$ which gives the needed contradiction.

- (b) C becomes enabled at some point in the course of a trajectory τ and remains enabled in $\mathcal{A}$ until the occurrence of $(\mathbf{x}, a, \mathbf{x}')$.

Let $\mathbf{y}$ be a state in the range of τ where C becomes enabled. By item 3(c) in the definition of $\mathcal{T}'$ we know that first is set to $\mathbf{y}(\text{now}) + l$ when C becomes enabled and it remains constant in τ so that $\mathbf{x}(\text{first}) = \mathbf{y}(\text{now}) + l$. By item 3(b) in the definition of $\mathcal{D}'$ and 3(a) in the definition of $\mathcal{T}'$, we know that first remains constant until the occurrence of $(\mathbf{x}, a, \mathbf{x}')$. Since $(\mathbf{x}, a, \mathbf{x}')$ is a discrete transition of $\text{Extend}(\mathcal{A})$, it must be the case that $\mathbf{x}(\text{first}) \leq \mathbf{x}(\text{now})$. Since $\mathbf{x}(\text{now}) \leq \mathbf{y}(\text{now}) + \alpha.\text{itime}$ and $\mathbf{x}(\text{first}) = \mathbf{y}(\text{now}) + l$ it follows that $\mathbf{y}(\text{now}) + l \leq \mathbf{y}(\text{now}) + \alpha.\text{itime}$. But we know by assumption that $\alpha.\text{itime} < l$ which gives the needed contradiction.

■

Example 5.20 (Fischer’s mutual exclusion algorithm specified using tasks and bounds) In Example 4.5 we presented the specification of Fischer’s mutual exclusion algorithm as a TA. This example illustrates an alternative way of specifying the same algorithm by using a TA with bounds.

Recall that, formally, we define a TA with bounds as a TA augmented with a single task along with lower and upper bounds for that task. The automaton in Figure 11 is, however, augmented with a set of tasks and bounds (we omit from the figure those transition definitions that are the same as in Example 4.5). This is for notational convenience and the automaton in Figure 11 should be viewed as the automaton representing the cumulative result of adding in successive steps two tasks for each index. We assume that Extend is applied once for each task. That is, we start with the timing-independent version of $\text{FischerME}(\mathbf{u_set}, \text{l_check:Real})$, apply Extend to the automaton augmented with the task $\{\text{set}(i)\}$ to add the lower bound 0 and the upper bound $\mathbf{u_set}$, then apply Extend

```

type PcValue = enumeration of rem, test, set, check,
                                leavetry, crit, leaveexit
type Index = enumeration of nil, p1, p2, p3, p4

automaton FischerME(u_set, l_check: Real)
  signature
    external try(i:Index), crit(i:Index), exit(i:Index), rem(i:Index)
    internal test(i:Index), set(i:Index), check(i:Index), reset(i:Index)
  states
    x: Index := nil,
    pc: Array[Index,PcValue] := constant(rem),
    initially u_set ≥ 0 ∧ l_check ≥ 0 ∧ u_set < l_check
  transitions
    internal test(i)
      pre
        pc[i] = test
      eff
        if x = p0 then
          pc[i] := set;
          lastset[i] := now + u_set
    internal set(i)
      pre
        pc[i] = set
      eff
        x := i;
        pc[i] := check;
        lastset[i] := infty;
        firstcheck[i] := now + l_check
    internal check(i)
      pre
        pc[i] = check ∧
          now ≥ firstcheck[i]
      eff
        if x = i then pc[i] := leavetry
        else pc[i] := test
  tasks
    set = {set(i)} for i: Index; check = {check(i)} for i: Index

  bounds
    set = [0, u_set]; check = [l_check, infty]

```

Figure 11: Fischer's mutual exclusion algorithm with bounds

to the resulting automaton augmented with $\{\text{check}(i)\}$ to add the lower bound l_check and the upper bound ∞ . Such two successive applications are allowed since the result of

the first application of `Extend` satisfies the the well-formedness conditions for the set of trajectories.

The result of these successive applications yields an automaton similar to the one in Example 4.5. The only difference is that the mechanical application of the transformation would reset the value of `firstcheck[i]` to 0 as an effect of `check(i)` while we do not reset `firstcheck[i]` explicitly in 4.5, when it becomes disabled. This is because we make use of the facts that the value of `firstcheck[i]` is used only in determining whether `check(i)` is enabled and that `check(i)` becomes enabled only in the poststate of `set(i)` which also sets the value of `firstcheck[i]`. Note that this discrepancy does not give rise to any difference in the behaviors of the two automata. ■

6 Timed I/O Automata

In this section we refine the timed automaton model of Section 4 by distinguishing between input and output actions. Typically, an interaction between a system and its environment is modeled by using output and input actions to represent, respectively, the external events under the control of the system and the environment. We extend the results on simulation relations and composition from Sections 4 and 5 to this new setting. We also introduce special kinds of timed I/O automata: I/O feasible, progressive, and receptive TIOAs.

6.1 Definition of Timed I/O Automata

A *timed I/O automaton (TIOA)* $\mathcal{A}$ is a tuple $(\mathcal{B}, I, O)$ where

- $\mathcal{B} = (X, Q, \Theta, E, H, \mathcal{D}, T)$ is a timed automaton.
- I and O partition E into *input* and *output actions*, respectively. Actions in $L \triangleq H \cup O$ are called *locally controlled*; as before we write $A \triangleq E \cup H$.
- The following additional axioms are satisfied:

E1 (Input action enabling)

For every $\mathbf{x} \in Q$ and every $a \in I$, there exists $\mathbf{x}' \in Q$ such that $\mathbf{x} \xrightarrow{a} \mathbf{x}'$.

E2 (Time-passage enabling)

For every $\mathbf{x} \in Q$, there exists $\tau \in \mathcal{T}$ such that $\tau.fstate = \mathbf{x}$ and either

1. $\tau.ltime = \infty$, or
2. τ is closed and some $l \in L$ is enabled in $\tau.lstate$.

Input action enabling is the input enabling condition of ordinary I/O automata; it says that a TIOA is able to perform an input action at any time. The time-passage enabling

condition says that a TIOA either allows time to advance forever, or it allows time to advance for a while, up to a point where it is prepared to react with some locally controlled action. Because TIOAs have no external variables, **E1** and **E2** are slightly simpler than the corresponding axioms for HIOAs.

Notation: As we did for TAs, we often denote the components of a TIOA $\mathcal{A}$ by $\mathcal{B}_{\mathcal{A}}, I_{\mathcal{A}}, O_{\mathcal{A}}, X_{\mathcal{A}}, Q_{\mathcal{A}}, \Theta_{\mathcal{A}}$, etc., and those of a TIOA $\mathcal{A}_i$ by $H_i, I_i, O_i, \dots, X_i, Q_i, \Theta_i$, etc. We sometimes omit these subscripts, where no confusion is likely. We abuse notation slightly by referring to a TIOA $\mathcal{A}$ as a TA when we intend to refer to $\mathcal{B}_{\mathcal{A}}$.

Example 6.1 (TAs viewed as TIOAs) The automaton `TimedChannel(b,M)` described in Example 4.1 can be turned into a TIOA by classifying the `send` actions as inputs, and the `receive` actions as outputs. Since there is no precondition for `send` actions, they are enabled in each state, so clearly the input enabling condition **E1** holds. It is also easy to see that axiom **E2** holds: in each state either `queue` is nonempty, in which case a `receive` output action is enabled after a point trajectory, or `queue` is empty, in which case time can advance forever.

The automaton `ClockSync(u,r,i)` of Example 4.6 can be turned into a TIOA by classifying the `send` actions as outputs, and the `receive` actions as inputs. Axiom **E1** then holds trivially. Axiom **E2** holds since from each state either time can advance forever, or we have an outgoing trajectory (possibly of length 0) to a state in which `physclock = nextsend`, and from there a `send` output action is enabled. ■

6.2 Executions and Traces

An *execution fragment*, *execution*, *trace fragment*, or *trace* of a TIOA $\mathcal{A}$ is defined to be an execution fragment, execution, trace fragment, or trace of the underlying TA $\mathcal{B}_{\mathcal{A}}$, respectively.

We say that an execution fragment of a TIOA is *locally-Zeno* if it is Zeno and contains infinitely many locally controlled actions, or equivalently, if it has finite limit time and contains infinitely many locally controlled actions.

6.3 Special Kinds of Timed I/O Automata

6.3.1 Feasible and I/O Feasible TIOAs

A TIOA $A = (\mathcal{B}, I, O)$ is defined to be feasible provided that its underlying TA $\mathcal{B}$ is feasible according to the definition given in Section 4.3. As noted in Section 4.3, feasibility is a basic requirement that any TA (or TIOA) should satisfy. I/O feasibility is a strengthened version of feasibility that take inputs into account. It says that the automaton is capable of

providing some response from any state, for any sequence of input actions and any amount of intervening time-passage. In particular, it should allow time to pass to infinity if the environment does not submit any input actions. Formally, we define a TIOA to be *I/O feasible* provided that, for each state $\mathbf{x}$ and each $(I, \emptyset)$ -sequence β , there is some execution fragment α from $\mathbf{x}$ such that $\alpha \upharpoonright (I, \emptyset) = \beta$. That is, an I/O feasible TIOA accommodates arbitrary input actions occurring at arbitrary times. The given $(I, \emptyset)$ -sequence β describes the inputs and the amounts of intervening times.

6.3.2 Progressive TIOAs

A progressive TIOA never generates infinitely many locally controlled actions in finite time. Formally, a TIOA $\mathcal{A}$ is *progressive* if it has no locally-Zeno execution fragments.

The following lemma says that any progressive TIOA is capable of advancing time forever.

Lemma 6.2 *Every progressive TIOA is feasible.*

Proof: Let $\mathcal{A}$ be a progressive TIOA and let $\mathbf{x}$ be a state of $\mathcal{A}$. Since $\mathcal{A}$ is a TIOA it satisfies axiom **E2**. We construct an admissible execution fragment $\alpha = \alpha_0 \frown \alpha_1 \frown \alpha_2 \cdots$ from $\mathbf{x}$ as follows.

1. $\alpha_0 = \wp(\mathbf{x})$.
2. For each $i > 0$,
 - (a) If there exists a trajectory τ from $\alpha_{i-1}.lstate$ such that $\tau.ltime = \infty$ then α_i is the final execution fragment in the sequence and $\alpha_i = \tau$.
 - (b) Otherwise, let τ_i be a closed execution fragment from $\alpha_{i-1}.lstate$ such that $l \in L$ is enabled in $\tau_i.lstate$. Define $\alpha_i = \tau_i \frown \tau_{i+1}$ where $\tau_{i+1} = \wp(\mathbf{y})$ and $\tau_i.lstate \xrightarrow{l} \mathbf{y}$.

The above construction either ends after finitely many stages such that the last trajectory of α is admissible, or goes through infinitely many stages such that α contains infinitely many local actions. In the former case, we know that α is admissible since it ends with an admissible trajectory. In the latter case, since $\mathcal{A}$ is progressive, the fact that α has infinitely many local actions implies that α is admissible, as needed. ■

The following lemma says that a progressive TIOA is capable of allowing any amount of time to pass from any state.

Lemma 6.3 *Let $\mathcal{A}$ be a progressive TIOA, let $\mathbf{x}$ be a state of $\mathcal{A}$, and let $\tau \in \text{trajs}(\emptyset)$. Then there exists an execution fragment α of $\mathcal{A}$ such that $\alpha.fstate = \mathbf{x}$ and $\alpha \upharpoonright (I, \emptyset) = \tau$.*

Proof: The result follows from the construction used in the proof of Lemma 6.2. Let α be an admissible execution fragment from $\mathbf{x}$ constructed as in the proof of Lemma 6.2. Let α' be a prefix of α such that $\alpha' \upharpoonright (\emptyset, \emptyset) = \tau$. Since our construction uses no actions from I , we have $\alpha' \upharpoonright (I, \emptyset) = \alpha' \upharpoonright (\emptyset, \emptyset) = \tau$, as needed. ■

The following theorem says that a progressive TIOA is capable not just of allowing arbitrary amounts of time to pass, but of allowing arbitrary input actions at arbitrary times.

Theorem 6.4 *Every progressive TIOA is I/O feasible.*

Proof: Let $\mathcal{A}$ be a progressive TIOA, let $\mathbf{x}$ be a state of $\mathcal{A}$, and let $\beta = \tau_0 a_1 \tau_1 a_2 \tau_2 \dots$ be an $(I, \emptyset)$ -sequence. We construct a finite or infinite sequence $\alpha_0 \alpha_1 \dots$ of execution fragments such that:

1. $\alpha_0.fstate = \mathbf{x}$.
2. For each nonfinal index i , $\alpha_i.lstate = \alpha_{i+1}.fstate$.
3. For each i , $(\alpha_0 \cap \alpha_1 \cap \dots \cap \alpha_i) \upharpoonright (I, \emptyset) = \tau_0 a_1 \tau_1 \dots \tau_i$.

The construction is carried out recursively. To define α_0 , we start with $\mathbf{x}$ and use Lemma 6.3 to span τ_0 . For $i > 0$, we define α_i by starting with $\alpha_{i-1}.lstate$, using axiom **E1** to perform the input action a_i and move to a new state and then using Lemma 6.3 to span τ_i .

Let $\alpha = \alpha_0 \cap \alpha_1 \cap \dots$. By Lemma 3.8, α is an execution fragment of $\mathcal{A}$ from $\mathbf{x}$ such that $\alpha \upharpoonright (I, \emptyset) = \beta$, as needed. ■

6.3.3 Receptive Timed I/O Automata

In this section, we define the notion of *receptiveness* for TIOAs. A TIOA will be defined to be receptive provided that it admits a *strategy* for resolving its nondeterministic choices that never generates infinitely many locally controlled actions in finite time. This notion has an important consequence: A receptive TIOA provides some response from any state, for any sequence of discrete input actions at any times. This implies that the automaton has a nontrivial set of execution fragments, in fact, it has execution fragments that accommodate any inputs from the environment. The automaton cannot simply stop at some point and refuse to allow time to elapse; it must allow time to pass to infinity if the environment does so. Previous studies of receptiveness properties include [10, 1, 40, 27].

The notion of receptiveness for TIOAs as discussed here is a special case of the same notion for HIOAs [25].

We build our definition of receptiveness on our earlier definition of progressive TIOAs. Namely, we define a *strategy* for resolving nondeterministic choices, and define receptiveness in terms of the existence of a progressive strategy.

We define a *strategy* for a TIOA $\mathcal{A}$ to be a TIOA $\mathcal{A}'$ that differs from $\mathcal{A}$ only in that $\mathcal{D}' \subseteq \mathcal{D}$ and $\mathcal{T}' \subseteq \mathcal{T}$. That is, we require:

- $\mathcal{D}' \subseteq \mathcal{D}$.
- $\mathcal{T}' \subseteq \mathcal{T}$.
- $X = X'$, $Q = Q'$, $\Theta = \Theta'$, $E = E'$, $H = H'$, $I = I'$, and $O = O'$.

Our strategies are nondeterministic and memoryless. They provide a way of choosing some of the evolutions that are possible from each state $\mathbf{x}$ of $\mathcal{A}$. The fact that the state set Q' of $\mathcal{A}'$ is the same as the state set Q of $\mathcal{A}$ implies that $\mathcal{A}'$ chooses evolutions from every state of $\mathcal{A}$.

Notions of strategy have been used also in previous studies of receptiveness [10, 1, 40, 27]. However, in these earlier works, strategies have been formalized using two-player games rather than automata. Defining strategies using automata allows us to avoid introducing extra mathematical machinery.

Lemma 6.5 *If $\mathcal{A}'$ is a strategy for $\mathcal{A}$, then every execution fragment of $\mathcal{A}'$ is also an execution fragment of $\mathcal{A}$.*

We define a TIOA to be *receptive* if it has a progressive strategy. The following theorem says that any receptive TIOA can respond to any inputs from the environment.

Theorem 6.6 *Every receptive TIOA is I/O feasible.*

Proof: The proof is similar to that of the corresponding theorem for HIOAs [25]. ■

Example 6.7 (Progressive and receptive TIOAs) The time-bounded channel automaton described in Example 4.1 is not progressive since it allows for an infinite execution in which `send` and `receive` actions alternate without any passage of time in between. The time-bounded channel automaton is receptive, however, as we may construct a progressive strategy for it by adding a condition $u = \text{now}$ to the precondition of the `receive` action. In this way we enforce that the channel operates maximally slow and messages are only delivered at their delivery deadline. The clock synchronization automaton of Example 4.6 is progressive (and therefore receptive) since it can only generate a locally controlled action each time its physical clock advances by u time units and the real time that elapses between two locally produced actions is at least $u \times (1-r)$ time units. ■

6.4 Implementation Relationships

Two TIOAs $\mathcal{A}_1$ and $\mathcal{A}_2$ are *comparable* if their inputs and outputs coincide, that is, if $I_1 = I_2$ and $O_1 = O_2$. If $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable, then $\mathcal{A}_1 \leq \mathcal{A}_2$ is defined to mean that the traces of $\mathcal{A}_1$ are included among those of $\mathcal{A}_2$: $\mathcal{A}_1 \leq \mathcal{A}_2 \triangleq \text{traces}_{\mathcal{A}_1} \subseteq \text{traces}_{\mathcal{A}_2}$.

Lemma 6.8 *Let $\mathcal{A}_1, \mathcal{A}_2$ be two comparable TIOAs and let $\mathcal{B}_1, \mathcal{B}_2$ be, respectively, the underlying TAs for $\mathcal{A}_1$ and $\mathcal{A}_2$. Then $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable and $\mathcal{A}_1 \leq \mathcal{A}_2$ iff $\mathcal{B}_1 \leq \mathcal{B}_2$.*

Proof: Immediate from the definitions. ■

6.5 Simulation Relations

The definition of forward simulation for TIOAs is the same as for TAs. Formally, if $\mathcal{A}_1 = (\mathcal{B}_1, I_1, O_1)$ and $\mathcal{A}_2 = (\mathcal{B}_2, I_2, O_2)$ are two comparable TIOAs, then a forward simulation from $\mathcal{A}_1$ to $\mathcal{A}_2$ is a forward simulation from $\mathcal{B}_1$ to $\mathcal{B}_2$.

Theorem 6.9 *If $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable TIOAs and there is a forward simulation from $\mathcal{A}_1$ to $\mathcal{A}_2$, then $\mathcal{A}_1 \leq \mathcal{A}_2$.*

The definitions and results about backward simulations, history and prophecy relations for timed automata from Section 4 carry over to timed automata with input and output distinction in a similar fashion.

7 Operations on Timed I/O Automata

7.1 Composition

In this section, we define the operations of composition and hiding and present projection, pasting and substitutivity results for TIOAs. We revisit the special kinds of TIOAs introduced in Section 6 and show that the classes of progressive and receptive timed I/O automata are closed under composition, while this is not true for the class of I/O feasible automata.

7.1.1 Definitions and Basic Results

The definition of composition for TIOAs is based on the corresponding definition for TAs, but also takes the input/output structure into account. We say that TIOAs $\mathcal{A}_1$ and $\mathcal{A}_2$ are *compatible* if, for $i \neq j$, $X_i \cap X_j = H_i \cap A_j = O_i \cap O_j = \emptyset$.

Lemma 7.1 *If $\mathcal{A}_1 = (\mathcal{B}_1, I_1, O_1)$ and $\mathcal{A}_2 = (\mathcal{B}_2, I_2, O_2)$ are compatible TIOAs, then $\mathcal{B}_1$ and $\mathcal{B}_2$ are compatible TAs.*

If $\mathcal{A}_1$ and $\mathcal{A}_2$ are compatible TIOAs then their *composition* $\mathcal{A}_1 \parallel \mathcal{A}_2$ is defined to be the tuple $\mathcal{A} = (\mathcal{B}, I, O)$ where

- $\mathcal{B} = \mathcal{B}_1 \parallel \mathcal{B}_2$,
- $I = (I_1 \cup I_2) - (O_1 \cup O_2)$
- $O = O_1 \cup O_2$.

Thus, an external action of the composition is classified as an output if it is an output of one of the component automata, and otherwise it is classified as an input. The composition of two TIOAs is guaranteed to be a TIOA:

Theorem 7.2 *If $\mathcal{A}_1$ and $\mathcal{A}_2$ are TIOAs then $\mathcal{A}_1 \parallel \mathcal{A}_2$ is a TIOA.*

Proof: The proof is straightforward except for showing that Axiom **E2** is satisfied by the composition. Let $\mathbf{x}$ be a state of $\mathcal{A}_1 \parallel \mathcal{A}_2$. We need to show the existence of a trajectory from $\mathbf{x}$ that satisfies **E2**.

By definition of $\mathcal{A}_1 \parallel \mathcal{A}_2$, $\mathbf{x} \upharpoonright X_1$ is a state of $\mathcal{A}_1$ and $\mathbf{x} \upharpoonright X_2$ is a state of $\mathcal{A}_2$. We know that both $\mathcal{A}_1$ and $\mathcal{A}_2$ satisfy **E2**. Let τ_1 be a trajectory of $\mathcal{A}_1$ with $\tau_1.fstate = \mathbf{x} \upharpoonright X_1$ that satisfies **E2**, let τ_2 be a trajectory of $\mathcal{A}_2$ with $\tau_2.fstate = \mathbf{x} \upharpoonright X_2$ that satisfies **E2**, and consider the following cases:

1. $\tau_1.ltime = \infty$ and $\tau_2.ltime = \infty$.
Then, define τ such that $\tau \downarrow X_1 = \tau_1$ and $\tau \downarrow X_2 = \tau_2$.
2. $\tau_1.ltime = \infty$ and τ_2 is closed where some $l \in L_2$ is enabled in $\tau_2.lstate$.
Then, define τ such that $\tau \downarrow X_1 = \tau_1 \upharpoonright \text{dom}(\tau_2)$ and $\tau \downarrow X_2 = \tau_2$.
3. τ_1 is closed where some $l \in L_1$ is enabled in $\tau_1.lstate$ and $\tau_2.ltime = \infty$.
Then, define τ such that $\tau \downarrow X_1 = \tau_1$ and $\tau \downarrow X_2 = \tau_2 \upharpoonright \text{dom}(\tau_1)$.
4. τ_1 is closed where some $l \in L_1$ is enabled in $\tau_1.lstate$ and τ_2 is closed where some $l \in L_2$ is enabled in $\tau_2.lstate$.
If $\text{dom}(\tau_1) \subseteq \text{dom}(\tau_2)$, then define τ such that $\tau \downarrow X_1 = \tau_1$ and $\tau \downarrow X_2 = \tau_2 \upharpoonright \text{dom}(\tau_1)$. Otherwise, define τ such that $\tau \downarrow X_1 = \tau_1 \upharpoonright \text{dom}(\tau_2)$ and $\tau \downarrow X_2 = \tau_2$.

In all the cases, by definition of trajectories for a TIOA, τ is a trajectory of $\mathcal{A}_1 \parallel \mathcal{A}_2$ from $\mathbf{x}$, which satisfies **E2** by construction.

■

Note that this theorem is stronger than the corresponding theorem (Theorem 6.12 in [25]) for general HIOAs. Two HIOAs $\mathcal{A}_1$ and $\mathcal{A}_2$ are required to be “strongly compatible” for their composition to be a hybrid I/O automaton. This extra condition is needed to rule out dependencies among external variables that may prevent the component automata from evolving together. The absence of external variables in TIOA eliminates this kind of problematic behavior. Thus, for the timed case, we do not require the notion of strong compatibility that was needed for the hybrid case.

Composition of TIOAs satisfies the following projection and pasting result, which follows from Theorem 5.4.

Theorem 7.3 *Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be comparable TIOAs, and let $\mathcal{A} = \mathcal{A}_1 \parallel \mathcal{A}_2$. Then $\text{traces}_{\mathcal{A}}$ is exactly the set of $(E, \emptyset)$ -sequences whose restrictions to $\mathcal{A}_1$ and $\mathcal{A}_2$ are traces of $\mathcal{A}_1$ and $\mathcal{A}_2$, respectively. That is, $\text{traces}_{\mathcal{A}} = \{\beta \mid \beta \text{ is an } (E, \emptyset)\text{-sequence and } \beta \upharpoonright (E_i, \emptyset) \in \text{traces}_{\mathcal{A}_i}, i = \{1, 2\}\}$.*

7.1.2 Substitutivity Results

The following theorem is analogous to Theorem 5.8 for TAs without input/output distinction. It shows that the introduction of the input/output distinction does not cause any changes to the substitutivity results we obtained for general TAs.

Theorem 7.4 *Suppose $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable TIOAs with $\mathcal{A}_1 \leq \mathcal{A}_2$. Suppose that $\mathcal{B}$ is a TIOA that is compatible with each of $\mathcal{A}_1$ and $\mathcal{A}_2$. Then $\mathcal{A}_1 \parallel \mathcal{B} \leq \mathcal{A}_2 \parallel \mathcal{B}$.*

The corollaries below follow from the Corollaries 5.9 and 5.10 of Theorem 5.8.

Corollary 7.5 *Suppose $\mathcal{A}_1, \mathcal{A}_2, \mathcal{B}_1$, and $\mathcal{B}_2$ are TIOAs, $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable, $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable, and each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with each of $\mathcal{B}_1$ and $\mathcal{B}_2$. If $\mathcal{A}_1 \leq \mathcal{A}_2$ and $\mathcal{B}_1 \leq \mathcal{B}_2$ then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.*

Corollary 7.6 *Suppose $\mathcal{A}_1, \mathcal{A}_2, \mathcal{B}_1$, and $\mathcal{B}_2$ are TAs, $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable, $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable, and each of $\mathcal{A}_1$ and $\mathcal{A}_2$ is compatible with each of $\mathcal{B}_1$ and $\mathcal{B}_2$. If $\mathcal{A}_1 \parallel \mathcal{B}_2 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ and $\mathcal{B}_1 \leq \mathcal{B}_2$ then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.*

The basic substitutivity theorem, Theorem 7.4, is desirable for any formalism for interacting processes. For design purposes, it enables one to refine individual components without violating the correctness of the system as a whole. For verification purposes, it enables one to prove that a composite system satisfies its specification by proving that

each component satisfies its specification, thereby breaking down the verification task into more manageable pieces. However, it might not always be possible or easy to show that each component $\mathcal{A}_1$ (resp. $\mathcal{B}_1$) satisfies its specification $\mathcal{A}_2$ (resp. $\mathcal{B}_2$) without using any assumptions about the environment of the component. *Assume-guarantee* style results such as those presented in [18, 36, 44, 1, 2, 17, 45] are special kinds of substitutivity results that state what *guarantees* are expected from each component in an environment constrained by certain *assumptions*. Since the environment of each component consists of the other components in the system, assume-guarantee style results need to break the circular dependencies between the assumptions and guarantees for components. We present below two assume-guarantee style theorems Theorem 7.7 and Corollary 7.8, which can be used for proving that a system specified as a composite automaton $\mathcal{A}_1 \parallel \mathcal{B}_1$ implements a specification represented by a composite automaton $\mathcal{A}_2 \parallel \mathcal{B}_2$.

The main idea behind Theorem 7.7 is to assume that $\mathcal{A}_1$ implements $\mathcal{A}_2$ in a context represented by $\mathcal{B}_2$, and symmetrically that $\mathcal{B}_1$ implements $\mathcal{B}_2$ in a context represented by $\mathcal{A}_2$ where $\mathcal{A}_2$ and $\mathcal{B}_2$ are automata whose trace sets are closed under limits. The requirement about limit-closure implies that $\mathcal{A}_2$ and $\mathcal{B}_2$ specify trace safety properties. Moreover, we assume that the trace sets of $\mathcal{A}_2$ and $\mathcal{B}_2$ are closed under time-extension. That is, the automata allow arbitrary time-passage. This is the most general assumption one could make to ensure that $\mathcal{A}_2 \parallel \mathcal{B}_2$ does not impose stronger constraints on time-passage than $\mathcal{A}_1 \parallel \mathcal{B}_1$. Note that the definitions of limit and time extension of a hybrid sequence can be found in Section 3.4.1.

Theorem 7.7 *Suppose $\mathcal{A}_1, \mathcal{A}_2, \mathcal{B}_1, \mathcal{B}_2$ are TIOAs such that $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable, $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable, and $\mathcal{A}_i$ is compatible with $\mathcal{B}_i$ for $i \in \{1, 2\}$. Suppose further that:*

1. *The sets $\text{traces}_{\mathcal{A}_2}$ and $\text{traces}_{\mathcal{B}_2}$ are closed under limits.*
2. *The sets $\text{traces}_{\mathcal{A}_2}$ and $\text{traces}_{\mathcal{B}_2}$ are closed under time-extension.*
3. *$\mathcal{A}_1 \parallel \mathcal{B}_2 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ and $\mathcal{A}_2 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.*

Then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.

Proof: We first prove by induction on the length of traces of $\mathcal{A}_1 \parallel \mathcal{B}_1$ that every closed trace of $\mathcal{A}_1 \parallel \mathcal{B}_1$ is a trace of $\mathcal{A}_2 \parallel \mathcal{B}_2$.

For the base case, let β be a trace of $\mathcal{A}_1 \parallel \mathcal{B}_1$ such that $\beta \in \text{trajs}(\emptyset)$ (a single trajectory over the empty set of variables). By Axiom **T0** in the definition of a TA, we know that $\mathcal{A}_2$ and $\mathcal{B}_2$ have traces α_1 and α_2 such that $\alpha_1.ltime = \alpha_2.ltime = 0$. By Assumption 2 we have $\alpha_1 \frown \beta \in \text{traces}_{\mathcal{A}_2}$ and $\alpha_2 \frown \beta \in \text{traces}_{\mathcal{B}_2}$. Since, $\alpha_1 \frown \beta = \beta$ and $\alpha_2 \frown \beta = \beta$, it follows that $\beta \in \text{traces}_{\mathcal{A}_2}$ and $\beta \in \text{traces}_{\mathcal{B}_2}$. By pasting using Theorem 7.3, $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$, as needed.

For the inductive step we consider the following cases:

1. $\beta = \beta' a \tau$, where a is an output action of $\mathcal{A}_1$ and τ is a point trajectory.

Then $\beta \upharpoonright (E_{\mathcal{A}_1}, \emptyset) \in \text{traces}_{\mathcal{A}_1}$ by projection using Theorem 7.3. By inductive hypothesis, $\beta' \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$. So $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$, by projection using Theorem 7.3. Let α be an execution of $\mathcal{B}_2$ such that $\text{trace}(\alpha) = \beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$. Since $\mathcal{A}_1$ and $\mathcal{B}_1$ are compatible TIOAs, $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable, and a is an output action of $\mathcal{A}_1$, we know that either a is an input action of $\mathcal{B}_2$ or the action set of $\mathcal{B}_2$ does not contain a . In the former case, by the input-enabling axiom (E1) we know that there exists $\mathbf{x}'$ such that $(\alpha.lstate, a, \mathbf{x}')$ is a discrete transition of $\mathcal{B}_2$. It follows that $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. In the latter case, since $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) = \beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$ and $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$ we get $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. By pasting using Theorem 7.3, $\beta \in \text{traces}_{\mathcal{A}_1 \parallel \mathcal{B}_2}$. Then by Assumption 3, $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$.

2. $\beta = \beta' b \tau$, where b is an output action of $\mathcal{B}_1$ and τ is a point trajectory.

This case is symmetric with the previous one.

3. $\beta = \beta' c \tau$, where c is an input action of both $\mathcal{A}_1$ and $\mathcal{B}_1$ and τ is a point trajectory.

By inductive hypothesis, $\beta' \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$. By projection using Theorem 7.3 we get $\beta' \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$ and $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. Let α be an execution of $\mathcal{A}_2$ such that $\text{trace}(\alpha) = \beta' \upharpoonright (E_{\mathcal{A}_2}, \emptyset)$. Since $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable and a is an input action of $\mathcal{A}_1$ we know that a is an input action of $\mathcal{A}_2$. By the input-enabling axiom (E1) we know that there exists $\mathbf{x}'$ such that $(\alpha.lstate, a, \mathbf{x}')$ is a discrete transition of $\mathcal{A}_2$. It follows that $\beta \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$. Similarly, let α' be an execution of $\mathcal{B}_2$ such that $\text{trace}(\alpha') = \beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$. Since $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable and a is an input action of $\mathcal{B}_1$ we know that a is an input action of $\mathcal{B}_2$. By the input-enabling axiom (E1) we know that there exists $\mathbf{y}'$ such that $(\alpha'.lstate, a, \mathbf{y}')$ is a discrete transition of $\mathcal{B}_2$. It follows that $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. By pasting using Theorem 7.3, we get $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$.

4. $\beta = \beta' d \tau$, where d is an input action of $\mathcal{A}_1$ but not an action of $\mathcal{B}_1$ and τ is a point trajectory.

By inductive hypothesis, $\beta' \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$. By projection using Theorem 7.3, we have $\beta' \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$ and $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. Let α be an execution of $\mathcal{A}_2$ such that $\text{trace}(\alpha) = \beta' \upharpoonright (E_{\mathcal{A}_2}, \emptyset)$. Since $\mathcal{A}_1$ and $\mathcal{A}_2$ are comparable TIOAs and a is an input action of $\mathcal{A}_1$, a must be an input action of $\mathcal{A}_2$. By the input-enabling axiom (E1) we know that there exists $\mathbf{x}'$ such that $(\alpha.lstate, a, \mathbf{x}')$ is a discrete transition of $\mathcal{A}_2$. It follows that $\beta \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$. Since $\mathcal{B}_1$ and $\mathcal{B}_2$ are comparable and a is not an action of $\mathcal{B}_1$, a cannot be an external action of $\mathcal{B}_2$. Therefore, $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) = \beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$. Since $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$ we get $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. By pasting using Theorem 7.3, we get $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$.

5. $\beta = \beta' d \tau$, where d is an input action of $\mathcal{B}_1$ but not an action of $\mathcal{A}_1$ and τ is a point trajectory.

This case is symmetric with the previous one.

6. $\beta = \beta' \frown \beta''$, where β'' is a hybrid sequence consisting of a single trajectory τ .

By inductive hypothesis, $\beta' \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$. By projection using Theorem 7.3, we get $\beta' \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$ and $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. By Assumption 2, we have $\beta' \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \frown \beta'' \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$ and $\beta' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \frown \beta'' \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. Then by pasting using Theorem 7.3, $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$, as needed.

We have thus shown that every closed trace of $\mathcal{A}_1 \parallel \mathcal{B}_1$ is a trace of $\mathcal{A}_2 \parallel \mathcal{B}_2$. Now consider any non-closed trace β of $\mathcal{A}_1 \parallel \mathcal{B}_1$. This β can be written as the limit of a sequence $\beta_1 \beta_2 \dots$ of closed traces of $\mathcal{A}_1 \parallel \mathcal{B}_1$. By the first part of the proof we know that each $\beta_i \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$, and by projection using Theorem 7.3 each $\beta_i \upharpoonright (E_{\mathcal{A}_2}, \emptyset)$ is a closed trace of $\mathcal{A}_2$, and $\beta_i \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$ is a closed trace of $\mathcal{B}_2$. We know that $\beta \upharpoonright (E_{\mathcal{A}_2}, \emptyset)$ is the limit of the $\beta_i \upharpoonright (E_{\mathcal{A}_2}, \emptyset)$ and similarly $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$ is the limit of the $\beta_i \upharpoonright (E_{\mathcal{B}_2}, \emptyset)$. Since the sets $\text{traces}_{\mathcal{A}_2}$ and $\text{traces}_{\mathcal{B}_2}$ are limit-closed by Assumption 1, we get $\beta \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$ and $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. Finally, by pasting using Theorem 7.3, we get $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$. ■

Note that automata with FIN and timing-independence (see Section 4.3 for definitions) constitute examples for context automata $\mathcal{A}_2$ and $\mathcal{B}_2$ that satisfy Assumptions 1 and 2. The property FIN implies Assumption 1 (Lemma 4.18) and timing-independence implies Assumption 2.

Theorem 7.7 has a corollary, Corollary 7.8 below, which can be used in the decomposition of proofs even when $\mathcal{A}_2$ and $\mathcal{B}_2$ neither admit arbitrary time-passage nor have limit-closed trace sets. The main idea behind this corollary is to assume that $\mathcal{A}_1$ implements $\mathcal{A}_2$ in a context $\mathcal{B}_3$ that is a variant of $\mathcal{B}_2$, and symmetrically that $\mathcal{B}_1$ implements $\mathcal{B}_2$ in a context that is a variant of $\mathcal{A}_2$. That is, the correctness of implementation relationship between $\mathcal{A}_1$ and $\mathcal{A}_2$ does not depend on all the environment constraints, just on those expressed by $\mathcal{B}_3$ (symmetrically for $\mathcal{B}_1, \mathcal{B}_2$, and $\mathcal{A}_3$). In order to use this corollary to prove $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ one needs to be able to find appropriate variants of $\mathcal{A}_2$ and $\mathcal{B}_2$ that meet the required closure properties. This corollary prompts one to pin down what is essential about the behavior of the environment in proving the intended implementation relationship, and also allows one to avoid the unnecessary details of the environment in proofs.

Corollary 7.8 *Suppose $\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3$ are TIOAs such that $\mathcal{A}_1, \mathcal{A}_2$, and $\mathcal{A}_3$ are comparable, $\mathcal{B}_1, \mathcal{B}_2$, and $\mathcal{B}_3$ are comparable, and $\mathcal{A}_i$ is compatible with $\mathcal{B}_i$ for $i \in \{1, 2, 3\}$. Suppose further that:*

1. *The sets $\text{traces}_{\mathcal{A}_3}$ and $\text{traces}_{\mathcal{B}_3}$ are closed under limits.*
2. *The sets $\text{traces}_{\mathcal{A}_3}$ and $\text{traces}_{\mathcal{B}_3}$ are closed under time-extension.*
3. *$\mathcal{A}_2 \parallel \mathcal{B}_3 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$ and $\mathcal{A}_3 \parallel \mathcal{B}_2 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$.*
4. *$\mathcal{A}_1 \parallel \mathcal{B}_3 \leq \mathcal{A}_2 \parallel \mathcal{B}_3$ and $\mathcal{A}_3 \parallel \mathcal{B}_1 \leq \mathcal{A}_3 \parallel \mathcal{B}_2$.*

Then $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.

Proof: Since $\mathcal{A}_2 \leq \mathcal{A}_3$ by Assumption 3 and $\mathcal{A}_1 \parallel \mathcal{B}_3 \leq \mathcal{A}_2 \parallel \mathcal{B}_3$ by Assumption 4, we get $\mathcal{A}_1 \parallel \mathcal{B}_3 \leq \mathcal{A}_2 \parallel \mathcal{B}_3 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$, by Theorem 7.4. Similarly, we have $\mathcal{A}_3 \parallel \mathcal{B}_1 \leq \mathcal{A}_3 \parallel \mathcal{B}_2 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$. Since $\mathcal{A}_1 \parallel \mathcal{B}_3 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$ and $\mathcal{A}_3 \parallel \mathcal{B}_1 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$, by using Assumptions 1 and 2, and Theorem 7.7 we have $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$.

Let β be a trace of $\mathcal{A}_1 \parallel \mathcal{B}_1$. By projection using Theorem 7.3, $\beta \upharpoonright (E_{\mathcal{A}_1}, \emptyset) \in \text{traces}_{\mathcal{A}_1}$ and $\beta \upharpoonright (E_{\mathcal{B}_1}, \emptyset) \in \text{traces}_{\mathcal{B}_1}$. Since $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_3 \parallel \mathcal{B}_3$, we know that $\beta \in \text{traces}_{\mathcal{A}_3 \parallel \mathcal{B}_3}$. By projection using Theorem 7.3, $\beta \upharpoonright (E_{\mathcal{A}_3}, \emptyset) \in \text{traces}_{\mathcal{A}_3}$ and $\beta \upharpoonright (E_{\mathcal{B}_3}, \emptyset) \in \text{traces}_{\mathcal{B}_3}$. By pasting using Theorem 7.3, we have $\beta \in \text{traces}_{\mathcal{A}_1 \parallel \mathcal{B}_3}$ and $\beta \in \text{traces}_{\mathcal{A}_3 \parallel \mathcal{B}_1}$. By Assumption 4, we get $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_3}$ and $\beta \in \text{traces}_{\mathcal{A}_3 \parallel \mathcal{B}_2}$. Then, by projection using Theorem 7.3, $\beta \upharpoonright (E_{\mathcal{A}_2}, \emptyset) \in \text{traces}_{\mathcal{A}_2}$ and $\beta \upharpoonright (E_{\mathcal{B}_2}, \emptyset) \in \text{traces}_{\mathcal{B}_2}$. Finally, by pasting using Theorem 7.3 we have $\beta \in \text{traces}_{\mathcal{A}_2 \parallel \mathcal{B}_2}$, as needed. $\blacksquare$

Example 7.9 (Using environment assumptions to prove safety) This example illustrates that, in cases where specifications $\mathcal{A}_2$ and $\mathcal{B}_2$ satisfy certain closure properties, it is possible to decompose the proof of $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ by using Theorem 7.7, even if it is not the case that $\mathcal{A}_1 \leq \mathcal{A}_2$ or $\mathcal{B}_1 \leq \mathcal{B}_2$.

The automata `AlternateA` and `AlternateB` in Figure 12 are timing-independent automata in which no consecutive outputs occur without inputs happening in between. `AlternateA` and `AlternateB` perform a handshake, outputting an alternating sequence of *a* and *b* actions when they are composed. The automata `CatchUpA` and `CatchUpB` in Figure 9 are timing-dependent automata that do not necessarily alternate inputs and outputs as `AlternateA` and `AlternateB`. `CatchUpA` can perform an arbitrary number of *b* actions, and can perform an *a* provided that `counta` $\leq$ `countb`. It allows `counta` to increase to one more than `countb`. `CatchUpB` can perform an arbitrary number of *a* actions, and can perform a *b* provided that `counta` $\geq$ `countb` + 1. It allows `countb` to reach `counta`. Timing constraints require each output to occur exactly one time unit after the last action. `CatchUpA` and `CatchUpB` perform an alternating sequence of *a* actions and *b* actions when they are composed.

Suppose that we want to prove that $\text{CatchUpA} \parallel \text{CatchUpB} \leq \text{AlternateA} \parallel \text{AlternateB}$. We cannot apply the basic substitutivity theorem Theorem 7.7, in particular Corollary 7.5, since the assertions $\text{CatchUpA} \leq \text{AlternateA}$ and $\text{CatchUpB} \leq \text{AlternateB}$ are not true. Consider the trace $\tau_0 \mathbf{b} \tau_1 \mathbf{a} \tau_2 \mathbf{a} \tau_3$ of `CatchUpA` where τ_0 , τ_1 , τ_2 and τ_3 are trajectories with limit time 1. After having performed one *b* and one *a*, `CatchUpA` can perform another *a*. But, this is impossible for `AlternateA` which needs an input to enable the second *a*. `AlternateA` and `CatchUpA` behave similarly only when put in a context that imposes alternation.

It is easy to check that `AlternateA` and `AlternateB` satisfy the closure properties required by Assumptions 1 and 2 of Theorem 7.7 and, hence can be substituted for $\mathcal{A}_2$

```

automaton AlternateA
  signature
    output a, input b
  states
    myturn: Bool := true
  transitions
    output a
      pre
        myturn
      eff
        myturn := false
    input b
      eff
        myturn := true

```

```

automaton AlternateB
  signature
    input a, output b
  states
    myturn: Bool := false
  transitions
    input a
      eff
        myturn := true
    output b
      pre
        myturn
      eff
        myturn := false

```

Figure 12: AlternateA and AlternateB

and $\mathcal{B}_2$ respectively. Similarly, we can easily check that Assumption 3 is satisfied if we substitute CatchUpA for $\mathcal{A}_1$ and CatchUpB for $\mathcal{B}_1$. ■

Example 7.10 (Extracting essential environment assumptions with auxiliary automata) This example illustrates that it may be possible to decompose verification, using Corollary 7.8, in cases where Theorem 7.7 is not applicable. If the aim is to show $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$ where $\mathcal{A}_2$ and $\mathcal{B}_2$ do not satisfy the assumptions of Theorem 7.7, then we find appropriate context automata $\mathcal{A}_3$ and $\mathcal{B}_3$ that abstract from those details of $\mathcal{A}_2$ and $\mathcal{B}_2$ that are not essential in proving $\mathcal{A}_1 \parallel \mathcal{B}_1 \leq \mathcal{A}_2 \parallel \mathcal{B}_2$.

Consider the automata UseOldInputA and UseOldInputB in Figure 13. UseOldInputA keeps track of whether or not it is UseOldInputA 's turn, and when it is UseOldInputA 's turn, it keeps track of the next time it is supposed to perform an output. The number of outputs that UseOldInputA can perform is bounded by a natural number. In the case

```

signature
  output a, input b
states
  myturn: Bool := true, maxout: Nat,
  now: Real := 0, next: AugmentedReal := 0
transitions
  output a
    pre
      myturn  $\wedge$  (maxout > 0)
       $\wedge$  (now = next)
    eff
      myturn := false;
      maxout := maxout - 1;
      next := infty
  input b
    eff
      myturn := true;
      if next = infty
        then next := now + 1
trajectories
  stop when
    now = next
  evolve
    d(now) = 1

```

```

signature
  input a, output b
states
  myturn: Bool := true, maxout: Nat,
  now: Real := 0, next: AugmentedReal := 0
transitions
  input a
    eff
      myturn := true;
      if next = infty
        then next := now + 1
  output b
    pre
      myturn  $\wedge$  (maxout > 0)
       $\wedge$  (now = next)
    eff
      myturn := false;
      maxout := maxout - 1;
      next := infty
trajectories
  stop when
    now = next
  evolve
    d(now) = 1

```

Figure 13: UseOldInputA and UseOldInputB

```

signature
  output a, input b
states
  myturn: Bool := true, maxout: Nat,
  now: Real := 0, next: AugmentedReal := 0
transitions
  output a
    pre
      myturn  $\wedge$  (maxout > 0)
       $\wedge$  (now = next)
    eff
      myturn := false;
      maxout := maxout - 1;
      next := infty
  input b
    eff
      myturn := true;
      next := now + 1
trajectories
  stop when
    now = next
  evolve
    d(now) = 1

```

```

signature
  input a, output b
states
  myturn: Bool := true, maxout: Nat,
  now: Real := 0, next: AugmentedReal := 0
transitions
  input a
    eff
      myturn := true;
      next := now + 1
  output b
    pre
      myturn  $\wedge$  (maxout > 0)
       $\wedge$  (now = next)
    eff
      myturn := false;
      maxout := maxout - 1;
      next := infty
trajectories
  stop when
    now = next
  evolve
    d(now) = 1

```

Figure 14: UseNewInputA and UseNewInputB

of repeated b inputs, it is the oldest input that determines when the next output will occur. The automaton `UseOldInputB` is the same as `UseOldInputA` (inputs and outputs reversed) except that the turn variable of `UseOldInputB` is set to false initially. Note that `UseOldInputA` and `UseOldInputB` are not timing-independent and their trace sets are not limit-closed. For each automaton, there are infinitely many start states, one for each natural number. We can build an infinite chain of traces, where each element in the chain corresponds to an execution starting from a distinct start state. The limit of such a chain, which contains infinitely many outputs, cannot be a trace of `UseOldInputA` or `UseOldInputB` since the number of outputs they can perform is bounded by a natural number. The automaton `UseNewInputA` in Figure 14 behaves similarly to `UseOldInputA` except for the handling of inputs. In the case of repeated b inputs, it is the most recent input that determines when the next output will occur. The automaton `UseNewInputB` in Figure 14 is the same as `UseNewInputA` (inputs and outputs reversed) except that the turn variable of `UseNewInputB` is set to false initially.

Suppose that we want to prove that: $\text{UseNewInputA} \parallel \text{UseNewInputB} \leq \text{UseOldInputA} \parallel \text{UseOldInputB}$.

Theorem 7.7 is not applicable here because the high-level automata `UseOldInputA` and `UseOldInputB` do not satisfy the required closure properties. However, we can use Corollary 7.8 to decompose verification. It requires us to find auxiliary automata that are less restrictive than `UseOldInputA` and `UseOldInputB` but that are restrictive enough to express the constraints that should be satisfied by the environment, for `UseNewInputA` to implement `UseOldInputA` and for `UseNewInputB` to implement `UseOldInputB`.

The automata `AlternateA` and `AlternateB` in Figure 12 can be used as auxiliary automata in this example. They satisfy the closure properties required by Corollary 7.8 and impose alternation, which is the only additional condition to ensure the needed trace inclusion.

We can define a forward simulation relation from $\text{UseNewInputA} \parallel \text{UseNewInputB}$ to $\text{UseOldInputA} \parallel \text{UseOldInputB}$, which is based on the equality of the turn variables of the implementation and the specification automata. The fact that this simulation relation only uses the equality of turn variables reinforces the idea that the auxiliary contexts, which only keep track of their turn, capture exactly what is needed for the proof of $\text{UseNewInputA} \parallel \text{UseNewInputB} \leq \text{UseOldInputA} \parallel \text{UseOldInputB}$. We can observe that a direct proof of this assertion would require one to deal with state variables such as `maxout` and `next` of both `UseOldInputA` and `UseOldInputB` which do not play any essential role in the proof. On the other hand, by decomposing the proof along the lines of Corollary 7.8 some of the unnecessary details can be avoided. Even though, this is a toy example with an easy proof it should not be hard to observe how this simplification would scale to large proofs.

■

7.1.3 Composition of Special Kinds of TIOAs

The following example illustrates that the set of I/O feasible TIOAs is not closed under composition:

Example 7.11 (Two I/O feasible TIOAs whose composition is not I/O feasible)

Consider two I/O feasible TIOAs $\mathcal{A}$ and $\mathcal{B}$, where $O_{\mathcal{A}} = I_{\mathcal{B}} = \{a\}$ and $O_{\mathcal{B}} = I_{\mathcal{A}} = \{b\}$. Suppose that $\mathcal{A}$ performs its output a at time 0 and then waits, allowing time to pass, until it receives input b . If and when it receives b , it responds with output a without allowing any time to pass (and ignoring any inputs that occur before it has a chance to perform its output). On the other hand, $\mathcal{B}$ starts out waiting, allowing time to pass, until it receives input a . If and when it receives a , it responds with output b without allowing time to pass.

It is not difficult to see that $\mathcal{A}$ and $\mathcal{B}$ are individually I/O feasible. We claim that the composition $\mathcal{A} \parallel \mathcal{B}$ is not I/O feasible. To see this, consider the start state of $\mathcal{A} \parallel \mathcal{B}$ and the unique input sequence β with $\beta.ltime = \infty$; β simply allows time to pass to infinity. The composition $\mathcal{A} \parallel \mathcal{B}$ has no way of accommodating this input, since it will never allow time to pass beyond 0. ■

On the other hand, the following theorems say that the classes of progressive and receptive TIOAs are closed under composition:

Theorem 7.12 *If $\mathcal{A}_1$ and $\mathcal{A}_2$ are compatible progressive TIOAs, then their composition is also progressive.*

Proof: The proof is similar to the proof of Theorem 7.4 in [25]. The main idea behind the proof is that a Zeno execution of $\mathcal{A}_1 \parallel \mathcal{A}_2$ with infinitely many locally controlled contains infinitely many locally controlled actions of either $\mathcal{A}_1$ or $\mathcal{A}_2$. Suppose without loss of generality that the automaton that contributes infinitely many locally controlled actions is $\mathcal{A}_1$. Then the projection onto $\mathcal{A}_1$ violates progressiveness for $\mathcal{A}_1$. ■

Theorem 7.13 *Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two compatible TIOAs with strategies $\mathcal{A}'_1$ and $\mathcal{A}'_2$, respectively. Then $\mathcal{A}'_1 \parallel \mathcal{A}'_2$ is a strategy for $\mathcal{A}_1 \parallel \mathcal{A}_2$.*

Proof: The proof is similar to the proof of Theorem 7.7 in [25]. ■

Now, we can state the main result of this section, which follows easily from the previous two theorems. It shows that the class of receptive TIOAs is closed under composition.

Theorem 7.14 *Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two compatible receptive TIOAs with progressive strategies $\mathcal{A}'_1$ and $\mathcal{A}'_2$, respectively. Then $\mathcal{A}_1 \parallel \mathcal{A}_2$ is a receptive TIOA with progressive strategy $\mathcal{A}'_1 \parallel \mathcal{A}'_2$.*

Example 7.15 (Composition of receptive TIOAs) Theorem 7.14 implies that the composition of clock synchronization automata with channel automata described in Example 5.7 (viewed as TIOAs as explained in Example 6.1) is receptive. By Theorem 6.6 we also have that it is I/O feasible. ■

In fact, the fact that the set of I/O feasible TIOAs is not closed under composition motivated the definition of the more restrictive class of receptive TIOAs. That is, receptiveness is a reasonable sufficient condition that implies I/O feasibility, and that also is preserved by composition.

The special case of the HIOA model, represented by the TIOA model, has simpler and stronger composition theorems than the general HIOA model. In particular, the main compositionality result for receptive HIOAs (Theorem 7.12 in [25]) has a more intricate proof than ours. It makes an assumption about the existence of strongly compatible strategies (discussed briefly at the end of Section 7.1.1) and needs an additional lemma that shows that if two HIOAs $\mathcal{A}_1$ and $\mathcal{A}_2$ which may not be strongly compatible have strongly compatible strategies $\mathcal{A}'_1$ and $\mathcal{A}'_2$, then $\mathcal{A}_1$ and $\mathcal{A}_2$ are also strongly compatible.

7.2 Hiding

We extend the definition of action hiding to any TIOA $\mathcal{A}$. For TIOAs, we consider hiding outputs only (but not inputs), by converting them to internal actions. Namely, if $O \subseteq O_{\mathcal{A}}$, then $\text{ActHide}(O, \mathcal{A})$ is the TIOA $\mathcal{B}$ that is equal to $\mathcal{A}$ except that $O_{\mathcal{B}} = O_{\mathcal{A}} - O$ and $H_{\mathcal{B}} = H_{\mathcal{A}} \cup O$.

Lemma 7.16 *If $\mathcal{A}$ is a TIOA and $O \subseteq O_{\mathcal{A}}$ then $\text{ActHide}(O, \mathcal{A})$ is a TIOA.*

Lemma 7.17 *If $\mathcal{A}$ is a TIOA and $O \subseteq O_{\mathcal{A}}$ then $\text{traces}_{\text{ActHide}(O, \mathcal{A})} = \{\beta \upharpoonright (O_{\mathcal{A}} - O, V_{\mathcal{A}}) \mid \beta \in \text{traces}_{\mathcal{A}}\}$.*

Theorem 7.18 *Suppose $\mathcal{A}$ and $\mathcal{B}$ are TIOAs with $\mathcal{A} \leq \mathcal{B}$, and suppose $O \subseteq O_{\mathcal{A}}$. Then $\text{ActHide}(O, \mathcal{A}) \leq \text{ActHide}(O, \mathcal{B})$.*

8 Conclusions and Future Work

In this monograph, we have presented a new framework for describing and analyzing the behavior of timed systems. This framework is a mathematical framework that uses timed I/O automata for the representation of systems. The TIOA framework is a special case of the hybrid I/O automaton modeling framework [25]. We used what we have learned in developing the HIOA framework to revise the earlier work on timed I/O automaton models. Our main motivation was to have a timed I/O automaton model that is compatible with

the new HIOA model. We sought to benefit from the new style used in describing hybrid behavior in simplifying the prior definitions and results on timed I/O automata.

Designers of real-time systems or timing-based algorithms can use the TIOA framework to describe complex systems and to decompose them into manageable pieces. In particular, they can use the TIOA framework to describe their systems at multiple levels of abstraction, to establish implementation relationships between these levels and to decompose their systems into more primitive, interacting components. Although the framework as presented in this monograph provides only conceptual tools for modeling, and manual proof methods, it also is a natural basis for building computerized modeling and analysis.

We are currently working on the development of a toolset based on this mathematical framework that will consist of: (a) a formal modeling language called TIOA, (b) the front-end processor for TIOA, incorporating syntax and static semantic checking, and providing interfaces to computer-aided design tools, (c) a simulation tool allowing simulation of specifications and paired simulations of a specification and an abstract implementation, and (d) a theorem-proving link through an interface to the theorem-prover PVS [3].³ The described project builds upon our prior work on the IOA language [12].

On the theoretical side, we have done preliminary research toward extending the TIOA framework with support for reasoning about safety and liveness properties of timed systems. We have defined notions of fairness and proved results that state under which conditions the “fair” traces of a TIOA can be shown to be included in the fair traces of another. We have started investigating the consequences of composition on automata with liveness properties and the use of receptiveness and strategies in this context[21]. The results of these lines of preliminary work are not included in this version of the monograph because the adequacy of our definitions and methods are yet to be assessed on a larger class of non-trivial examples.

We will also continue our work on establishing formal relationships with other models that are comparable to ours, showing that the TIOA framework is general enough to express previous results from other frameworks, such as [32, 31, 5, 30, 28, 40].

Acknowledgments: Dilsun Kaynar and Nancy Lynch were supported by DARPA/AFOSR MURI Contract F49620-02-1-0325, DARPA SEC contract F33615-01-C-1850, NSF ITR contract CCR-0121277, and Air Force Aerospace Research-OSR Contract F49620-00-1-0097. Frits Vaandrager was supported by EU IST project IST-2001-3530 Advanced Methods for Timed Systems (AMETIST) and PROGRESS project TES4999: Verification of Hard and Softly Timed Systems (HaaST).

³Parts of the TIOA software are available upon request for experimental use. The project web site (<http://tioa.csail.mit.edu>) contains all the material that are available for general public use.

References

- [1] M. Abadi and L. Lamport. Composing specifications. *ACM Transactions on Programming Languages and Systems*, 1(15):73–132, 1993.
- [2] M. Abadi and L. Lamport. Conjoining specifications. *ACM Transactions on Programming Languages and Systems*, 17(3):507–534, 1995.
- [3] R. Alur. Timed automata. In *Proc. of 11th International Conference on Computer-Aided Verification (CAV)*, volume 1633 of *LNCS*, pages 8–22. Springer-Verlag, 1999. An earlier and longer version appears in NATO-ASI Summer School on Verification of Digital and Hybrid Systems.
- [4] R. Alur, C. Courcoubetis, N. Halbwachs, T. A. Henzinger, P. H. Ho, X. Nicolin, A. Olivero, J. Sifakis, and Yovine S. The algorithmic analysis of hybrid systems. *Theoretical Computer Science*, 138:3–34, 1995.
- [5] R. Alur and D.L. Dill. A theory of timed automata. *Theoretical Computer Science*, 126:183–235, 1994.
- [6] R. Alur and P. Madhusudan. Decision problems for timed automata: A survey. 4th Intl. School on Formal Methods for Computer, Communication, and Software Systems: Real Time, 2004. Bertinoro, Italy.
- [7] R. Alur, S. La Torre, and P. Madhusudan. Perturbed timed automata,. In *Proceedings of the Eighth International Workshop on Hybrid Systems: Computation and Control (HSCC)*, 2004.
- [8] M. Bozga, S. Graf, Iul. Ober Il. Ober, and J. Sifakis. The IF toolset. In *Proceedings of Formal Methods for the Design of Real-Time Systems*, volume 3185 of *LNCS*, pages 237–267, September 2004.
- [9] Roberto DePrisco, Butler Lampson, and Nancy Lynch. Revisiting the Paxos algorithm. In Marios Mavronicolas and Philippas Tsigas, editors, *Distributed Algorithms* 11th International Workshop, WDAG’97, Saarbrücken, Germany, September 1997 Proceedings, volume 1320 of *Lecture Notes in Computer Science*, pages 111–125, Berlin-Heidelberg, 1997. Springer-Verlag.
- [10] D. Dill. *Trace Theory for Automatic Hierarchical Verification of Speed-Independent Circuits*. ACM Distinguished Dissertations. MIT Press, 1988.
- [11] S. Garland and N. Lynch. Using I/O automata for developing distributed systems. In Gary T. Leavens and Murali Sitaraman, editors, *Foundations of Component-Based Systems*, chapter 13, pages 285–312. Cambridge University Press, New York, 2000.

- [12] S. Garland, N. Lynch, and M. Vaziri. *IOA: A Language for Specifying, Programming, and Validating Distributed Systems*. MIT Laboratory for Computer Science, Cambridge, MA, 2001. URL <http://theory.lcs.mit.edu/tds/ioa.html>.
- [13] R. Gawlick, R. Segala, J.F. Søggaard-Andersen, and N.A. Lynch. Liveness in timed and untimed systems. In S. Abiteboul and E. Shamir, editors, *Proceedings 21th ICALP*, Jerusalem, volume 820 of *Lecture Notes in Computer Science*. Springer-Verlag, 1994. A full version appears as MIT Technical Report number MIT/LCS/TR-587.
- [14] C.A. Gunter. *Semantics of Programming Languages: Structures and Techniques*. MIT Press, Cambridge, Massachusetts, 1992.
- [15] M. Hennessy. *Algebraic Theory of Processes*. MIT Press, Cambridge, Massachusetts, 1988.
- [16] T. A. Henzinger, P.-H. Ho, and H. Wong-Toi. HYTECH: A Model Checker for Hybrid Systems. In O. Grumberg, editor, *Proceedings of the 9th International Conference on Computer Aided Verification*, volume 1254 of *Lecture Notes in Computer Science*, pages 460–463. Springer-Verlag, 1997.
- [17] T. A. Henzinger, S. Qadeer, and S. K. Rajamani. Decomposing refinement proofs using assume-guarantee reasoning. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD)*, pages 245–252. IEEE Computer Society Press, 2000.
- [18] C. B. Jones. Specification and design of parallel programs. In R. E. A. Mason, editor, *Information Processing 83: Proceedings of the IFIP 9th World Congress*, pages 321–332. North-Holland, 1983.
- [19] D. Kaynar, Nancy Lynch, and Sayan Mitra. Specifying and proving timing properties with tioa tools. In *Proceeding sof the 5th IEEE International Real-Time Systems Symposium, Work in Progress Session (RTSS WIP)*, pages 96–99, Lisbon, Portugal, December 2004.
- [20] Dilsun Kaynar, Nancy Lynch, Sayan Mitra, and Stephen Garland. The TIOA language. Manuscript, 2004.
- [21] Dilsun K. Kaynar, Nancy Lynch, Roberto Segala, and Frits Vaandrager. The theory of timed I/O automata. Technical Report MIT-LCS-TR-917a, MIT Laboratory for Computer Science, 2004. Available online at <http://theory.csail.mit.edu/tds/reflist.html>.
- [22] D. E. Knuth. *Fundamental Algorithms*, volume 1 of *The Art of Computer Programming*. Addison-Wesley, Reading, Massachusetts, second edition, 1973.
- [23] Kim G. Larsen, Paul Pettersson, and Wang Yi. Uppaal in a nutshell. *Journal of Software Tools for Technology Transfer*, 1–2:134–152, 1997.

- [24] N.A. Lynch. *Distributed Algorithms*. Morgan Kaufmann Publishers, Inc., San Francisco, California, 1996.
- [25] N.A. Lynch, R. Segala, and F.W. Vaandrager. Hybrid I/O automata. *Information and Computation*, 185(1):105–157, 2003. Also Technical Report MIT-LCS-TR-827d, MIT Laboratory for Computer Science.
- [26] N.A. Lynch, R. Segala, F.W. Vaandrager, and H.B. Weinberg. Hybrid I/O automata. In R. Alur, T.A. Henzinger, and E.D. Sontag, editors, *Hybrid Systems III*, volume 1066 of *Lecture Notes in Computer Science*, pages 496–510. Springer-Verlag, 1996.
- [27] N.A. Lynch, R. Segala, F.W. Vaandrager, and H.B. Weinberg. Hybrid I/O automata. Report CSI-R9907, Computing Science Institute, University of Nijmegen, April 1999.
- [28] N.A. Lynch and F.W. Vaandrager. Action transducers and timed automata. *Formal Aspects of Computing*, 8(5):499–538, 1996.
- [29] Nancy Lynch and Alex Shvartsman. RAMBO: A reconfigurable atomic memory service for dynamic networks. In D. Malkhi, editor, *Distributed Computing (Proceedings of the 16th International Symposium on Distributed Computing (DISC), Toulouse, France, October 2002)*, volume 2508 of *Lecture Notes in Computer Science*, pages 173–190. Springer-Verlag, 2002. Also, Technical Report MIT-LCS-TR-856.
- [30] Nancy Lynch and Frits Vaandrager. Forward and backward simulations — Part II: Timing-based systems. *Information and Computation*, 128(1):1–25, July 1996.
- [31] O. Maler, Z. Manna, and A. Pnueli. From timed to hybrid systems. In J.W. de Bakker, C. Huizing, W.P. de Roever, and G. Rozenberg, editors, *Proceedings REX Workshop on Real-Time: Theory in Practice*, Mook, The Netherlands, June 1991, volume 600 of *Lecture Notes in Computer Science*, pages 447–484. Springer-Verlag, 1992.
- [32] M. Merritt, F. Modugno, and M. Tuttle. Time constrained automata. In J.C.M. Baeten and J.F. Groote, editors, *Proceedings CONCUR 91*, Amsterdam, volume 527 of *Lecture Notes in Computer Science*, pages 408–423. Springer-Verlag, 1991.
- [33] R. Milner. *A Calculus of Communicating Systems*, volume 92 of *Lecture Notes in Computer Science*. Springer-Verlag, 1980.
- [34] S. Mitra, Y. Wang, N. Lynch, and E. Feron. Safety verification of pitch controller for model helicopter. In O. Maler and A. Pnueli, editors, *Proc. of Hybrid Systems: Computation and Control*, volume 2623 of *Lecture Notes in Computer Science*, pages 343–358, Prague, the Czech Republic April 3-5, 2003.
- [35] Paul Pettersson. *Modelling and Verification of Real-Time Systems Using Timed Automata: Theory and Practice*. PhD thesis, Department of Computer Systems, Uppsala University, 1999. Technical Report DoCs 99/101.

- [36] A. Pnueli. In transition from global to modular temporal reasoning about programs. In K. R. Apt, editor, *Logis and Models of Concurrent Systems*, NATO ASI, pages 123–144. Springer-Verlag, 1984.
- [37] A. Pnueli. Development of hybrid systems. In H. Langmaack, W.-P. de Roever, and J. Vytöpil, editors, *Proceedings of the Third International School and Symposium on Formal Techniques in Real-Time and Fault-Tolerant Systems (FTRTFT'94)*, Lübeck, Germany, September 1994, volume 863 of *Lecture Notes in Computer Science*, pages 77–85. Springer-Verlag, 1994.
- [38] J.W. Polderman and J.C. Willems. *Introduction to Mathematical Systems Theory: A Behavioural Approach*, volume 26 of *Texts in Applied Mathematics*. Springer-Verlag, 1998.
- [39] C. Robson. TIOA and UPPAAL. Master's thesis, MIT Department of Electrical Engineering and Computer Science, 2004.
- [40] R. Segala, R. Gawlick, J.F. Sögaard-Andersen, and N.A. Lynch. Liveness in timed and untimed systems. *Information and Computation*, 141(2):119–171, March 1998.
- [41] J. Sifakis. Modeling real-time systems – challenges and work directions. In *Proceedings of Embedded Software, First International Workshop (EMSOFT '01)*, volume 2211 of *LNCS*, pages 373–389, Tahoe City, CA, USA, October 2001.
- [42] J. Sifakis. Modeling real-time systems. In *Proceedings of the 25th IEEE Real-Time Systems Symposium (RTSS '04)*, pages 5–6. IEEE Computer Society, 2004. Invited Talk.
- [43] E.D. Sontag. *Mathematical Control Theory — Deterministic Finite Dimensional Systems*, volume 6 of *Texts in Applied Mathematics*. Springer-Verlag, 1990.
- [44] E. W. Stark. A proof technique for relt/guarantee properties. In S. N. Maheshwari, editor, *Foundations of Software Technology and Theoretical Computer Science*, volume 206 of *LNCS*, pages 369–391. Springer-Verlag, 1985.
- [45] S. Tasiran, R. Alur, R.P. Kurshan, and R.K. Brayton. Verifying abstractions of timed systems. In *Proceedings of the Seventh Conference on Concurrency Theory (CONCUR) 1996*, volume 1119 of *LNCS*.
- [46] S. Yovine. Kronos: A verification tool for real-time systems. *International Journal of Software Tools for Technology Transfer*, 1(1/2):123–133, October 1997.

A Notational Conventions

a, b	action
f, g, h	function
i, j	index
l	locally controlled action
t	time point
v, x	variable
A	set of actions
C	task
E	set of external actions
F	set of functions
H	set of internal (hidden) actions
I	set of input actions
J	interval
K	set of time points
L	set of locally controlled actions
O	set of output actions
P	set of elements in cpo
Q	set of automaton states
R	(simulation) relation
S	set
T	set of trajectories
V	set of variables
X	set of internal variables
$\mathbf{x}$	state
$\mathbf{v}$	valuation
$\mathcal{A}, \mathcal{B}, \mathcal{C}$	timed (I/O) automaton
$\mathcal{D}$	set of discrete transitions
$\mathcal{T}$	set of trajectories
$\mathbb{N}$	the natural numbers
$\mathbb{R}$	the real numbers
$\mathbb{T}$	the time axis
$\mathbb{Z}$	the integers
$\mathbb{V}$	the universe of variables
α, β, δ	(A, V) -sequence
γ	sequence
λ	the empty sequence
π	projection function
σ, ρ	sequence
τ, v	trajectory
Θ	set of start states

Index

- (A, V)-restriction, 18
- (A, V)-sequence, 16
- abstraction, 5
- admissible, 17, 19
- algebraic cpo, 11
- AlternateA, 79
- AlternateB, 79
- Alur-Dill timed automaton, 7
- analog variable, 13, 22
- assume-guarantee, 76
- backward simulation, *see* simulation relation, 45
- BoundedAlternateA, 57
- BoundedAlternateB, 57
- CatchUpA, 57, 79
- CatchUpB, 57, 79
- chain, 11
- clock synchronization, 29, 41
- ClockSync($u, r: \text{Real}, i: \text{Index}$), 29, 54, 69
- compact element of a cpo, 11
- comparable, 73
 - TA, 37
- compatible, 73
 - TA, 51
- complete partial order (cpo), 11
 - algebraic cpo, 11
 - compact element, 11
- composition, 5, 51, 74
- continuous, 11
- cpo, *see* complete partial order
- discrete action, 19
- discrete transition, 19
- discrete variable, 13, 22
- dynamic type, 13
- effect, 22
- enabled, 20
- execution, 30, 69
 - PeriodicSend($u: \text{Real}, M: \text{Type}$), 31
 - Timeout($b: \text{Real}, M: \text{Type}$), 32
- execution fragment, 30, 31, 69
- feasible, 36, 69
- FIN, *see* finite internal nondeterminism, 78
- finite internal nondeterminism (FIN), 35
- Fischer’s mutual exclusion, 26, 33, 66
- FischerME($u_set, l_check: \text{Real}$), 26
- FischerME($u_set, l_check: \text{Real}$), 66
- forward simulation, *see* simulation relation
 - clock synchronization, 41
 - time-bounded channels, 40
- hiding, 60
- HIOA, 6, 75
- history relation, 46, 47, 73
 - time-bounded channels, 49
- history variable, 46, 47
 - time-bounded channels, 46
- hybrid automaton, 20, 51
- Hybrid I/O Automaton modeling framework, 6, 85
- hybrid sequence, 16, 16
 - admissible, 17
 - closed, 17
 - concatenation, 17
 - limit time, 16
 - prefix, 17
 - time-bounded, 17
 - Zeno, 17
- HyTech, 8
- I/O feasibility, 85
- I/O feasible, 70, 84
- implementation, 5, 37
- invariant, 30
 - clock agreement, 56
 - clock validity, 55
 - ClockSync($u, r: \text{Real}, i: \text{Index}$), 55, 56

- failure and timeout, 54
- `FischerME(u_set, l_check:Real)`, 33
- `TimedChannel(b:Real, M:Type)`, 33
- timeout, 53
- isomorphism, 43
- Kronos, 8
- limit of a chain, 11
- linear hybrid automaton, 8
- locally Zeno, 69
- monotone, 11
- non-Zeno, 17, 19
- parallel composition, *see* composition
- partial order, 11
 - complete partial order, 11
- periodic sending process, 24, 31
- periodic sending process with failures, 24
- `PeriodicSend(u:Real, M:Type)`, 24, 52
- `PeriodicSend2(u:Real, M:Type)`, 24, 53
- point trajectory, *see* trajectory
- precondition, 22
- prefix, 10
- progressive, 70, 72
- prophecy relation, 49, 73
- prophecy variable, 49
- reachable, 30
- receptive, 72, 85
- receptiveness, 5, 71, 85
- refinement, 42
- sequence, 10
- simulation relation, 5, 37
 - backward simulation, 38, 43, 73
 - forward simulation, 38, 73
 - refinement, 42
- static type, 13
- strategy, 71, 72
- substitutivity, 56, 57, 75
- TA, *see* timed automaton
- TA with bounds, 61
- task, 61
 - lower bound, 61
 - upper bound, 61
- time axis, 12
- time interval, 12
 - closed, 12
 - left-closed, 12
 - right-closed, 12
- time-bounded channel, 23, 33, 40, 46, 49
- timed automaton (TA), 19
- timed automaton model, 19
- Timed I/O automaton (TIOA), 5, 68
- Timed Input/Output Automaton modeling
 - framework, 4
- `TimedChannel(b:Real, M:Type)`, 23, 52, 53, 69
- `Timeout(u:Real, M:Type)`, 53
- `Timeout(u:Real, M:Type)`, 25, 52
- timeout process, 25, 32
- timing-independent, 36, 78
- TIOA, *see* Timed I/O automaton
- trace, 5, 31, 69
 - `PeriodicSend(u:Real, M:Type)`, 32
 - `Timeout(u:Real, M:Type)`, 33
- trace fragment, 31, 69
- trajectory, 14, 19
 - closed, 14
 - concatenation, 15, 16
 - full, 14
 - limit time, 14
 - open, 14
 - point trajectory, 14, 16
 - prefix, 14
- Uppaal, 8
- `UseNewInputA`, 80
- `UseNewInputB`, 80
- `UseOldInputA`, 80
- `UseOldInputB`, 80
- variables, 13, 14, 19
 - analog, 13

discrete, 13
dynamic types, 13
static type, 13

Zeno, 5, 17, 34